

__init__

Python
<code>__init__(target: Executor str) -> None</code>

Parameters

 Expand table

Name	Description
target Required*	Executor str

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

target

Python
<code>target: Executor str</code>

Edge Class

Model a directed, optionally-conditional hand-off between two executors.

Each *Edge* captures the minimal metadata required to move a message from one executor to another inside the workflow graph. It optionally embeds a boolean predicate that decides if the edge should be taken at runtime. By serialising the edge down to primitives we can reconstruct the topology of a workflow irrespective of the original Python process.

Initialize a fully-specified edge between two workflow executors.

Constructor

Python
Edge()

Parameters

 Expand table

Name	Description
source_id Required*	str Canonical identifier of the upstream executor instance.
target_id Required*	str Canonical identifier of the downstream executor instance.
condition	Optional predicate that receives the message payload and returns <i>True</i> when the edge should be traversed. When omitted, the edge is considered unconditionally active. Default value: None
condition_name Required*	<xref:<xref:agent_framework.str None> > Optional override that pins a human-friendly name for the condition when the callable cannot be introspected (for example after deserialization).
Examples Required*	Python <pre>edge = Edge("fetch", "parse", condition=lambda data: data.is_valid)</pre>

Name	Description
	<pre>assert edge.source_id == "fetch" assert edge.target_id == "parse"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
condition_name	Default value: None

Examples

Python

```
edge = Edge(source_id="ingest", target_id="score", condition=lambda payload:
payload["ready"])
assert edge.should_route({"ready": True}) is True
assert edge.should_route({"ready": False}) is False
```

Methods

[Expand table](#)

<code>__init__</code>	Initialize a fully-specified edge between two workflow executors.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Reconstruct an <i>Edge</i> from its serialised dictionary form. The deserialised edge will lack the executable predicate because we do not attempt to hydrate Python callables from storage. Instead, the stored <i>condition_name</i> is preserved so that downstream consumers can detect missing callables and re-register them where appropriate.
<code>from_json</code>	
<code>should_route</code>	Evaluate the edge predicate against an incoming payload. When the edge was defined without an explicit predicate the method returns <i>True</i>, signalling an unconditional routing rule. Otherwise the user-supplied callable decides whether the

	message should proceed along this edge. Any exception raised by the callable is deliberately allowed to surface to the caller to avoid masking logic bugs.
<code>to_dict</code>	Produce a JSON-serialisable view of the edge metadata. The representation includes the source and target executor identifiers plus the condition name when it is known. Serialisation intentionally omits the live callable to keep payloads transport-friendly.
<code>to_json</code>	

`__init__`

Initialize a fully-specified edge between two workflow executors.

Python

```
__init__(source_id: str, target_id: str, condition: Callable[[Any], bool] | None = None, *, condition_name: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
source_id Required*	<code>str</code> Canonical identifier of the upstream executor instance.
target_id Required*	<code>str</code> Canonical identifier of the downstream executor instance.
condition	<code>Callable[[Any], bool] None</code> Optional predicate that receives the message payload and returns <i>True</i> when the edge should be traversed. When omitted, the edge is considered unconditionally active. Default value: None
condition_name Required*	<code>str None</code> Optional override that pins a human-friendly name for the condition when the callable cannot be introspected (for example after deserialization).
Examples Required*	Python <pre><code>edge = Edge("fetch", "parse", condition=lambda data: data.is_valid)</code></pre>

Name	Description
	<pre>assert edge.source_id == "fetch" assert edge.target_id == "parse"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
condition_name	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<pre>__new__(**kwargs)</pre>

clone

Python
<pre>clone(*, deep: bool = True) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

from_dict

Reconstruct an *Edge* from its serialised dictionary form.

The deserialised edge will lack the executable predicate because we do not attempt to hydrate Python callables from storage. Instead, the stored *condition_name* is preserved so that downstream consumers can detect missing callables and re-register them where appropriate.

Python

```
from_dict(data: dict[str, Any]) -> Edge
```

Parameters

[Expand table](#)

Name	Description
data Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
Edge	

Examples

Python

```
payload = {"source_id": "reader", "target_id": "writer", "condition_name":  
"is_ready"}  
edge = Edge.from_dict(payload)  
assert edge.source_id == "reader"  
assert edge.condition_name == "is_ready"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

should_route

Evaluate the edge predicate against an incoming payload.

When the edge was defined without an explicit predicate the method returns *True*, signalling an unconditional routing rule. Otherwise the user-supplied callable decides whether the message should proceed along this edge. Any exception raised by the callable is deliberately allowed to surface to the caller to avoid masking logic bugs.

Python

```
should_route(data: Any) -> bool
```

Parameters

[Expand table](#)

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
bool	

Examples

Python

```
edge = Edge("stage1", "stage2", condition=lambda payload: payload["score"] > 0.8)
assert edge.should_route({"score": 0.9}) is True
assert edge.should_route({"score": 0.4}) is False
```

to_dict

Produce a JSON-serialisable view of the edge metadata.

The representation includes the source and target executor identifiers plus the condition name when it is known. Serialisation intentionally omits the live callable to keep payloads transport-friendly.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Examples

Python
<pre> edge = Edge("reader", "writer", condition=lambda payload: payload["ok"]) snapshot = edge.to_dict() assert snapshot == {"source_id": "reader", "target_id": "writer", "condition_name": "<lambda>"} </pre>

to_json

Python
<pre> to_json() -> str </pre>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

id

Return the stable identifier used to reference this edge.

The identifier combines the source and target executor identifiers with a deterministic separator. This allows other graph structures such as adjacency lists or visualisations to refer to an edge without carrying the full object.

Examples

Python
<pre> edge = Edge("reader", "writer") </pre>

```
assert edge.id == "reader->writer"
```

ID_SEPARATOR

Python

```
ID_SEPARATOR: ClassVar[str] = '->'
```

condition_name

Python

```
condition_name: str | None
```

source_id

Python

```
source_id: str
```

target_id

Python

```
target_id: str
```

EdgeDuplicationError Class

Exception raised when duplicate edges are detected in the workflow.

Constructor

Python
<code>EdgeDuplicationError()</code>

Parameters

[Expand table](#)

Name	Description
<code>edge_id</code> Required*	

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Python
<code>__init__(edge_id: str)</code>

Parameters

[Expand table](#)

Name	Description
edge_id Required*	str

__new__

Python
<code>__new__(**kwargs)</code>

add_note

Exception.add_note(note) – add a note to the exception

Python
<code>add_note()</code>

with_traceback

Exception.with_traceback(tb) – set self.traceback to tb and return self.

Python
<code>with_traceback()</code>

Attributes

args

ErrorContent Class

Represents an error.

Remarks: Typically used for non-fatal errors, where something went wrong as part of the operation, but the operation was still able to continue.

Initializes an ErrorContent instance.

Constructor

Python
ErrorContent()

Keyword-Only Parameters

 Expand table

Name	Description
message	The error message. Default value: None
error_code	The error code associated with the error. Default value: None
details	Additional details about the error. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```
from agent_framework import ErrorContent

# Create an error content
error = ErrorContent(
    message="Failed to process request",
    error_code="PROCESSING_ERROR",
    details="The input format was invalid",
)
print(str(error)) # "Error PROCESSING_ERROR: Failed to process request"

# Error without code
simple_error = ErrorContent(message="Something went wrong")
print(str(simple_error)) # "Something went wrong"
```

Methods

 Expand table

<code>__init__</code>	Initializes an ErrorContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes an ErrorContent instance.

Python

```
__init__(*, message: str | None = None, error_code: str | None = None, details: str | None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
message Required*	str None
error_code Required*	str None
details Required*	str None
annotations Required*	Sequence[CitationAnnotation MutableMapping[str, Any]] None
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
message	The error message. Default value: None
error_code	The error code associated with the error. Default value: None
details	Additional details about the error. Default value: None

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

error_code

The error code associated with the error.

details

Additional details about the error.

message

The error message.

type

The type of content, which is always "error" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


Executor Class

Base class for all workflow executors that process messages and perform computations.

Overview

Executors are the fundamental building blocks of workflows, representing individual processing units that receive messages, perform operations, and produce outputs. Each executor is uniquely identified and can handle specific message types through decorated handler methods.

Type System

Executors have a rich type system that defines their capabilities:

Input Types

The types of messages an executor can process, discovered from handler method signatures:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_string(self, message: str, ctx: WorkflowContext) -> None:
        # This executor can handle 'str' input types
```

Access via the *input_types* property.

Output Types

The types of messages an executor can send to other executors via *ctx.send_message()*:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_data(self, message: str, ctx: WorkflowContext[int | bool]) ->
None:
        # This executor can send 'int' or 'bool' messages
```

Access via the *output_types* property.

Workflow Output Types

The types of data an executor can emit as workflow-level outputs via `ctx.yield_output()`:

Python

```
class MyExecutor(Executor):
    @handler
    async def process(self, message: str, ctx: WorkflowContext[int, str]) -> None:
        # Can send 'int' messages AND yield 'str' workflow outputs
```

Access via the `workflow_output_types` property.

Handler Discovery

Executors discover their capabilities through decorated methods:

@handler Decorator

Marks methods that process incoming messages:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_text(self, message: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(message.upper())
```

Sub-workflow Request Interception

Use `@handler` methods to intercept sub-workflow requests:

Python

```
class ParentExecutor(Executor):
    @handler
    async def handle_subworkflow_request(
        self,
        request: SubWorkflowRequestMessage,
        ctx: WorkflowContext[SubWorkflowResponseMessage],
    ) -> None:
        if self.is_allowed(request.domain):
            response = request.create_response(data=True)
```

```
        await ctx.send_message(response, target_id=request.executor_id)
    else:
        await ctx.request_info(request.source_event,
                                response_type=request.source_event.response_type)
```

Context Types

Handler methods receive different WorkflowContext variants based on their type annotations:

WorkflowContext (no type parameters)

For handlers that only perform side effects without sending messages or yielding outputs:

Python

```
class LoggingExecutor(Executor):
    @handler
    async def log_message(self, msg: str, ctx: WorkflowContext) -> None:
        print(f"Received: {msg}") # Only logging, no outputs
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out via *ctx.send_message()*:

Python

```
class ProcessorExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(42) # Can send int messages
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```
class DualOutputExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx: WorkflowContext[int, str]) -> None:
        await ctx.send_message(42) # Send int message
        await ctx.yield_output("done") # Yield str workflow output
```

Function Executors

Simple functions can be converted to executors using the `@executor` decorator:

Python

```
@executor
async def process_text(text: str, ctx: WorkflowContext[str]) -> None:
    await ctx.send_message(text.upper())

# Or with custom ID:
@executor(id="text_processor")
def sync_process(text: str, ctx: WorkflowContext[str]) -> None:
    ctx.send_message(text.lower()) # Sync functions run in thread pool
```

Sub-workflow Composition

Executors can contain sub-workflows using `WorkflowExecutor`. Sub-workflows can make requests that parent workflows can intercept. See `WorkflowExecutor` documentation for details on workflow composition patterns and request/response handling.

State Management

Executors can contain states that persist across workflow runs and checkpoints. Override the `on_checkpoint_save` and `on_checkpoint_restore` methods to implement custom state serialization and restoration logic.

Implementation Notes

- Do not call `execute()` directly - it's invoked by the workflow engine
- Do not override `execute()` - define handlers using decorators instead
- Each executor must have at least one `@handler` method
- Handler method signatures are validated at initialization time

Initialize the executor with a unique identifier.

Constructor

Python

Executor()

Parameters

[Expand table](#)

Name	Description
id Required*	A unique identifier for the executor.

Keyword-Only Parameters

[Expand table](#)

Name	Description
type	The executor type name. If not provided, uses class name. Default value: None
type	Alternative parameter name for executor type.
defer_discovery	If True, defer handler method discovery until later. Default value: False
**_	Additional keyword arguments. Unused in this implementation.
type_	Default value: None

Methods

[Expand table](#)

__init__	Initialize the executor with a unique identifier.
__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters. - Do not call this method directly - it is invoked by the workflow engine. - Do not override this method. Instead, define handlers using <code>@handler</code> decorator.

<code>from_dict</code>	
<code>from_json</code>	
<code>is_request_supported</code>	Check if the executor supports request of the given type and handling a response of the given type.
<code>on_checkpoint_restore</code>	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
<code>on_checkpoint_save</code>	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint. The returned state dictionary will be passed to <i>on_checkpoint_restore</i> when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.
<code>to_dict</code>	Serialize executor definition for workflow topology export.
<code>to_json</code>	

`__init__`

Initialize the executor with a unique identifier.

Python

```
__init__(id: str, *, type: str | None = None, type_: str | None = None,
defer_discovery: bool = False, **_: Any) -> None
```

Parameters

 Expand table

Name	Description
id Required*	<code>str</code> A unique identifier for the executor.
type Required*	<code>str</code> <code>None</code>
type_ Required*	<code>str</code> <code>None</code>

Name	Description
defer_discovery Required*	bool
_ Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
type	The executor type name. If not provided, uses class name. Default value: None
type	Alternative parameter name for executor type.
defer_discovery	If True, defer handler method discovery until later. Default value: False
**_	Additional keyword arguments. Unused in this implementation.
type_	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

can_handle

Check if the executor can handle a given message type.

Python

```
can_handle(message: Message) -> bool
```

Parameters

[Expand table](#)

Name	Description
message Required*	Message The message to check.

Returns

[Expand table](#)

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

[Expand table](#)

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using `@handler` decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] None Optional source span IDs from multiple sources for linking. Default value: None

Returns

[Expand table](#)

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

[Expand table](#)

Name	Description
data Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_model_utils.TModel>	

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python
<code>is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool</code>

Parameters

[Expand table](#)

Name	Description
request_type Required*	<code>type[Any]</code> The type of the request message
response_type Required*	<code>type[Any]</code> The type of the expected response message

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

 Expand table

Name	Description
state Required*	<code>dict[str, Any]</code> The state dictionary that was saved during checkpointing.

Returns

 Expand table

Type	Description
<code>None</code>	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Python
<code>to_json() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

[Expand table](#)

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via `send_message()`.

Returns

[Expand table](#)

Type	Description
	A list of the output types inferred from the handlers' <code>WorkflowContext[T]</code> annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via `yield_output()`.

Returns

[Expand table](#)

Type	Description
	A list of the workflow output types inferred from handlers' <code>WorkflowContext[T, U]</code> annotations.

ExecutorCompletedEvent Class

Event triggered when an executor handler is completed.

Initialize the executor event with an executor ID and optional data.

Constructor

Python
<code>ExecutorCompletedEvent()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code> Required*	
<code>data</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the executor event with an executor ID and optional data.
<code>__new__</code>	

`__init__`

Initialize the executor event with an executor ID and optional data.

Python
<code>__init__(executor_id: str, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
data	Any None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

ExecutorEvent Class

Base class for executor events.

Initialize the executor event with an executor ID and optional data.

Constructor

Python
<code>ExecutorEvent()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code> Required*	
<code>data</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the executor event with an executor ID and optional data.
<code>__new__</code>	

`__init__`

Initialize the executor event with an executor ID and optional data.

Python
<code>__init__(executor_id: str, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
data	Any None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

ExecutorFailedEvent Class

Event triggered when an executor handler raises an error.

Constructor

Python
<code>ExecutorFailedEvent()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code> Required*	
<code>details</code> Required*	

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__(executor_id: str, details: WorkflowErrorDetails)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
details Required*	WorkflowErrorDetails

__new__

Python
<code>__new__(**kwargs)</code>

ExecutorInvokedEvent Class

Event triggered when an executor handler is invoked.

Initialize the executor event with an executor ID and optional data.

Constructor

Python
<code>ExecutorInvokedEvent()</code>

Parameters

[Expand table](#)

Name	Description
<code>executor_id</code> Required*	
<code>data</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the executor event with an executor ID and optional data.
<code>__new__</code>	

`__init__`

Initialize the executor event with an executor ID and optional data.

Python
<code>__init__(executor_id: str, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
executor_id Required*	str
data	Any None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

FanInEdgeGroup Class

Represent a converging set of edges that feed a single downstream executor.

Fan-in groups are typically used when multiple upstream stages independently produce messages that should all arrive at the same downstream processor.

Build a fan-in mapping that merges several sources into one target.

Constructor

Python
<code>FanInEdgeGroup()</code>

Parameters

 Expand table

Name	Description
<code>source_ids</code> Required*	Sequence of upstream executor identifiers contributing messages.
<code>target_id</code> Required*	Downstream executor that receives every message emitted by the sources.
<code>id</code> Required*	<code>str</code> Optional explicit identifier for the edge group.
<code>Examples</code> Required*	Python <pre>group = FanInEdgeGroup(["parser", "enricher"], target_id="writer") assert group.to_dict()["edges"][0]["target_id"] == "writer"</pre>

Keyword-Only Parameters

 Expand table

Name	Description
<code>id</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Build a fan-in mapping that merges several sources into one target.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Hydrate the correct <i>EdgeGroup</i> subclass from serialised state. The method inspects the <i>type</i> field, allocates the corresponding class without executing subclass <i>init</i>, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
<code>from_json</code>	
<code>register</code>	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their <i>name</i>, which must therefore remain stable across versions when persisted workflows are in circulation.
<code>to_dict</code>	Serialise the group metadata and contained edges into primitives. The payload captures each edge through its own <i>to_dict</i> call, enabling round-tripping through formats such as JSON without leaking Python objects.
<code>to_json</code>	

`__init__`

Build a fan-in mapping that merges several sources into one target.

Python

```
__init__(source_ids: Sequence[str], target_id: str, *, id: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
source_ids Required*	Sequence[str] Sequence of upstream executor identifiers contributing messages.
target_id Required*	str Downstream executor that receives every message emitted by the sources.
id Required*	str None Optional explicit identifier for the edge group.
Examples Required*	Python <pre>group = FanInEdgeGroup(["parser", "enricher"], target_id="writer") assert group.to_dict()["edges"][0]["target_id"] == "writer"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	Default value: None

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

 Expand table

Name	Description
deep Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
deep	Default value: True

Returns

 Expand table

Type	Description
<code>Self</code>	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass *init*, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._edge.EdgeGroup></code>	

Examples

Python

```
payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":  
"b"}]}  
group = EdgeGroup.from_dict(payload)  
assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python
<pre>register(subclass: type[EdgeGroup]) -> type[EdgeGroup]</pre>

Parameters

 Expand table

Name	Description
subclass Required*	type [<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

 Expand table

Type	Description
type [<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python
<pre>@EdgeGroup.register class CustomGroup(EdgeGroup): pass assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup</pre>

to_dict

Serialise the group metadata and contained edges into primitives.

The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.

Python

```
to_dict() -> dict[str, Any]
```

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

Examples

Python

```
group = EdgeGroup([Edge("read", "write")])
snapshot = group.to_dict()
assert snapshot["edges"][0]["source_id"] == "read"
```

to_json

Python

```
to_json() -> str
```

Returns

 Expand table

Type	Description
<code>str</code>	

Attributes

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

type

Python

type: str

FanOutEdgeGroup Class

Represent a broadcast-style edge group with optional selection logic.

A fan-out forwards a message produced by a single source executor to one or more downstream executors. At runtime we may further narrow the targets by executing a *selection_func* that inspects the payload and returns the subset of ids that should receive the message.

Create a fan-out mapping from a single source to many targets.

Constructor

Python
FanOutEdgeGroup()

Parameters

 Expand table

Name	Description
source_id Required*	Identifier of the upstream executor broadcasting the message.
target_ids Required*	Ordered set of downstream executor identifiers that may receive the message. At least two targets are required to preserve the fan-out semantics.
selection_func	Optional callable that returns the subset of <i>target_ids</i> that should be active for a given payload. The callable receives the original message plus a copy of all configured target ids. Default value: None
selection_func_name Required*	<xref:<xref:agent_framework.str None>> Static identifier used when persisting the fan-out. Needed when the callable cannot be introspected or is unavailable during deserialisation.
id Required*	str Stable identifier for the group; defaults to an autogenerated UUID.
Examples Required*	Python <pre>def choose_targets(message: dict[str, Any], available:</pre>

Name	Description
	<pre>list[str]) -> list[str]: return [target for target in available if message.get(target)] group = FanOutEdgeGroup("sensor", ["db", "cache"], selection_func=choose_targets) assert group.selection_func is choose_targets</pre>

Keyword-Only Parameters

 Expand table

Name	Description
selection_func_name	Default value: None
id	Default value: None

Methods

 Expand table

<code>__init__</code>	Create a fan-out mapping from a single source to many targets.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Hydrate the correct <i>EdgeGroup</i> subclass from serialised state. The method inspects the <i>type</i> field, allocates the corresponding class without executing subclass <i>init</i>, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
<code>from_json</code>	
<code>register</code>	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their <i>name</i>, which must therefore remain stable across versions when persisted workflows are in circulation.
<code>to_dict</code>	Serialise the fan-out group while preserving selection metadata. In addition to the base <i>EdgeGroup</i> payload we embed the human-friendly name of the selection function. The callable itself is not persisted.

to_json

`__init__`

Create a fan-out mapping from a single source to many targets.

Python

```
__init__(source_id: str, target_ids: Sequence[str], selection_func: Callable[[Any, list[str]], list[str]] | None = None, *, selection_func_name: str | None = None, id: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
source_id Required*	str Identifier of the upstream executor broadcasting the message.
target_ids Required*	Sequence[str] Ordered set of downstream executor identifiers that may receive the message. At least two targets are required to preserve the fan-out semantics.
selection_func	Callable[[Any, list[str]], list[str]] None Optional callable that returns the subset of <i>target_ids</i> that should be active for a given payload. The callable receives the original message plus a copy of all configured target ids. Default value: None
selection_func_name Required*	str None Static identifier used when persisting the fan-out. Needed when the callable cannot be introspected or is unavailable during deserialisation.
id Required*	str None Stable identifier for the group; defaults to an autogenerated UUID.
Examples Required*	Python <pre><code>def choose_targets(message: dict[str, Any], available: list[str]) -> list[str]: return [target for target in available if message.get(target)] ...</code></pre>

Name	Description
	<pre>group = FanOutEdgeGroup("sensor", ["db", "cache"], selection_func=choose_targets) assert group.selection_func is choose_targets</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
selection_func_name	Default value: None
id	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<pre>__new__(**kwargs)</pre>

clone

Python
<pre>clone(*, deep: bool = True) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass *init*, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

[Expand table](#)

Name	Description
data Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._edge.EdgeGroup>	

Examples

Python
<pre> payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id": "b"}]} group = EdgeGroup.from_dict(payload) assert isinstance(group, EdgeGroup) </pre>

from_json

Python
<pre> from_json(raw: str) -> TModel </pre>

Parameters

[Expand table](#)

Name	Description
raw Required*	str

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

[Expand table](#)

Name	Description
subclass Required*	<code>type[<xref:agent_framework._workflows._edge.EdgeGroup>]</code>

Returns

[Expand table](#)

Type	Description
<code>type[<xref:agent_framework._workflows._edge.EdgeGroup>]</code>	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass

assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the fan-out group while preserving selection metadata.

In addition to the base *EdgeGroup* payload we embed the human-friendly name of the selection function. The callable itself is not persisted.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Examples

Python

```
group = FanOutEdgeGroup("source", ["a", "b"], selection_func=lambda *_: ["a"])
snapshot = group.to_dict()
assert snapshot["selection_func_name"] == "<lambda>"
```

to_json

Python

```
to_json() -> str
```

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

selection_func

Expose the runtime callable used to select active fan-out targets.

When no selection function was supplied the property returns *None*, signalling that all targets must receive the payload.

Examples

Python

```
group = FanOutEdgeGroup("source", ["x", "y"], selection_func=None)
```

```
assert group.selection_func is None
```

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

target_ids

Return a shallow copy of the configured downstream executor ids.

The list is defensively copied to prevent callers from mutating the internal state while still providing deterministic ordering.

Examples

Python

```
group = FanOutEdgeGroup("node", ["alpha", "beta"])
assert group.target_ids == ["alpha", "beta"]
```


edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

selection_func_name

Python

```
selection_func_name: str | None
```

type

Python

```
type: str
```

FileCheckpointStorage Class

File-based checkpoint storage for persistence.

Initialize the file storage.

Constructor

Python
<code>FileCheckpointStorage()</code>

Parameters

[⌵](#) Expand table

Name	Description
<code>storage_path</code> Required*	

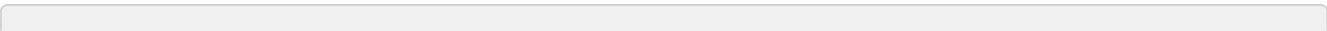
Methods

[⌵](#) Expand table

<code>__init__</code>	Initialize the file storage.
<code>__new__</code>	
<code>delete_checkpoint</code>	Delete a checkpoint by ID.
<code>list_checkpoint_ids</code>	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
<code>list_checkpoints</code>	List checkpoint objects. If workflow_id is provided, filter by that workflow.
<code>load_checkpoint</code>	Load a checkpoint by ID.
<code>save_checkpoint</code>	Save a checkpoint and return its ID.

`__init__`

Initialize the file storage.



Python

```
__init__(storage_path: str | Path)
```

Parameters

[Expand table](#)

Name	Description
storage_path Required*	<code>str</code> <code>Path</code>

__new__

Python

```
__new__(**kwargs)
```

delete_checkpoint

Delete a checkpoint by ID.

Python

```
async delete_checkpoint(checkpoint_id: str) -> bool
```

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>bool</code>	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoint_ids(workflow_id: str | None = None) -> list[str]
```

Parameters

 Expand table

Name	Description
workflow_id	str None Default value: None

Returns

 Expand table

Type	Description
list[str]	

list_checkpoints

List checkpoint objects. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoints(workflow_id: str | None = None) -> list[WorkflowCheckpoint]
```

Parameters

 Expand table

Name	Description
workflow_id	str None Default value: None

Returns

[Expand table](#)

Type	Description
<code>list[WorkflowCheckpoint]</code>	

load_checkpoint

Load a checkpoint by ID.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

[Expand table](#)

Name	Description
<code>checkpoint_id</code> Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>WorkflowCheckpoint None</code>	

save_checkpoint

Save a checkpoint and return its ID.

Python

```
async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str
```

Parameters

[Expand table](#)

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

 Expand table

Type	Description
str	

FinishReason Class

Represents the reason a chat response completed.

Initialize FinishReason with a value.

Constructor

Python
<code>FinishReason()</code>

Parameters

 Expand table

Name	Description
<code>value</code> Required*	The string representation of the finish reason.

Examples

Python
<pre>from agent_framework import FinishReason # Use predefined finish reason constants stop_reason = FinishReason.STOP # Normal completion length_reason = FinishReason.LENGTH # Max tokens reached tool_calls_reason = FinishReason.TOOL_CALLS # Tool calls triggered filter_reason = FinishReason.CONTENT_FILTER # Content filter triggered # Check finish reason if stop_reason == FinishReason.STOP: print("Response completed normally")</pre>

Methods

 Expand table

<code>__init__</code>	Initialize FinishReason with a value.
-----------------------	---------------------------------------

<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize FinishReason with a value.

Python

```
__init__(value: str) -> None
```

Parameters

 Expand table

Name	Description
<code>value</code> Required*	<code>str</code> The string representation of the finish reason.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

value

The string representation of the finish reason.

CONTENT_FILTER

Python

```
CONTENT_FILTER: FinishReason = FinishReason(value='content_filter')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python


```
INJECTABLE: ClassVar[set[str]] = {}
```

LENGTH

Python

```
LENGTH: FinishReason = FinishReason(value='length')
```

STOP

Python

```
STOP: FinishReason = FinishReason(value='stop')
```

TOOL_CALLS

Python

```
TOOL_CALLS: FinishReason = FinishReason(value='tool_calls')
```

FunctionApprovalRequestContent Class

Represents a request for user approval of a function call.

Initializes a FunctionApprovalRequestContent instance.

Constructor

Python
<code>FunctionApprovalRequestContent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>id</code>	The unique identifier for the request.
<code>function_call</code>	The function call content to be approved. Can be a FunctionCallContent object or dict.
<code>annotations</code>	Optional list of annotations for the request. Default value: None
<code>additional_properties</code>	Optional additional properties for the request. Default value: None
<code>raw_representation</code>	Optional raw representation of the request. Default value: None
<code>**kwargs</code>	Additional keyword arguments.

Examples

Python
<pre>from agent_framework import FunctionApprovalRequestContent, FunctionCallContent # Create a function approval request func_call = FunctionCallContent(call_id="call_123", name="send_email", arguments={"to": "user@example.com", "subject": "Hello"},</pre>

```

)
approval_request = FunctionApprovalRequestContent(
    id="approval_001",
    function_call=func_call,
)

# Create response
approval_response = approval_request.create_response(approved=True)
print(approval_response.approved) # True

```

Methods

 Expand table

<code>__init__</code>	Initializes a FunctionApprovalRequestContent instance.
<code>__new__</code>	
<code>create_response</code>	Create a response for the function approval request.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a FunctionApprovalRequestContent instance.

Python

```
__init__(*, id: str, function_call: FunctionCallContent | MutableMapping[str, Any], annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
id Required*	<code>str</code>
function_call Required*	<code>FunctionCallContent MutableMapping[str, Any]</code>
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
additional_properties Required*	<code>dict[str, Any] None</code>
raw_representation Required*	<code>Any None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request. Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None

Name	Description
**kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

create_response

Create a response for the function approval request.

Python
<code>create_response(approved: bool) -> FunctionApprovalResponseContent</code>

Parameters

[Expand table](#)

Name	Description
approved Required*	bool

Returns

[Expand table](#)

Type	Description
FunctionApprovalResponseContent	

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: Simple injection: <code>{"<type>": {"<parameter>": value}}</code>

Name	Description
	- Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> - Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}
```



```

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value	<code>str</code>

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

FunctionApprovalResponseContent Class

Represents a response for user approval of a function call.

Initializes a FunctionApprovalResponseContent instance.

Constructor

Python
<code>FunctionApprovalResponseContent()</code>

Parameters

 Expand table

Name	Description
approved Required*	Whether the function call was approved.

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request. Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None
**kwargs	Additional keyword arguments.

Examples

Python

```
from agent_framework import FunctionApprovalResponseContent, FunctionCallContent

# Create a function approval response
func_call = FunctionCallContent(
    call_id="call_123",
    name="send_email",
    arguments={"to": "user@example.com"},
)
response = FunctionApprovalResponseContent(
    approved=False,
    id="approval_001",
    function_call=func_call,
)
print(response.approved) # False
```

Methods

 Expand table

<code>__init__</code>	Initializes a <code>FunctionApprovalResponseContent</code> instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a FunctionApprovalResponseContent instance.

Python

```
__init__(approved: bool, *, id: str, function_call: FunctionCallContent |  
MutableMapping[str, Any], annotations: Sequence[CitationAnnotation |  
MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] |  
None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
approved Required*	<code>bool</code> Whether the function call was approved.
id Required*	<code>str</code>
function_call Required*	<code>FunctionCallContent</code> <code>MutableMapping[str, Any]</code>
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]]</code> <code>None</code>
additional_properties Required*	<code>dict[str, Any]</code> <code>None</code>
raw_representation Required*	<code>Any</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request.

Name	Description
	Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None
**kwargs	Additional keyword arguments.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```


FunctionCallContent Class

Represents a function call request.

Initializes a FunctionCallContent instance.

Constructor

Python
<code>FunctionCallContent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>call_id</code>	The function call identifier.
<code>name</code>	The name of the function requested.
<code>arguments</code>	The arguments requested to be provided to the function, can be a string to allow gradual completion of the args. Default value: None
<code>exception</code>	Any exception that occurred while mapping the original function call data to this representation. Default value: None
<code>annotations</code>	Optional annotations associated with the content. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content. Default value: None
<code>**kwargs</code>	Any additional keyword arguments.

Examples

Python

```

from agent_framework import FunctionCallContent

# Create a function call
func_call = FunctionCallContent(
    call_id="call_123",
    name="get_weather",
    arguments={"location": "Seattle", "unit": "celsius"},
)

# Parse arguments
args = func_call.parse_arguments()
print(args["location"]) # "Seattle"

# Create with string arguments (gradual completion)
func_call_partial_1 = FunctionCallContent(
    call_id="call_124",
    name="search",
    arguments='{"query": ',
)
func_call_partial_2 = FunctionCallContent(
    call_id="call_124",
    name="search",
    arguments='"latest news"',
)
full_call = func_call_partial_1 + func_call_partial_2
args = full_call.parse_arguments()
print(args["query"]) # "latest news"

```

Methods

[Expand table](#)

<code>__init__</code>	Initializes a FunctionCallContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>parse_arguments</code>	Parse the arguments into a dictionary.

	If they cannot be parsed as json or if the resulting json is not a dict, they are returned as a dictionary with a single key "raw".
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code> . All the same serialization rules apply as in <code>to_dict()</code> , including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a `FunctionCallContent` instance.

Python

```
__init__(*, call_id: str, name: str, arguments: str | dict[str, Any | None] | None = None, exception: Exception | None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
<code>call_id</code> Required*	<code>str</code>
<code>name</code> Required*	<code>str</code>
<code>arguments</code> Required*	<code>str dict[str, Any None] None</code>
<code>exception</code> Required*	<code>Exception None</code>
<code>annotations</code> Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
<code>additional_properties</code> Required*	<code>dict[str, Any] None</code>

Name	Description
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
call_id	The function call identifier.
name	The name of the function requested.
arguments	The arguments requested to be provided to the function, can be a string to allow gradual completion of the args. Default value: None
exception	Any exception that occurred while mapping the original function call data to this representation. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```

```

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",

```

```

    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

parse_arguments

Parse the arguments into a dictionary.

If they cannot be parsed as json or if the resulting json is not a dict, they are returned as a dictionary with a single key "raw".

Python

```
parse_arguments() -> dict[str, Any | None] | None
```

Returns

 Expand table

Type	Description
<code>dict[str, Any None] None</code>	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[↗](#) Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[↗](#) Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`call_id`

The function call identifier.

`name`

The name of the function requested.

`arguments`

The arguments requested to be provided to the function.

`exception`

Any exception that occurred while mapping the original function call data to this representation.

`type`

The type of content, which is always "function_call" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

FunctionExecutor Class

Executor that wraps a user-defined function.

This executor allows users to define simple functions (both sync and async) and use them as workflow executors without needing to create full executor classes.

Synchronous functions are executed in a thread pool using `asyncio.to_thread()` to avoid blocking the event loop.

Initialize the `FunctionExecutor` with a user-defined function.

Constructor

Python
<code>FunctionExecutor()</code>

Parameters

 Expand table

Name	Description
<code>func</code> Required*	The function to wrap as an executor (can be sync or async)
<code>id</code>	Optional executor ID. If None, uses the function name. Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the <code>FunctionExecutor</code> with a user-defined function.
<code>__new__</code>	
<code>can_handle</code>	Check if the executor can handle a given message type.
<code>clone</code>	
<code>execute</code>	Execute the executor with a given message and context parameters.

	• Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using <code>@handler</code> decorator.
<code>from_dict</code>	
<code>from_json</code>	
<code>is_request_supported</code>	Check if the executor supports request of the given type and handling a response of the given type.
<code>on_checkpoint_restore</code>	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
<code>on_checkpoint_save</code>	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint. The returned state dictionary will be passed to <i>on_checkpoint_restore</i> when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.
<code>to_dict</code>	Serialize executor definition for workflow topology export.
<code>to_json</code>	

`__init__`

Initialize the FunctionExecutor with a user-defined function.

Python

```
__init__(func: Callable[[...], Any], id: str | None = None)
```

Parameters

 Expand table

Name	Description
<code>func</code> Required*	<code>Callable[[...], Any]</code> The function to wrap as an executor (can be sync or async)
<code>id</code>	<code>str None</code> Optional executor ID. If None, uses the function name.

Name	Description
	Default value: None

Exceptions

[Expand table](#)

Type	Description
ValueError	If func is a staticmethod or classmethod (use @handler on instance methods instead)

`__new__`

Python
<code>__new__(**kwargs)</code>

`can_handle`

Check if the executor can handle a given message type.

Python
<code>can_handle(message: Message) -> bool</code>

Parameters

[Expand table](#)

Name	Description
message Required*	Message The message to check.

Returns

[Expand table](#)

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

[Expand table](#)

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state: SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] | None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] None Optional source span IDs from multiple sources for linking. Default value: None

Returns

[Expand table](#)

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

from_json

Python
<code>from_json(raw: str) -> TModel</code>

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

[Expand table](#)

Name	Description
request_type Required*	<code>type[Any]</code> The type of the request message
response_type Required*	<code>type[Any]</code> The type of the expected response message

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
state Required*	<code>dict[str, Any]</code> The state dictionary that was saved during checkpointing.

Returns

[Expand table](#)

Type	Description
None	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Python

```
to_json() -> str
```

Returns

[Expand table](#)

Type	Description
str	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

[Expand table](#)

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via `send_message()`.

Returns

[Expand table](#)

Type	Description
	A list of the output types inferred from the handlers' <code>WorkflowContext[T]</code> annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via `yield_output()`.

Returns

 Expand table

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

FunctionInvocationConfiguration Class

Configuration for function invocation in chat clients.

This class is created automatically on every chat client that supports function invocation. This means that for most cases you can just alter the attributes on the instance, rather than creating a new one.

Initialize FunctionInvocationConfiguration.

Constructor

Python
<code>FunctionInvocationConfiguration()</code>

Parameters

 Expand table

Name	Description
<code>enabled</code>	Whether function invocation is enabled. Default value: True
<code>max_iterations</code>	Maximum number of function invocation iterations. Default value: 40
<code>max_consecutive_errors_per_request</code>	Maximum consecutive errors allowed per request. Default value: 3
<code>terminate_on_unknown_calls</code>	Whether to terminate on unknown function calls. Default value: False
<code>additional_tools</code>	Additional tools to include for function execution. Default value: None
<code>include_detailed_errors</code>	Whether to include detailed error information in function results. Default value: False

Methods

<code>__init__</code>	Initialize FunctionInvocationConfiguration.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with '_').
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize FunctionInvocationConfiguration.

Python

```
__init__(enabled: bool = True, max_iterations: int = DEFAULT_MAX_ITERATIONS,
max_consecutive_errors_per_request: int =
DEFAULT_MAX_CONSECUTIVE_ERRORS_PER_REQUEST, terminate_on_unknown_calls: bool =
False, additional_tools: Sequence[ToolProtocol] | None = None,
include_detailed_errors: bool = False) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>enabled</code>	<code>bool</code> Whether function invocation is enabled. Default value: True
<code>max_iterations</code>	<code>int</code> Maximum number of function invocation iterations. Default value: DEFAULT_MAX_ITERATIONS
<code>max_consecutive_errors_per_request</code>	<code>int</code> Maximum consecutive errors allowed per request. Default value: DEFAULT_MAX_CONSECUTIVE_ERRORS_PER_REQUEST
<code>terminate_on_unknown_calls</code>	<code>bool</code> Whether to terminate on unknown function calls. Default value: False
<code>additional_tools</code>	<code>Sequence[ToolProtocol] None</code> Additional tools to include for function execution. Default value: None
<code>include_detailed_errors</code>	<code>bool</code> Whether to include detailed error information in function results. Default value: False

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any] None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: - Simple injection: <code>{"<type>": {"<parameter>": value}}</code> - Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> - Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code>

Name	Description
	Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

enabled

Whether function invocation is enabled. When this is set to False, the client will not attempt to invoke any functions, because the tool mode will be set to None.

max_iterations

Maximum number of function invocation iterations. Each request to this client might end up making multiple requests to the model. Each time the model responds with a function call request, this client might perform that invocation and send the results back to the model in a new request. This property limits the number of times such a roundtrip is performed. The value must be at least one, as it includes the initial request. If you want to fully disable function invocation, use the `enabled` property. The default is 40.

max_consecutive_errors_per_request

Maximum consecutive errors allowed per request. The maximum number of consecutive function call errors allowed before stopping further function calls for the request. The default is 3.

terminate_on_unknown_calls

Whether to terminate on unknown function calls. When False, call requests to any tools that aren't available to the client will result in a response message automatically being created and returned to the inner client stating that the tool couldn't be found. This behavior can help in cases where a model hallucinates a function, but it's problematic if the model has been made aware of the existence of tools outside of the normal mechanisms, and requests one of those. `additional_tools` can be used to help with that. But if instead the consumer wants to know about all function call requests that the client can't handle, this can be set to True. Upon receiving a request to call a function that the client doesn't know about, it will terminate the function calling loop and return the response, leaving the handling of the function call requests to the consumer of the client.

additional_tools

Additional tools to include for function execution. These will not impact the requests sent by the client, which will pass through the `tools` unmodified. However, if the inner client requests the invocation of a tool that was not in `ChatOptions.tools`, this `additional_tools` collection will also be consulted to look for a corresponding tool. This is useful when the service might have been pre-configured to be aware of certain tools that aren't also sent on each individual request. These tools are treated the same as `declaration_only` tools and will be returned to the user.

include_detailed_errors

Whether to include detailed error information in function results. When set to True, detailed error information such as exception type and message will be included in the function result content when a function invocation fails. When False, only a generic error message will be included.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

FunctionInvocationContext Class

Context object for function middleware invocations.

This context is passed through the function middleware pipeline and contains all information about the function invocation.

Initialize the FunctionInvocationContext.

Constructor

Python
<code>FunctionInvocationContext()</code>

Parameters

 Expand table

Name	Description
function Required*	The function being invoked.
arguments Required*	The validated arguments for the function.
metadata	Metadata dictionary for sharing data between function middleware. Default value: None
result	Function execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the chat method that invoked this function. Default value: None

Examples

Python
<pre>from agent_framework import FunctionMiddleware, FunctionInvocationContext</pre>

```

class ValidationMiddleware(FunctionMiddleware):
    async def process(self, context: FunctionInvocationContext, next):
        print(f"Function: {context.function.name}")
        print(f"Arguments: {context.arguments}")

        # Validate arguments
        if not self.validate(context.arguments):
            context.result = {"error": "Validation failed"}
            context.terminate = True
            return

        # Continue execution
        await next(context)

```

Methods

 Expand table

<code>__init__</code>	Initialize the FunctionInvocationContext.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the FunctionInvocationContext.

Python

```
__init__(function: AIFunction[Any, Any], arguments: BaseModel, metadata: dict[str, Any] | None = None, result: Any = None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

 Expand table

Name	Description
function Required*	AIFunction[Any, Any] The function being invoked.
arguments Required*	<xref:BaseModel> The validated arguments for the function.
metadata	dict[str, Any] None Metadata dictionary for sharing data between function middleware. Default value: None
result	Any Function execution result. Default value: None
terminate	bool A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	dict[str, Any] None Additional keyword arguments passed to the chat method that invoked this function. Default value: None

Returns

 Expand table

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```



```

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",

```

```

    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with `'_'`).

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	

Name	Description
	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

function

The function being invoked.

arguments

The validated arguments for the function.

metadata

Metadata dictionary for sharing data between function middleware.

result

Function execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result.

terminate

A flag indicating whether to terminate execution after current middleware. When set to True, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the chat method that invoked this function.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'arguments', 'function', 'result'}
```

FunctionMiddleware Class

Abstract base class for function middleware that can intercept function invocations.

Function middleware allows you to intercept and modify function/tool invocations before and after execution. You can validate arguments, cache results, log invocations, or override function execution.

ⓘ Note

FunctionMiddleware is an abstract base class. You must subclass it and implement the `process()` method to create custom function middleware.

Constructor

Python

```
FunctionMiddleware()
```

Examples

Python

```
from agent_framework import FunctionMiddleware, FunctionInvocationContext, ChatAgent

class CachingMiddleware(FunctionMiddleware):
    def __init__(self):
        self.cache = {}

    async def process(self, context: FunctionInvocationContext, next):
        cache_key = f"{context.function.name}:{context.arguments}"

        # Check cache
        if cache_key in self.cache:
            context.result = self.cache[cache_key]
            context.terminate = True
            return

        # Execute function
        await next(context)
```



```

        # Cache result
        if context.result:
            self.cache[cache_key] = context.result

    # Use with an agent
    agent = ChatAgent(chat_client=client, name="assistant",
middleware=CachingMiddleware())

```

Methods

[Expand table](#)

`__init__`

`__new__`

process Process a function invocation.

ⓘ Note

Middleware should not return anything. All data manipulation should happen within the context object. Set `context.result` to override execution, or observe `context.result` after calling `next()` for actual results.

`__init__`

Python

`__init__()`

`__new__`

Python

`__new__(**kwargs)`

process

Process a function invocation.

ⓘ Note

Middleware should not return anything. All data manipulation should happen within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

Python

```
abstract async process(context: FunctionInvocationContext, next: Callable[[FunctionInvocationContext], Awaitable[None]]) -> None
```

Parameters

[Expand table](#)

Name	Description
context Required*	FunctionInvocationContext Function invocation context containing function, arguments, and metadata. Middleware can set context.result to override execution, or observe the actual execution result after calling next().
next Required*	Callable [[FunctionInvocationContext], Awaitable [None]] Function to call the next middleware or final function execution. Does not return anything - all data flows through the context.

Returns

[Expand table](#)

Type	Description
None	

FunctionResultContent Class

Represents the result of a function call.

Initializes a FunctionResultContent instance.

Constructor

Python
<code>FunctionResultContent()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>call_id</code>	The identifier of the function call for which this is the result.
<code>result</code>	The result of the function call, or a generic error message if the function call failed. Default value: None
<code>exception</code>	An exception that occurred if the function call failed. Default value: None
<code>annotations</code>	Optional annotations associated with the content. Default value: None
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content. Default value: None
<code>**kwargs</code>	Any additional keyword arguments.

Examples

Python
<pre>from agent_framework import FunctionResultContent</pre>

```

# Create a successful function result
result = FunctionResultContent(
    call_id="call_123",
    result={"temperature": 22, "condition": "sunny"},
)

# Create a failed function result
failed_result = FunctionResultContent(
    call_id="call_124",
    result="Function execution failed",
    exception=ValueError("Invalid location"),
)

```

Methods

 Expand table

<code>__init__</code>	Initializes a FunctionResultContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a FunctionResultContent instance.

```
__init__(*, call_id: str, result: Any | None = None, exception: Exception | None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
call_id Required*	<code>str</code>
result Required*	<code>Any None</code>
exception Required*	<code>Exception None</code>
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
additional_properties Required*	<code>dict[str, Any] None</code>
raw_representation Required*	<code>Any None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
call_id	The identifier of the function call for which this is the result.
result	The result of the function call, or a generic error message if the function call failed. Default value: None
exception	An exception that occurred if the function call failed. Default value: None
annotations	Optional annotations associated with the content. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```



```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
<code>JSONDecodeError</code>	If the JSON string is malformed.
<code>ValueError</code>	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[↗](#) Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[↗](#) Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

call_id

The identifier of the function call for which this is the result.

result

The result of the function call, or a generic error message if the function call failed.

exception

An exception that occurred if the function call failed.

type

The type of content, which is always "function_result" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
    'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

GraphConnectivityError Class

Exception raised when graph connectivity issues are detected.

Constructor

Python
<code>GraphConnectivityError()</code>

Parameters

[Expand table](#)

Name	Description
<code>message</code> Required*	

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Python
<code>__init__(message: str)</code>

Parameters

[Expand table](#)

Name	Description
message Required*	str

`__new__`

Python
<code>__new__(**kwargs)</code>

`add_note`

`Exception.add_note(note)` – add a note to the exception

Python
<code>add_note()</code>

`with_traceback`

`Exception.with_traceback(tb)` – set `self.traceback` to `tb` and return `self`.

Python
<code>with_traceback()</code>

Attributes

`args`

GroupChatBuilder Class

High-level builder for manager-directed group chat workflows with dynamic orchestration.

GroupChat coordinates multi-agent conversations using a manager that selects which participant speaks next. The manager can be a simple Python function ([set_select_speakers_func](#)) or an agent-based selector via [set_manager](#). These two approaches are mutually exclusive.

Core Workflow:

1. Define participants: list of agents (uses their `.name`) or dict mapping names to agents
2. Configure speaker selection: [set_select_speakers_func](#) OR [set_manager](#) (not both)
3. Optional: set round limits, checkpointing, termination conditions
4. Build and run the workflow

Speaker Selection Patterns:

Pattern 1: Simple function-based selection (recommended)

Python

```
from agent_framework import GroupChatBuilder, GroupChatStateSnapshot

def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    # state contains: task, participants, conversation, history, round_index
    if state["round_index"] >= 5:
        return None # Finish
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"

workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher_agent, writer_agent]) # Uses agent.name
    .build()
)
```

Pattern 2: LLM-based selection

Python

```
from agent_framework import ChatAgent
from agent_framework.azure import AzureOpenAIChatClient
```

```

manager_agent = AzureOpenAIChatClient().create_agent(
    instructions="Coordinate the conversation and pick the next speaker.",
    name="Coordinator",
    temperature=0.3,
    seed=42,
    max_tokens=500,
)

workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent, display_name="Coordinator")
    .participants([researcher, writer]) # Or use dict: researcher=r, writer=w
    .with_max_rounds(10)
    .build()
)

```

Pattern 3: Request info for mid-conversation feedback

Python

```

from agent_framework import GroupChatBuilder

# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer, editor])
    .with_request_info(agents=[editor]) # Only pause before editor responds
    .build()
)

```

Participant Specification:

Two ways to specify participants:

- List form: `[agent1, agent2]` - uses `agent.name` attribute for participant names
- Dict form: `{name1: agent1, name2: agent2}` - explicit name control
- Keyword form: `participants(name1=agent1, name2=agent2)` - explicit name control

State Snapshot Structure:

The `GroupChatStateSnapshot` passed to `set_select_speakers_func` contains:

- `task`: `ChatMessage` - Original user task

- *participants*: dict[str, str] - Mapping of participant names to descriptions
- *conversation*: tuple[ChatMessage, ...] - Full conversation history
- *history*: tuple[GroupChatTurn, ...] - Turn-by-turn record with speaker attribution
- *round_index*: int - Number of manager selection rounds so far
- *pending_agent*: str | None - Name of agent currently processing (if any)

Important Constraints:

- Cannot combine [set_select_speakers_func](#) and [set_manager](#)
- Participant names must be unique
- When using list form, agents must have a non-empty *name* attribute

Initialize the GroupChatBuilder.

Constructor

Python
GroupChatBuilder()

Parameters

[Expand table](#)

Name	Description
<code>_orchestrator_factory</code> Required*	Internal extension point for custom orchestrator implementations. Used by Magentic. Not part of public API - subject to change.
<code>_participant_factory</code> Required*	Internal extension point for custom participant pipelines. Used by Magentic. Not part of public API - subject to change.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>_orchestrator_factory</code>	Default value: None
<code>_participant_factory</code>	Default value: None

Methods

[Expand table](#)

__init__	Initialize the GroupChatBuilder.
--------------------------	----------------------------------

build

Build and validate the group chat workflow.

Assembles the orchestrator, participants, and their interconnections into a complete workflow graph. The orchestrator delegates speaker selection to the manager, routes requests to the appropriate participants, and collects their responses to continue or complete the conversation.

Wiring pattern:

- Orchestrator receives initial input (str, ChatMessage, or list[ChatMessage])
- Orchestrator queries manager for next action (participant selection or finish)
- If participant selected: request routed directly to participant entry node
- Participant pipeline: AgentExecutor for agents or custom executor chains
- Participant response flows back to orchestrator
- Orchestrator updates state and queries manager again
- When manager returns finish directive: orchestrator yields final message and becomes idle

Usage:

Python

```
from agent_framework import GroupChatBuilder

# Execute the workflow
workflow =
GroupChatBuilder().set_manager(manager_agent).participants(
agent1=agent1, agent2=agent2).build()
    async for message in workflow.run("Solve this problem
collaboratively"):
        print(message.text)
```

participants

Define participants for this group chat workflow.

Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Provide a mapping of name → participant for explicit control, or pass a sequence and names will be inferred from the agent's name attribute (or executor id).

Usage:

Python

```
from agent_framework import GroupChatBuilder

workflow = (
GroupChatBuilder().set_manager(manager_agent).participants(
```

```
[writer_agent, reviewer_agent]).build()
)
```

set_manager

Configure the manager/coordinator agent for group chat orchestration.

The manager coordinates participants by selecting who speaks next based on conversation state and task requirements. The manager is a full workflow participant with access to all agent infrastructure (tools, context, observability).

The manager agent must produce structured output compatible with `ManagerSelectionResponse` to communicate its speaker selection decisions. Use `response_format` for reliable parsing. `GroupChatBuilder` enforces this when the manager is a `ChatAgent` and rejects incompatible response formats.

Example:

Python

```
from agent_framework import GroupChatBuilder, ChatAgent
from agent_framework.openai import OpenAIChatClient

# Coordinator agent - response_format is enforced to
ManagerSelectionResponse
coordinator = ChatAgent(
    name="Coordinator",
    description="Coordinates multi-agent collaboration",
    instructions='''
        You coordinate a team conversation. Review the
        conversation history
        and select the next participant to speak.

        When ready to finish, set finish=True and provide a
        summary in final_message.
    ''',
    chat_client=OpenAIChatClient(),
)

workflow = (
    GroupChatBuilder()
    .set_manager(coordinator,
display_name="Orchestrator")
    .participants([researcher, writer])
    .build()
)
```

⚠ Note

The manager agent's `response_format` must be `ManagerSelectionResponse` for structured output.

Custom response formats raise `ValueError` instead of being overridden.

The manager can be included in [with_request_info](#) to pause before the manager

runs, allowing human steering of orchestration decisions. If no filter is specified,

the manager is included automatically. To filter explicitly:

```
.with_request_info(agents=[manager, writer]) # Pause before manager and writer
```

set_select_speakers_func

Configure speaker selection using a pure function that examines group chat state.

This is the primary way to control orchestration flow in a GroupChat. Your selector function receives an immutable snapshot of the current conversation state and returns the name of the next participant to speak, or None to finish the conversation.

The selector function can implement any logic including:

- Simple round-robin or rule-based selection
- LLM-based decision making with custom prompts
- Conversation summarization before routing to the next agent
- Custom metadata or context passing

For advanced scenarios, return a GroupChatDirective instead of a string to include custom instructions or metadata for the next participant.

The selector function signature: `def select_next_speaker(state: GroupChatStateSnapshot) -> str | None: # state contains: task, participants, conversation, history, round_index # Return participant name to continue, or None to finish ...`

Example (simple):

Python

```
def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    if state["round_index"] >= 3:
        return None # Finish after 3 rounds
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"

workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants(researcher=researcher_agent, writer=writer_agent)
    .build()
)
```

Example (with LLM and custom instructions):

Python

```
from agent_framework import GroupChatDirective

async def llm_based_selector(state:
GroupChatStateSnapshot) -> GroupChatDirective | None:
    if state["round_index"] >= 5:
        return GroupChatDirective(finish=True)

    # Use LLM to decide next speaker and summarize
    conversation
    conversation_summary = await
    summarize_with_llm(state["conversation"])
    next_agent = await
    pick_agent_with_llm(state["participants"], state["task"])

    # Pass custom instruction to the selected agent
    return GroupChatDirective(
        agent_name=next_agent,
        instruction=f"Context summary:
{conversation_summary}",
    )

workflow =
GroupChatBuilder().set_select_speakers_func(llm_based_selector).participants(...).build()
```

⚠ Note

Cannot be combined with [set_manager](#). Choose one orchestration strategy.

with_checkpointing

Enable checkpointing for the built workflow using the provided storage.

Checkpointing allows the workflow to persist state and resume from interruption points, enabling long-running conversations and failure recovery.

Usage:

Python

```
from agent_framework import GroupChatBuilder,
MemoryCheckpointStorage

storage = MemoryCheckpointStorage()
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_checkpointing(storage)
```

```
.build()
)
```

with_max_rounds

Set a maximum number of manager rounds to prevent infinite conversations.

When the round limit is reached, the workflow automatically completes with a default completion message. Setting to None allows unlimited rounds.

Usage:

Python

```
from agent_framework import GroupChatBuilder

# Limit to 15 rounds
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_max_rounds(15)
    .build()
)

# Unlimited rounds
workflow = (

GroupChatBuilder().set_manager(manager_agent).participants(
agent1=agent1).with_max_rounds(None).build()
)
```

with_request_handler

Register an interceptor factory that creates executors for special requests.

with_request_info

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
    .participants([optimist, pragmatist, creative])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
```



```

        .participants([optimist, pragmatist, creative])
        .with_request_info(agents=[pragmatist]) # Only
        pause before pragmatist
        .build()
    )

```

with_termination_condition Define a custom termination condition for the group chat workflow.

The condition receives the full conversation (including manager and agent messages) and may be async. When it returns True, the orchestrator halts the conversation and emits a completion message authored by the manager.

Example:

Python

```

from agent_framework import ChatMessage,
GroupChatBuilder, Role

def stop_after_two_calls(conversation:
list[ChatMessage]) -> bool:
    calls = sum(1 for msg in conversation if msg.role ==
Role.ASSISTANT and msg.author_name == "specialist")
    return calls >= 2

specialist_agent = ...
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(lambda _: "specialist")
    .participants(specialist=specialist_agent)
    .with_termination_condition(stop_after_two_calls)
    .build()
)

```

`__init__`

Initialize the GroupChatBuilder.

Python

```

__init__(*, _orchestrator_factory: Callable[[GroupChatConfig], Executor] | None =
None, _participant_factory: Callable[[GroupChatParticipantSpec, GroupChatConfig],
Sequence[Executor]] | None = None) -> None

```

Parameters

[Expand table](#)

Name	Description
_orchestrator_factory Required*	Callable [[<xref:agent_framework._workflows._group_chat._GroupChatConfig>], Executor], None Internal extension point for custom orchestrator implementations. Used by Magentic. part of public API - subject to change.
_participant_factory Required*	Callable [[<xref:agent_framework._workflows._participant_utils.GroupChatParticipantSp<xref:agent_framework._workflows._group_chat._GroupChatConfig>], Sequence [Executor]] None Internal extension point for custom participant pipelines. Used by Magentic. Not part public API - subject to change.

Keyword-Only Parameters

[Expand table](#)

Name	Description
_orchestrator_factory	Default value: None
_participant_factory	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

build

Build and validate the group chat workflow.

Assembles the orchestrator, participants, and their interconnections into a complete workflow graph. The orchestrator delegates speaker selection to the manager, routes requests to the appropriate participants, and collects their responses to continue or complete the conversation.

Wiring pattern:

- Orchestrator receives initial input (str, ChatMessage, or list[ChatMessage])

- Orchestrator queries manager for next action (participant selection or finish)
- If participant selected: request routed directly to participant entry node
- Participant pipeline: AgentExecutor for agents or custom executor chains
- Participant response flows back to orchestrator
- Orchestrator updates state and queries manager again
- When manager returns finish directive: orchestrator yields final message and becomes idle

Usage:

Python

```
from agent_framework import GroupChatBuilder

# Execute the workflow
workflow =
GroupChatBuilder().set_manager(manager_agent).participants(agent1=agent1,
agent2=agent2).build()
async for message in workflow.run("Solve this problem collaboratively"):
    print(message.text)
```

Python

```
build() -> Workflow
```

Returns

 Expand table

Type	Description
Workflow	Validated Workflow instance ready for execution

Exceptions

 Expand table

Type	Description
ValueError	If manager or participants are not configured (when using default factory)

participants

Define participants for this group chat workflow.

Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances.

Provide a mapping of name → participant for explicit control, or pass a sequence and names

will be inferred from the agent's name attribute (or executor id).

Usage:

Python

```
from agent_framework import GroupChatBuilder

workflow = (
    GroupChatBuilder().set_manager(manager_agent).participants([writer_agent,
reviewer_agent]).build()
)
```

Python

```
participants(participants: Mapping[str, AgentProtocol | Executor] |
Sequence[AgentProtocol | Executor] | None = None, /, **named_participants:
AgentProtocol | Executor) -> GroupChatBuilder
```

Positional-Only Parameters

 Expand table

Name	Description
participants	Default value: None

Parameters

 Expand table

Name	Description
participants Required*	Mapping[str, AgentProtocol Executor] Sequence[AgentProtocol Executor] None Optional mapping or sequence of participant definitions
**named_participants Required*	AgentProtocol Executor Keyword arguments mapping names to agent/executor instances

Returns

 Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

Exceptions

Type	Description
<code>ValueError</code>	If participants are empty, names are duplicated, or names are empty strings

set_manager

Configure the manager/coordinator agent for group chat orchestration.

The manager coordinates participants by selecting who speaks next based on conversation state and task requirements. The manager is a full workflow participant with access to all agent infrastructure (tools, context, observability).

The manager agent must produce structured output compatible with `ManagerSelectionResponse` to communicate its speaker selection decisions. Use `response_format` for reliable parsing. `GroupChatBuilder` enforces this when the manager is a `ChatAgent` and rejects incompatible response formats.

Example:

Python

```
from agent_framework import GroupChatBuilder, ChatAgent
from agent_framework.openai import OpenAIChatClient

# Coordinator agent - response_format is enforced to ManagerSelectionResponse
coordinator = ChatAgent(
    name="Coordinator",
    description="Coordinates multi-agent collaboration",
    instructions='''
    You coordinate a team conversation. Review the conversation history
    and select the next participant to speak.

    When ready to finish, set finish=True and provide a summary in
    final_message.
    ''',
    chat_client=OpenAIChatClient(),
)

workflow = (
    GroupChatBuilder()
    .set_manager(coordinator, display_name="Orchestrator")
    .participants([researcher, writer])
    .build()
)
```

ⓘ Note

The manager agent's response_format must be ManagerSelectionResponse for structured output.

Custom response formats raise ValueError instead of being overridden.

The manager can be included in [with_request_info](#) to pause before the manager runs, allowing human steering of orchestration decisions. If no filter is specified, the manager is included automatically. To filter explicitly:

```
.with_request_info(agents=[manager, writer]) # Pause before manager and writer
```

Python

```
set_manager(manager: AgentProtocol | Executor, *, display_name: str | None = None)
-> GroupChatBuilder
```

Parameters

[Expand table](#)

Name	Description
manager Required*	AgentProtocol Executor Agent or executor responsible for speaker selection and coordination. Must return ManagerSelectionResponse or compatible dict/JSON structure.
display_name Required*	str None Optional name for manager messages in conversation history. If not provided, uses manager.name for AgentProtocol or manager.id for Executor.

Keyword-Only Parameters

[Expand table](#)

Name	Description
display_name	Default value: None

Returns

[Expand table](#)

Type	Description
GroupChatBuilder	Self for fluent chaining.

Exceptions

Type	Description
ValueError	If manager is already configured via
set_select_speakers_func	
TypeError	If manager is not AgentProtocol or Executor instance

set_select_speakers_func

Configure speaker selection using a pure function that examines group chat state.

This is the primary way to control orchestration flow in a GroupChat. Your selector function receives an immutable snapshot of the current conversation state and returns the name of the next participant to speak, or None to finish the conversation.

The selector function can implement any logic including:

- Simple round-robin or rule-based selection
- LLM-based decision making with custom prompts
- Conversation summarization before routing to the next agent
- Custom metadata or context passing

For advanced scenarios, return a GroupChatDirective instead of a string to include custom instructions or metadata for the next participant.

The selector function signature: `def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:` # state contains: task, participants, conversation, history, round_index # Return participant name to continue, or None to finish ...

Example (simple):

Python

```
def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    if state["round_index"] >= 3:
        return None # Finish after 3 rounds
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"

workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants(researcher=researcher_agent, writer=writer_agent)
```

```

        .build()
    )

```

Example (with LLM and custom instructions):

Python

```

from agent_framework import GroupChatDirective

async def llm_based_selector(state: GroupChatStateSnapshot) ->
GroupChatDirective | None:
    if state["round_index"] >= 5:
        return GroupChatDirective(finish=True)

    # Use LLM to decide next speaker and summarize conversation
    conversation_summary = await summarize_with_llm(state["conversation"])
    next_agent = await pick_agent_with_llm(state["participants"],
state["task"])

    # Pass custom instruction to the selected agent
    return GroupChatDirective(
        agent_name=next_agent,
        instruction=f"Context summary: {conversation_summary}",
    )

workflow =
GroupChatBuilder().set_select_speakers_func(llm_based_selector).participants(...).
build()

```

⚠ Note

Cannot be combined with [set_manager](#). Choose one orchestration strategy.

Python

```

set_select_speakers_func(selector: Callable[[Mapping[str, Any]], Awaitable[str |
None]] | Callable[[Mapping[str, Any]], str | None], *, display_name: str | None =
None, final_message: ChatMessage | str | Callable[[Mapping[str, Any]], Any] | None
= None) -> GroupChatBuilder

```

Parameters

[Expand table](#)

Name	Description
selector Required*	<code>Callable[[Mapping[str, Any]], Awaitable[str None]] Callable[[Mapping[str, Any]], str None]</code> Function that takes <code>GroupChatStateSnapshot</code> and returns the next speaker's name (str) to continue the conversation, or <code>None</code> to finish. May be sync or async. Can also

Name	Description
	return GroupChatDirective for advanced control (instruction, metadata).
display_name Required*	str None Optional name shown in conversation history for orchestrator messages (defaults to "manager").
final_message Required*	ChatMessage str Callable[[Mapping[str, Any]], Any] None Optional final message (or factory) emitted when selector returns None (defaults to "Conversation completed." authored by the manager).

Keyword-Only Parameters

[Expand table](#)

Name	Description
display_name	Default value: None
final_message	Default value: None

Returns

[Expand table](#)

Type	Description
GroupChatBuilder	Self for fluent chaining.

with_checkpointing

Enable checkpointing for the built workflow using the provided storage.

Checkpointing allows the workflow to persist state and resume from interruption points, enabling long-running conversations and failure recovery.

Usage:

Python

```
from agent_framework import GroupChatBuilder, MemoryCheckpointStorage

storage = MemoryCheckpointStorage()
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_checkpointing(storage)
    .build()
)
```

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> GroupChatBuilder
```

Parameters

 Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage Storage implementation for persisting workflow state

Returns

 Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

with_max_rounds

Set a maximum number of manager rounds to prevent infinite conversations.

When the round limit is reached, the workflow automatically completes with a default completion message. Setting to None allows unlimited rounds.

Usage:

Python

```
from agent_framework import GroupChatBuilder

# Limit to 15 rounds
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_max_rounds(15)
    .build()
)

# Unlimited rounds
workflow = (
    GroupChatBuilder().set_manager(manager_agent).participants(agent1=agent1).with_max_
    _rounds(None).build()
)
```

Python

```
with_max_rounds(max_rounds: int | None) -> GroupChatBuilder
```

Parameters

[Expand table](#)

Name	Description
max_rounds Required*	<code>int None</code> Maximum number of manager selection rounds, or None for unlimited

Returns

[Expand table](#)

Type	Description
<code>GroupChatBuilder</code>	Self for fluent chaining

with_request_handler

Register an interceptor factory that creates executors for special requests.

Python

```
with_request_handler(handler: Callable[[_GroupChatConfig], Executor] | Executor,  
*, condition: Callable[[Any], bool]) -> GroupChatBuilder
```

Parameters

[Expand table](#)

Name	Description
handler Required*	<code>Callable[[_GroupChatConfig], Executor] Executor</code> Callable that receives the wiring and returns an executor, or a pre-built executor
condition Required*	<code>Callable[[Any], bool]</code> Filter determining which orchestrator messages the interceptor should process

Keyword-Only Parameters

[Expand table](#)

Name	Description
condition Required*	

Returns

 Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

with_request_info

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
    .participants([optimist, pragmatist, creative])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
    .participants([optimist, pragmatist, creative])
    .with_request_info(agents=[pragmatist]) # Only pause before pragmatist
    .build()
)
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None = None) -> GroupChatBuilder
```

Parameters

[Expand table](#)

Name	Description
agents Required*	Sequence[str AgentProtocol Executor] None Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every participant.

Keyword-Only Parameters

[Expand table](#)

Name	Description
agents	Default value: None

Returns

[Expand table](#)

Type	Description
<xref:self>	The builder instance for fluent chaining.

with_termination_condition

Define a custom termination condition for the group chat workflow.

The condition receives the full conversation (including manager and agent messages) and may be async. When it returns True, the orchestrator halts the conversation and emits a completion message authored by the manager.

Example:

Python

```
from agent_framework import ChatMessage, GroupChatBuilder, Role

def stop_after_two_calls(conversation: list[ChatMessage]) -> bool:
    calls = sum(1 for msg in conversation if msg.role == Role.ASSISTANT and
msg.author_name == "specialist")
    return calls >= 2

specialist_agent = ...
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(lambda _: "specialist")
    .participants(specialist=specialist_agent)
```

```
        .with_termination_condition(stop_after_two_calls)
        .build()
    )
```

Python

```
with_termination_condition(condition: Callable[[list[ChatMessage]], bool | Awaitable[bool]]) -> GroupChatBuilder
```

Parameters

 Expand table

Name	Description
condition Required*	Callable [[list [ChatMessage]], bool Awaitable [bool]]

Returns

 Expand table

Type	Description
GroupChatBuilder	

GroupChatDirective Class

Instruction emitted by a group chat manager implementation.

Constructor

Python
<code>GroupChatDirective()</code>

Parameters

[Expand table](#)

Name	Description
<code>agent_name</code>	Default value: None
<code>instruction</code>	Default value: None
<code>metadata</code>	Default value: None
<code>finish</code>	Default value: False
<code>final_message</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<pre>__init__(agent_name: str None = None, instruction: str None = None, metadata: dict[str, Any] None = None, finish: bool = False, final_message: ChatMessage None = None) -> None</pre>

Parameters

[Expand table](#)

Name	Description
<code>agent_name</code>	<code>str</code> <code>None</code> Default value: <code>None</code>
<code>instruction</code>	<code>str</code> <code>None</code> Default value: <code>None</code>
<code>metadata</code>	<code>dict[str, Any]</code> <code>None</code> Default value: <code>None</code>
<code>finish</code>	<code>bool</code> Default value: <code>False</code>
<code>final_message</code>	<code>ChatMessage</code> <code>None</code> Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

Attributes

`agent_name`

Python
<code>agent_name: str</code> <code>None</code> = <code>None</code>

`final_message`

Python

```
final_message: ChatMessage | None = None
```

finish

Python

```
finish: bool = False
```

instruction

Python

```
instruction: str | None = None
```

metadata

Python

```
metadata: dict[str, Any] | None = None
```

HandoffBuilder Class

Fluent builder for conversational handoff workflows with coordinator and specialist agents.

The handoff pattern enables a coordinator agent to route requests to specialist agents.

Interaction mode controls whether the workflow requests user input after each agent response or completes autonomously once agents finish responding. A termination condition determines when the workflow should stop requesting input and complete.

Routing Patterns:

Single-Tier (Default): Only the coordinator can hand off to specialists. By default, after any specialist responds, control returns to the user for more input. This creates a cyclical flow: user -> coordinator -> [optional specialist] -> user -> coordinator -> ... Use `with_interaction_mode("autonomous")` to skip requesting additional user input and yield the final conversation when an agent responds without delegating.

Multi-Tier (Advanced): Specialists can hand off to other specialists using `.add_handoff()`. This provides more flexibility for complex workflows but is less controllable than the single-tier pattern. Users lose real-time visibility into intermediate steps during specialist-to-specialist handoffs (though the full conversation history including all handoffs is preserved and can be inspected afterward).

Key Features:

- **Automatic handoff detection:** The coordinator invokes a handoff tool whose arguments (for example `{"handoff_to": "shipping_agent"}`) identify the specialist to receive control.
- **Auto-generated tools:** By default the builder synthesizes `handoff_to_<agent>` tools for the coordinator, so you don't manually define placeholder functions.
- **Full conversation history:** The entire conversation (including any `ChatMessage.additional_properties`) is preserved and passed to each agent.
- **Termination control:** By default, terminates after 10 user messages. Override with `.with_termination_condition(lambda conv: ...)` for custom logic (e.g., detect "goodbye").
- **Interaction modes:** Choose `human_in_loop` (default) to prompt users between agent turns, or `autonomous` to continue routing back to agents without prompting for user input until a handoff occurs or a termination/turn limit is reached (default autonomous turn limit: 50).

- **Checkpointing:** Optional persistence for resumable workflows.

Usage (Single-Tier):

Python

```
from agent_framework import HandoffBuilder
from agent_framework.openai import OpenAIChatClient

chat_client = OpenAIChatClient()

# Create coordinator and specialist agents
coordinator = chat_client.create_agent(
    instructions=(
        "You are a frontline support agent. Assess the user's issue and decide "
        "whether to hand off to 'refund_agent' or 'shipping_agent'. When delegation
is "
        "required, call the matching handoff tool (for example
`handoff_to_refund_agent`)."
    ),
    name="coordinator_agent",
)

refund = chat_client.create_agent(
    instructions="You handle refund requests. Ask for order details and process
refunds.",
    name="refund_agent",
)

shipping = chat_client.create_agent(
    instructions="You resolve shipping issues. Track packages and update delivery
status.",
    name="shipping_agent",
)

# Build the handoff workflow - default single-tier routing
workflow = (
    HandoffBuilder(
        name="customer_support",
        participants=[coordinator, refund, shipping],
    )
    .set_coordinator(coordinator)
    .build()
)

# Run the workflow
events = await workflow.run_stream("My package hasn't arrived yet")
async for event in events:
    if isinstance(event, RequestInfoEvent):
        # Request user input
        user_response = input("You: ")
        await workflow.send_response(event.data.request_id, user_response)
```

Multi-Tier Routing with .add_handoff():

Python

```
# Enable specialist-to-specialist handoffs with fluent API
workflow = (
    HandoffBuilder(participants=[coordinator, replacement, delivery, billing])
        .set_coordinator(coordinator)
        .add_handoff(coordinator, [replacement, delivery, billing]) # Coordinator
# routes to all
        .add_handoff(replacement, [delivery, billing]) # Replacement delegates to
# delivery/billing
        .add_handoff(delivery, billing) # Delivery escalates to billing
        .build()
)

# Flow: User → Coordinator → Replacement → Delivery → Back to User
# (Replacement hands off to Delivery without returning to user)
```

Use Participant Factories for State Isolation:

Custom Termination Condition:

Python

```
# Terminate when user says goodbye or after 5 exchanges
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
        .set_coordinator(coordinator)
        .with_termination_condition(
            lambda conv: (
                sum(1 for msg in conv if msg.role.value == "user") >= 5
                or any("goodbye" in msg.text.lower() for msg in conv[-2:])
            )
        )
        .build()
)
```

Checkpointing:

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
```

```
.set_coordinator(coordinator)
.with_checkpointing(storage)
.build()
)
```

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. `.participants([...])` - Register agents
2. or `.participant_factories({...})` - Register agent/executor factories
3. `.set_coordinator(...)` - Designate which agent receives initial user input
4. `.build()` - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

ⓘ Note

Participants must have stable names/ids because the workflow maps the handoff tool arguments to these identifiers. Agent names should match the strings emitted by the coordinator's handoff tool (e.g., a tool that outputs `{"handoff_to": "billing"}` requires an agent named `billing`).

Constructor

Python

`HandoffBuilder()`

Parameters

[Expand table](#)

Name	Description
name Required*	Optional workflow name for identification and logging.
participants	

Name	Description
Required*	List of agents (AgentProtocol) or executors to participate in the handoff. The first agent you specify as coordinator becomes the orchestrating agent.
participant_factories Required*	Mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
description Required*	Optional human-readable description of the workflow.
name Required*	Optional workflow identifier used in logging and debugging. If not provided, a default name will be generated.
participants Required*	Optional list of agents (AgentProtocol) or executors that will participate in the handoff workflow. You can also call <i>.participants([...])</i> later. Each participant must have a unique identifier (name for agents, id for executors).
participant_factories Required*	Optional mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
description Required*	Optional human-readable description explaining the workflow's purpose. Useful for documentation and observability.

Keyword-Only Parameters

 Expand table

Name	Description
name	Default value: None
participants	Default value: None
participant_factories	Default value: None
description	Default value: None

Methods

 Expand table

<u>__init__</u>	Initialize a HandoffBuilder for creating conversational handoff workflows. The builder starts in an unconfigured state and requires you to call: 1. <i>.participants([...])</i> - Register agents
---------------------------------	---

2. or `.participant_factories({...})` - Register agent/executor factories
3. `.set_coordinator(...)` - Designate which agent receives initial user input
4. `.build()` - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

ⓘ Note

Participants must have stable names/ids because the workflow maps the

handoff tool arguments to these identifiers. Agent names should match

the strings emitted by the coordinator's handoff tool (e.g., a tool that

outputs `{"handoff_to": "billing"}` requires an agent named billing).

[__new__](#)

[add_handoff](#)

Add handoff routing from a source agent to one or more target agents.

This method enables specialist-to-specialist handoffs by configuring which agents can hand off to which others. Call this method multiple times to build a complete routing graph. By default, only the starting agent can hand off to all other participants; use this method to enable additional routing paths.

ⓘ Note

Handoff tools are automatically registered for each source agent

If a source agent is configured multiple times via `add_handoff`, targets are merged

[build](#)

Construct the final Workflow instance from the configured builder.

This method validates the configuration and assembles all internal components:

- Input normalization executor
- Starting agent executor
- Handoff coordinator
- Specialist agent executors
- User input gateway

- Request/response handling

Example (Minimal):

Python

```
workflow = (  
    HandoffBuilder(participants=[coordinator, refund,  
billing]).set_coordinator("coordinator").build()  
)  
  
# Run the workflow  
async for event in workflow.run_stream("I need help"):  
    # Handle events...  
    pass
```

Example (Full Configuration):

Python

```
from agent_framework import InMemoryCheckpointStorage  
  
storage = InMemoryCheckpointStorage()  
workflow = (  
    HandoffBuilder(  
        name="support_workflow",  
        participants=[coordinator, refund, billing],  
        description="Customer support with specialist  
routing",  
    )  
    .set_coordinator("coordinator")  
    .with_termination_condition(lambda conv: len(conv) >  
20)  
    .request_prompt("How can we help?")  
    .with_checkpointing(storage)  
    .build()  
)
```

ⓘ Note

After calling build(), the builder instance should not be reused. Create a

new builder if you need to construct another workflow with different configuration.

When enabled, after a specialist responds without requesting another handoff, user input routes directly back to that same specialist instead of always routing back to the coordinator agent for re-evaluation.

This is useful when a specialist needs multiple turns with the user to gather information or resolve an issue, avoiding unnecessary coordinator involvement while maintaining context.

Flow Comparison:

Default (disabled): User -> Coordinator -> Specialist -> User -> Coordinator -> Specialist -> ...

With return_to_previous (enabled): User -> Coordinator -> Specialist -> User -> Specialist -> ...

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[triage,
technical_support, billing])
    .set_coordinator("trriage")
    .add_handoff(triage, [technical_support, billing])
    .enable_return_to_previous() # Enable direct return
routing
    .build()
)

# Flow: User asks question
# -> Triage routes to Technical Support
# -> Technical Support asks clarifying question
# -> User provides more info
# -> Routes back to Technical Support (not Triage)
# -> Technical Support continues helping
```

Multi-tier handoff example:

Python

```
workflow = (
    HandoffBuilder(participants=[trriage, specialist_a,
specialist_b])
    .set_coordinator("trriage")
    .add_handoff(trriage, [specialist_a, specialist_b])
    .add_handoff(specialist_a, specialist_b)
    .enable_return_to_previous()
    .build()
)
```

```
# Flow: User asks question
# -> Triage routes to Specialist A
# -> Specialist A hands off to Specialist B
# -> Specialist B asks clarifying question
# -> User provides more info
# -> Routes back to Specialist B (who is currently
handling the conversation)
```

ⓘ Note

This feature routes to whichever agent most recently responded, whether that's

the coordinator or a specialist. The conversation continues with that agent until

they either hand off to another agent or the termination condition is met.

participant_factories

Register factories that produce agents or executors for the handoff workflow.

Each factory is a callable that returns an AgentProtocol or Executor instance. Factories are invoked when building the workflow, allowing for lazy instantiation and state isolation per workflow instance.

Example: .. code-block:: python

```
from agent_framework import ChatAgent, HandoffBuilder
```

```
def create_coordinator() -> ChatAgent: return ...
```

```
def create_refund_agent() -> ChatAgent: return ...
```

```
def create_billing_agent() -> ChatAgent: return ...
```

```
factories = { "coordinator": create_coordinator, "refund": create_refund_agent,
"billing": create_billing_agent,
```

```
}
```

```
builder = HandoffBuilder().participant_factories(factories)
```

Use the factory IDs to create handoffs and set the coordinator

```
builder.add_handoff("coordinator", ["refund", "billing"])
builder.set_coordinator("coordinator")
```

participants

Register the agents or executors that will participate in the handoff workflow.

Each participant must have a unique identifier (name for agents, id for executors). The workflow will automatically create an alias map so agents can be referenced by their name, display_name, or executor id when routing.

Example:

Python

```
from agent_framework import HandoffBuilder
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient()
coordinator = client.create_agent(instructions="...",
name="coordinator")
refund = client.create_agent(instructions="...",
name="refund_agent")
billing = client.create_agent(instructions="...",
name="billing_agent")

builder = HandoffBuilder().participants([coordinator,
refund, billing])
# Now you can call .set_coordinator() to designate the
entry point
```

ⓘ Note

This method resets any previously configured coordinator, so you must call

`.set_coordinator(...)` again after changing participants.

request_prompt

Set a custom prompt message displayed when requesting user input.

By default, the workflow uses a generic prompt: "Provide your next input for the conversation." Use this method to customize the message shown to users when the workflow needs their response.

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[triage, refund,
```

```

billing])
    .set_coordinator("triage")
    .request_prompt("How can we help you today?")
    .build()
)

# For more context-aware prompts, you can access the
prompt via
# RequestInfoEvent.data.prompt in your event handling
loop

```

ⓘ Note

The prompt is static and set once during workflow construction. If you need

dynamic prompts based on conversation state, you'll need to handle that in

your application's event processing logic.

set_coordinator

Designate which agent receives initial user input and orchestrates specialist routing.

The coordinator agent is responsible for analyzing user requests and deciding whether to:

1. Handle the request directly and respond to the user, OR
2. Hand off to a specialist agent by including handoff metadata in the response

After a specialist responds, the workflow automatically returns control to the user (default) creating a cyclical flow: user -> coordinator -> [specialist] -> user -> ... Configure *with_interaction_mode("autonomous")* to continue with the responding agent without requesting another user turn until a handoff occurs or a termination/turn limit is met.

Example:

Python

```

# Use factory name with `.participant_factories()`
builder = (
    HandoffBuilder()
    .participant_factories({
        "coordinator": create_coordinator,
        "refund": create_refund_agent,
        "billing": create_billing_agent,
    })
)

```

```
.set_coordinator("coordinator")
)

# Or pass the agent object directly
builder = HandoffBuilder().participants([coordinator,
refund, billing]).set_coordinator(coordinator)
```

ⓘ Note

The coordinator determines routing by invoking a handoff tool call whose

arguments identify the target specialist (for example {"handoff_to": "billing"}).

Decorate the tool with `approval_mode="always_require"` to ensure the workflow

intercepts the call before execution and can make the transition.

with_checkpointing

Enable workflow state persistence for resumable conversations.

Checkpointing allows the workflow to save its state at key points, enabling you to:

- Resume conversations after application restarts
- Implement long-running support tickets that span multiple sessions
- Recover from failures without losing conversation context
- Audit and replay conversation history

Example (In-Memory):

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[triage, refund,
billing])
    .set_coordinator("triage")
    .with_checkpointing(storage)
    .build()
)

# Run workflow with a session ID for resumption
async for event in workflow.run_stream("Help me",
session_id="user_123"):
    # Process events...
```

```
pass

# Later, resume the same conversation
async for event in workflow.run_stream("I need a
refund", session_id="user_123"):
    # Conversation continues from where it left off
    pass
```

Use Cases:

- Customer support systems with persistent ticket history
- Multi-day conversations that need to survive server restarts
- Compliance requirements for conversation auditing
- A/B testing different agent configurations on same conversation

ⓘ Note

Checkpointing adds overhead for serialization and storage I/O.
Use it when

persistence is required, not for simple stateless request-response patterns.

[with_interaction_mode](#)

Choose whether the workflow requests user input or runs autonomously after agent replies.

In autonomous mode, agents (including specialists) continue iterating on their task until they explicitly invoke a handoff tool or the turn limit is reached. This allows specialists to perform long-running autonomous tasks (e.g., research, coding, analysis) without prematurely returning control to the coordinator or user.

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[coordinator,
research_agent])
    .set_coordinator(coordinator)
    .add_handoff(coordinator, research_agent)
    .add_handoff(research_agent, coordinator)
    .with_interaction_mode("autonomous",
autonomous_turn_limit=20)
    .build()
)

# Flow: User asks a question
# -> Coordinator routes to Research Agent
```

```
# -> Research Agent iterates (researches, analyzes,
refines)
# -> Research Agent calls handoff_to_coordinator when
done
# -> Coordinator provides final response
```

`with_request_info`

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a `RequestInfoEvent` that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard `response_handler/request_info` pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    HandoffBuilder(participants=[coordinator, refund,
shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info()
    .build()
)

# Pause only before specialist agents (not coordinator)
workflow = (
    HandoffBuilder(participants=[coordinator, refund,
shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info(agents=[refund, shipping])
    .build()
)
```

`with_termination_condition`

Set a custom termination condition for the handoff workflow.

The condition can be either synchronous or asynchronous.

Example:

Python

```
# Synchronous condition
builder.with_termination_condition(
    lambda conv: len(conv) > 20 or any("goodbye" in
msg.text.lower() for msg in conv[-2:])
)
```

```
# Asynchronous condition
async def check_termination(conv: list[ChatMessage]) ->
bool:
    # Can perform async operations
    return len(conv) > 20

builder.with_termination_condition(check_termination)
```

`__init__`

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. `.participants([...])` - Register agents
2. or `.participant_factories({...})` - Register agent/executor factories
3. `.set_coordinator(...)` - Designate which agent receives initial user input
4. `.build()` - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

⚠ Note

Participants must have stable names/ids because the workflow maps the handoff tool arguments to these identifiers. Agent names should match the strings emitted by the coordinator's handoff tool (e.g., a tool that outputs `{"handoff_to": "billing"}` requires an agent named `billing`).

Python

```
__init__(*, name: str | None = None, participants: Sequence[AgentProtocol |
Executor] | None = None, participant_factories: Mapping[str, Callable[[
AgentProtocol | Executor]] | None = None, description: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
name Required*	<code>str</code> <code>None</code> Optional workflow identifier used in logging and debugging. If not provided, a default name will be generated.
participants Required*	<code>Sequence[AgentProtocol Executor]</code> <code>None</code> Optional list of agents (<code>AgentProtocol</code>) or executors that will participate in the handoff workflow. You can also call <code>.participants(...)</code> later. Each participant must have a unique identifier (name for agents, id for executors).
participant_factories Required*	<code>Mapping[str, Callable[[], AgentProtocol Executor]]</code> <code>None</code> Optional mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
description Required*	<code>str</code> <code>None</code> Optional human-readable description explaining the workflow's purpose. Useful for documentation and observability.

Keyword-Only Parameters

 Expand table

Name	Description
name	Default value: <code>None</code>
participants	Default value: <code>None</code>
participant_factories	Default value: <code>None</code>
description	Default value: <code>None</code>

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

add_handoff

Add handoff routing from a source agent to one or more target agents.

This method enables specialist-to-specialist handoffs by configuring which agents can hand off to which others. Call this method multiple times to build a complete routing graph. By default, only the starting agent can hand off to all other participants; use this method to enable additional routing paths.

ⓘ Note

Handoff tools are automatically registered for each source agent

If a source agent is configured multiple times via `add_handoff`, targets are merged

Python

```
add_handoff(source: str | AgentProtocol | Executor, targets: str | AgentProtocol | Executor | Sequence[str | AgentProtocol | Executor], *, tool_name: str | None = None, tool_description: str | None = None) -> HandoffBuilder
```

Parameters

 Expand table

Name	Description
source Required*	str AgentProtocol Executor The agent that can initiate the handoff. Can be: - Factory name (str): If using participant factories - AgentProtocol instance: The actual agent object - Executor instance: A custom executor wrapping an agent - Cannot mix factory names and instances across source and targets
targets Required*	str AgentProtocol Executor Sequence [str AgentProtocol Executor] One or more target agents that the source can hand off to. Can be: - Factory name (str): If using participant factories - AgentProtocol instance: The actual agent object - Executor instance: A custom executor wrapping an agent - Single target: "billing_agent" or agent_instance

Name	Description
	- Multiple targets: ["billing_agent", "support_agent"] or [agent1, agent2] - Cannot mix factory names and instances across source and targets
tool_name Required*	str None Optional custom name for the handoff tool. Currently not used in the implementation - tools are always auto-generated as "handoff_to_<target>". Reserved for future enhancement.
tool_description Required*	str None Optional custom description for the handoff tool. Currently not used in the implementation - descriptions are always auto-generated as "Handoff to the <target> agent.". Reserved for future enhancement.

Keyword-Only Parameters

[Expand table](#)

Name	Description
tool_name	Default value: None
tool_description	Default value: None

Returns

[Expand table](#)

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

[Expand table](#)

Type	Description
ValueError	If source or targets are not in the participants list, or if participants(...) hasn't been called yet. 2) If source or targets are factory names (str) but participant_factories(...) hasn't been called yet, or if they are not in the participant_factories list.
TypeError	If mixing factory names (str) and AgentProtocol/Executor instances

Examples

Single target (using factory name):

Python

```
builder.add_handoff("triage_agent", "billing_agent")
```

Multiple targets (using factory names):

Python

```
builder.add_handoff("triage_agent", ["billing_agent", "support_agent",  
"escalation_agent"])
```

Multiple targets (using agent instances):

Python

```
builder.add_handoff(triage, [billing, support, escalation])
```

Chain multiple configurations:

Python

```
workflow = (  
    HandoffBuilder(participants=[triage, replacement, delivery, billing])  
        .set_coordinator(triage)  
        .add_handoff(triage, [replacement, delivery, billing])  
        .add_handoff(replacement, [delivery, billing])  
        .add_handoff(delivery, billing)  
        .build()  
)
```

build

Construct the final Workflow instance from the configured builder.

This method validates the configuration and assembles all internal components:

- Input normalization executor
- Starting agent executor
- Handoff coordinator

- Specialist agent executors
- User input gateway
- Request/response handling

Example (Minimal):

Python

```
workflow = (
    HandoffBuilder(participants=[coordinator, refund,
billing]).set_coordinator("coordinator").build()
)

# Run the workflow
async for event in workflow.run_stream("I need help"):
    # Handle events...
    pass
```

Example (Full Configuration):

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(
        name="support_workflow",
        participants=[coordinator, refund, billing],
        description="Customer support with specialist routing",
    )
    .set_coordinator("coordinator")
    .with_termination_condition(lambda conv: len(conv) > 20)
    .request_prompt("How can we help?")
    .with_checkpointing(storage)
    .build()
)
```

ⓘ Note

After calling build(), the builder instance should not be reused. Create a new builder if you need to construct another workflow with different configuration.

Python

```
build() -> Workflow
```

Returns

[Expand table](#)

Type	Description
Workflow	A fully configured Workflow ready to execute via <code>.run()</code> or <code>.run_stream()</code> .

Exceptions

[Expand table](#)

Type	Description
ValueError	If participants or coordinator were not configured, or if required configuration is invalid.

enable_return_to_previous

Enable direct return to the current agent after user input, bypassing the coordinator.

When enabled, after a specialist responds without requesting another handoff, user input routes directly back to that same specialist instead of always routing back to the coordinator agent for re-evaluation.

This is useful when a specialist needs multiple turns with the user to gather information or resolve an issue, avoiding unnecessary coordinator involvement while maintaining context.

Flow Comparison:

Default (disabled): User -> Coordinator -> Specialist -> User -> Coordinator -> Specialist -> ...

With `return_to_previous` (enabled): User -> Coordinator -> Specialist -> User -> Specialist -> ...

Example:

Python

```
workflow = (  
    HandoffBuilder(participants=[triage, technical_support, billing])
```

```

        .set_coordinator("triage")
        .add_handoff(triage, [technical_support, billing])
        .enable_return_to_previous() # Enable direct return routing
        .build()
    )

    # Flow: User asks question
    # -> Triage routes to Technical Support
    # -> Technical Support asks clarifying question
    # -> User provides more info
    # -> Routes back to Technical Support (not Triage)
    # -> Technical Support continues helping

```

Multi-tier handoff example:

Python

```

workflow = (
    HandoffBuilder(participants=[triage, specialist_a, specialist_b])
        .set_coordinator("triage")
        .add_handoff(triage, [specialist_a, specialist_b])
        .add_handoff(specialist_a, specialist_b)
        .enable_return_to_previous()
        .build()
)

# Flow: User asks question
# -> Triage routes to Specialist A
# -> Specialist A hands off to Specialist B
# -> Specialist B asks clarifying question
# -> User provides more info
# -> Routes back to Specialist B (who is currently handling the conversation)

```

⚠ Note

This feature routes to whichever agent most recently responded, whether that's the coordinator or a specialist. The conversation continues with that agent until they either hand off to another agent or the termination condition is met.

Python

```
enable_return_to_previous(enabled: bool = True) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
enabled	<code>bool</code> Whether to enable return-to-previous routing. Default is True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>HandoffBuilder</code>	Self for method chaining.

participant_factories

Register factories that produce agents or executors for the handoff workflow.

Each factory is a callable that returns an `AgentProtocol` or `Executor` instance. Factories are invoked when building the workflow, allowing for lazy instantiation and state isolation per workflow instance.

Example: .. code-block:: python

```
from agent_framework import ChatAgent, HandoffBuilder
```

```
def create_coordinator() -> ChatAgent: return ...
```

```
def create_refund_agent() -> ChatAgent: return ...
```

```
def create_billing_agent() -> ChatAgent: return ...
```

```
factories = { "coordinator": create_coordinator, "refund": create_refund_agent, "billing":  
create_billing_agent,
```

```
}
```

```
bundler = HandoffBuilder().participant_factories(factories)
```

Use the factory IDs to create handoffs and set the coordinator


```
builder.add_handoff("coordinator", ["refund", "billing"]) builder.set_coordinator("coordinator")
```

Python

```
participant_factories(participant_factories: Mapping[str, Callable[[], AgentProtocol | Executor]]) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
participant_factories Required*	Mapping [str , Callable [], AgentProtocol Executor] Mapping of factory names to callables that return AgentProtocol or Executor instances. Each produced participant must have a unique identifier (name for agents, id for executors).

Returns

[Expand table](#)

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

[Expand table](#)

Type	Description
ValueError	If participant_factories is empty or <i>.participants(...)</i> or <i>.participant_factories(...)</i> has already been called.

participants

Register the agents or executors that will participate in the handoff workflow.

Each participant must have a unique identifier (name for agents, id for executors). The workflow will automatically create an alias map so agents can be referenced by their name, display_name, or executor id when routing.

Example:

Python

```
from agent_framework import HandoffBuilder
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient()
coordinator = client.create_agent(instructions="...", name="coordinator")
refund = client.create_agent(instructions="...", name="refund_agent")
billing = client.create_agent(instructions="...", name="billing_agent")

builder = HandoffBuilder().participants([coordinator, refund, billing])
# Now you can call .set_coordinator() to designate the entry point
```

ⓘ Note

This method resets any previously configured coordinator, so you must call `.set_coordinator(...)` again after changing participants.

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) -> HandoffBuilder
```

Parameters

[🔗](#) Expand table

Name	Description
participants Required*	Sequence[AgentProtocol Executor] Sequence of AgentProtocol or Executor instances. Each must have a unique identifier. For agents, the name attribute is used as the primary identifier and must match handoff target strings.

Returns

[🔗](#) Expand table

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

Type	Description
ValueError	If participants is empty, contains duplicates, or <code>.participants(...)</code> or <code>.participant_factories(...)</code> has already been called.
TypeError	If participants are not AgentProtocol or Executor instances.

request_prompt

Set a custom prompt message displayed when requesting user input.

By default, the workflow uses a generic prompt: "Provide your next input for the conversation." Use this method to customize the message shown to users when the workflow needs their response.

Example:

Python

```
workflow = (  
    HandoffBuilder(participants=[triage, refund, billing])  
    .set_coordinator("triage")  
    .request_prompt("How can we help you today?")  
    .build()  
)  
  
# For more context-aware prompts, you can access the prompt via  
# RequestInfoEvent.data.prompt in your event handling loop
```

ⓘ Note

The prompt is static and set once during workflow construction. If you need dynamic prompts based on conversation state, you'll need to handle that in your application's event processing logic.

Python

```
request_prompt(prompt: str | None) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
prompt Required*	<code>str</code> <code>None</code> Custom prompt text to display, or None to use the default prompt.

Returns

[Expand table](#)

Type	Description
<code>HandoffBuilder</code>	Self for method chaining.

set_coordinator

Designate which agent receives initial user input and orchestrates specialist routing.

The coordinator agent is responsible for analyzing user requests and deciding whether to:

1. Handle the request directly and respond to the user, OR
2. Hand off to a specialist agent by including handoff metadata in the response

After a specialist responds, the workflow automatically returns control to the user (default) creating a cyclical flow: user -> coordinator -> [specialist] -> user -> ... Configure *with_interaction_mode("autonomous")* to continue with the responding agent without requesting another user turn until a handoff occurs or a termination/turn limit is met.

Example:

Python

```
# Use factory name with `.participant_factories()`
builder = (
    HandoffBuilder()
    .participant_factories({
        "coordinator": create_coordinator,
        "refund": create_refund_agent,
        "billing": create_billing_agent,
    })
    .set_coordinator("coordinator")
)

# Or pass the agent object directly
```

```
builder = HandoffBuilder().participants([coordinator, refund,
billing]).set_coordinator(coordinator)
```

⚠ Note

The coordinator determines routing by invoking a handoff tool call whose arguments identify the target specialist (for example {"handoff_to": "billing"}).

Decorate the tool with approval_mode="always_require" to ensure the workflow intercepts the call before execution and can make the transition.

Python

```
set_coordinator(agent: str | AgentProtocol | Executor) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
agent Required*	str AgentProtocol Executor The agent to use as the coordinator. Can be: - Factory name (str): If using participant factories - AgentProtocol instance: The actual agent object - Executor instance: A custom executor wrapping an agent

Returns

[Expand table](#)

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If <i>agent</i> is an AgentProtocol or Executor instance but <i>.participants(...)</i> hasn't been called yet, or if it is not in the participants list. 2) If <i>agent</i> is a factory name (str) but <i>.participant_factories(...)</i> hasn't been called yet, or if it is not in the participant_factories list.
<code>TypeError</code>	If <i>agent</i> is not a str, AgentProtocol, or Executor instance.

with_checkpointing

Enable workflow state persistence for resumable conversations.

Checkpointing allows the workflow to save its state at key points, enabling you to:

- Resume conversations after application restarts
- Implement long-running support tickets that span multiple sessions
- Recover from failures without losing conversation context
- Audit and replay conversation history

Example (In-Memory):

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[triage, refund, billing])
    .set_coordinator("triage")
    .with_checkpointing(storage)
    .build()
)

# Run workflow with a session ID for resumption
async for event in workflow.run_stream("Help me", session_id="user_123"):
    # Process events...
    pass

# Later, resume the same conversation
async for event in workflow.run_stream("I need a refund",
session_id="user_123"):
    # Conversation continues from where it left off
    pass
```

Use Cases:

- Customer support systems with persistent ticket history

- Multi-day conversations that need to survive server restarts
- Compliance requirements for conversation auditing
- A/B testing different agent configurations on same conversation

⚠ Note

Checkpointing adds overhead for serialization and storage I/O. Use it when persistence is required, not for simple stateless request-response patterns.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
checkpoint_storage Required*	CheckpointStorage Storage backend implementing CheckpointStorage interface. Common implementations: InMemoryCheckpointStorage (testing), database-backed storage (production).

Returns

[Expand table](#)

Type	Description
HandoffBuilder	Self for method chaining.

with_interaction_mode

Choose whether the workflow requests user input or runs autonomously after agent replies.

In autonomous mode, agents (including specialists) continue iterating on their task until they explicitly invoke a handoff tool or the turn limit is reached. This allows specialists to perform long-running autonomous tasks (e.g., research, coding, analysis) without prematurely returning control to the coordinator or user.

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[coordinator, research_agent])
    .set_coordinator(coordinator)
    .add_handoff(coordinator, research_agent)
    .add_handoff(research_agent, coordinator)
    .with_interaction_mode("autonomous", autonomous_turn_limit=20)
    .build()
)

# Flow: User asks a question
# -> Coordinator routes to Research Agent
# -> Research Agent iterates (researches, analyzes, refines)
# -> Research Agent calls handoff_to_coordinator when done
# -> Coordinator provides final response
```

Python

```
with_interaction_mode(interaction_mode: Literal['human_in_loop', 'autonomous'] =
    'human_in_loop', *, autonomous_turn_limit: int | None = None) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
interaction_mode	Literal ['human_in_loop', 'autonomous'] "human_in_loop" (default) requests user input after each agent response that does not trigger a handoff. "autonomous" lets agents continue working until they invoke a handoff tool or the turn limit is reached. Default value: "human_in_loop"
autonomous_turn_limit Required*	int None

Keyword-Only Parameters

[Expand table](#)

Name	Description
autonomous_turn_limit	Maximum number of agent responses before the workflow yields when in autonomous mode. Only applicable when interaction_mode is

Name	Description
	" <i>autonomous</i> ". Default is 50. Set to <i>None</i> to disable the limit (use with caution). Ignored with a warning if provided when <i>interaction_mode</i> is " <i>human_in_loop</i> ". Default value: None

Returns

[Expand table](#)

Type	Description
HandoffBuilder	Self for chaining.

with_request_info

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info()
    .build()
)

# Pause only before specialist agents (not coordinator)
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info(agents=[refund, shipping])
    .build()
)
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None =
None) -> HandoffBuilder
```

Parameters

[Expand table](#)

Name	Description
agents Required*	<code>Sequence[str AgentProtocol Executor] None</code> Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every participant.

Keyword-Only Parameters

[Expand table](#)

Name	Description
agents	Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:self></code>	The builder instance for fluent chaining.

with_termination_condition

Set a custom termination condition for the handoff workflow.

The condition can be either synchronous or asynchronous.

Example:

Python

```
# Synchronous condition
builder.with_termination_condition(
    lambda conv: len(conv) > 20 or any("goodbye" in msg.text.lower() for msg in
conv[-2:]))
```

```
)

# Asynchronous condition
async def check_termination(conv: list[ChatMessage]) -> bool:
    # Can perform async operations
    return len(conv) > 20

builder.with_termination_condition(check_termination)
```

Python

```
with_termination_condition(condition: Callable[[list[ChatMessage]], bool | Awaitable[bool]] -> HandoffBuilder
```

Parameters

 Expand table

Name	Description
condition Required*	Callable [[list [ChatMessage]], bool Awaitable [bool]] Function that receives the full conversation and returns True (or awaitable True) if the workflow should terminate (not request further user input).

Returns

 Expand table

Type	Description
HandoffBuilder	Self for chaining.

HandoffUserInputRequest Class

Request message emitted when the workflow needs fresh user input.

Note: The conversation field is intentionally excluded from checkpoint serialization to prevent duplication. The conversation is preserved in the coordinator's state and will be reconstructed on restore. See issue #2667.

Constructor

Python
<code>HandoffUserInputRequest()</code>

Parameters

 Expand table

Name	Description
<code>conversation</code> Required*	
<code>awaiting_agent_id</code> Required*	
<code>prompt</code> Required*	
<code>source_executor_id</code> Required*	

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>from_dict</code>	Deserialize from dict, initializing conversation as empty. The conversation will be reconstructed from the coordinator's state on restore.

`to_dict` Serialize to dict, excluding conversation to prevent checkpoint duplication.

The conversation is already preserved in the workflow coordinator's state. Including it here would cause duplicate messages when restoring from checkpoint.

`__init__`

Python

```
__init__(conversation: list[ChatMessage], awaiting_agent_id: str, prompt: str,
source_executor_id: str) -> None
```

Parameters

[Expand table](#)

Name	Description
conversation Required*	<code>list[ChatMessage]</code>
awaiting_agent_id Required*	<code>str</code>
prompt Required*	<code>str</code>
source_executor_id Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

`from_dict`

Deserialize from dict, initializing conversation as empty.

The conversation will be reconstructed from the coordinator's state on restore.

Python

```
from_dict(data: dict[str, Any]) -> HandoffUserInputRequest
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>HandoffUserInputRequest</code>	

to_dict

Serialize to dict, excluding conversation to prevent checkpoint duplication.

The conversation is already preserved in the workflow coordinator's state. Including it here would cause duplicate messages when restoring from checkpoint.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Attributes

awaiting_agent_id

Python

```
awaiting_agent_id: str
```

conversation

Python

```
conversation: list[ChatMessage]
```

prompt

Python

```
prompt: str
```

source_executor_id

Python

```
source_executor_id: str
```

HostedCodeInterpreterTool Class

Represents a hosted tool that can be specified to an AI service to enable it to execute generated code.

This tool does not implement code interpretation itself. It serves as a marker to inform a service that it is allowed to execute generated code if the service is capable of doing so.

Initialize the HostedCodeInterpreterTool.

Constructor

Python
<code>HostedCodeInterpreterTool()</code>

Keyword-Only Parameters

 Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should mostly be HostedFileContent or HostedVectorStoreContent. Can also be DataContent, depending on the service used. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
description	A description of the tool. Default value: None
additional_properties	Additional properties associated with the tool. Default value: None
**kwargs	Additional keyword arguments to pass to the base class.

Examples


```
from agent_framework import HostedCodeInterpreterTool

# Create a code interpreter tool
code_tool = HostedCodeInterpreterTool()

# With file inputs
code_tool_with_files = HostedCodeInterpreterTool(inputs=[{"file_id": "file-123"}, {"file_id": "file-456"}])
```

Methods

 Expand table

<code>__init__</code>	Initialize the HostedCodeInterpreterTool.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the HostedCodeInterpreterTool.

Python

```
__init__(*, inputs: Contents | dict[str, Any] | str | list[Contents | dict[str, Any] | str] | None = None, description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
inputs Required*	<xref:Contents> dict[str, Any] str list[<xref:Contents> dict[str, Any] str] None
description Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should mostly be HostedFileContent or HostedVectorStoreContent. Can also be DataContent, depending on the service used. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
description	A description of the tool. Default value: None

Name	Description
<code>additional_properties</code>	Additional properties associated with the tool. Default value: None
<code>**kwargs</code>	Additional keyword arguments to pass to the base class.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>value</code> Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python
<code>DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}</code>

INJECTABLE

Python
<code>INJECTABLE: ClassVar[set[str]] = {}</code>

HostedFileContent Class

Represents a hosted file content.

Initializes a HostedFileContent instance.

Constructor

Python
HostedFileContent()

Parameters

 Expand table

Name	Description
file_id Required*	The identifier of the hosted file.

Keyword-Only Parameters

 Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python
<pre>from agent_framework import HostedFileContent # Create hosted file content</pre>

```
file_content = HostedFileContent(file_id="file-abc123")
print(file_content.file_id)  # "file-abc123"
```

Methods

 Expand table

<code>__init__</code>	Initializes a HostedFileContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a HostedFileContent instance.

Python

```
__init__(file_id: str, *, additional_properties: dict[str, Any] | None = None,
raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
file_id Required*	str The identifier of the hosted file.
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code>

Name	Description
	Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```



```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`file_id`

The identifier of the hosted file.

type

The type of content, which is always "hosted_file" for this class.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

HostedFileSearchTool Class

Represents a file search tool that can be specified to an AI service to enable it to perform file searches.

Initialize a FileSearchTool.

Constructor

Python
<code>HostedFileSearchTool()</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should be one or more HostedVectorStoreContents. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
max_results	The maximum number of results to return from the file search. If None, max limit is applied. Default value: None
description	A description of the tool. Default value: None
additional_properties	Additional properties associated with the tool. Default value: None
**kwargs	Additional keyword arguments to pass to the base class.

Examples

```
from agent_framework import HostedFileSearchTool

# Create a basic file search tool
file_search = HostedFileSearchTool()

# With vector store inputs and max results
file_search_with_inputs = HostedFileSearchTool(
    inputs=[{"vector_store_id": "vs_123"}],
    max_results=10,
    description="Search files in vector store",
)
```

Methods

 Expand table

<code>__init__</code>	Initialize a FileSearchTool.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize a FileSearchTool.

Python

```
__init__(*, inputs: Contents | dict[str, Any] | str | list[Contents | dict[str, Any] | str] | None = None, max_results: int | None = None, description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
inputs Required*	<xref:Contents> dict[str, Any] str list [<xref:Contents> dict[str, Any] str] None
max_results Required*	int None
description Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should be one or more HostedVectorStoreContents. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com  ", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None

Name	Description
max_results	The maximum number of results to return from the file search. If None, max limit is applied. Default value: None
description	A description of the tool. Default value: None
additional_properties	Additional properties associated with the tool. Default value: None
**kwargs	Additional keyword arguments to pass to the base class.

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[🔗](#) Expand table

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}
```

```
# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>

Name	Description
<code>exclude_none</code>	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


HostedMCPSpecificApproval Class

Represents the specific mode for a hosted tool.

When using this mode, the user must specify which tools always or never require approval. This is represented as a dictionary with two optional keys:

Constructor

Python
<code>HostedMCPSpecificApproval()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>clear</code>	
<code>copy</code>	
<code>fromkeys</code>	Create a new dictionary with keys from iterable and values set to value.
<code>get</code>	Return the value for key if key is in the dictionary, else default.
<code>items</code>	
<code>keys</code>	
<code>pop</code>	If the key is not found, return the default if given; otherwise, raise a <code>KeyError</code> .
<code>popitem</code>	Remove and return a (key, value) pair as a 2-tuple. Pairs are returned in LIFO (last-in, first-out) order. Raises <code>KeyError</code> if the dict is empty.
<code>setdefault</code>	Insert key with a value of default if key is not in the dictionary. Return the value for key if key is in the dictionary, else default.
<code>update</code>	If E is present and has a <code>.keys()</code> method, then does: for k in E: D[k] = E[k] If E is present and lacks a <code>.keys()</code> method, then does: for k, v in E: D[k] = v In either case, this is followed by: for k in F: D[k] = F[k]

values

`__init__`

Python

```
__init__(*args, **kwargs)
```

`__new__`

Python

```
__new__(**kwargs)
```

clear

Python

```
clear() ->
```

copy

Python

```
copy() -> a shallow copy of D
```

fromkeys

Create a new dictionary with keys from iterable and values set to value.

Python

```
fromkeys(value=None, /)
```

Positional-Only Parameters

[Expand table](#)

Name	Description
iterable Required*	

Name	Description
value	Default value: None

get

Return the value for key if key is in the dictionary, else default.

Python
<code>get(key, default=None, /)</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
key Required*	
default	Default value: None

items

Python
<code>items()</code> -> a set-like object providing a view on D's items

keys

Python
<code>keys()</code> -> a set-like object providing a view on D's keys

pop

If the key is not found, return the default if given; otherwise, raise a KeyError.

Python
<code>pop(k, [d])</code> ->

popitem

Remove and return a (key, value) pair as a 2-tuple.

Pairs are returned in LIFO (last-in, first-out) order. Raises KeyError if the dict is empty.

Python

```
popitem()
```

setdefault

Insert key with a value of default if key is not in the dictionary.

Return the value for key if key is in the dictionary, else default.

Python

```
setdefault(key, default=None, /)
```

Positional-Only Parameters

 Expand table

Name	Description
key Required*	
default	Default value: None

update

If E is present and has a .keys() method, then does: for k in E: D[k] = E[k] If E is present and lacks a .keys() method, then does: for k, v in E: D[k] = v In either case, this is followed by: for k in F: D[k] = F[k]

Python

```
update([E], **F) ->
```

values

Python

values() -> an object providing a view on D's **values**

Attributes

always_require_approval

A sequence of tool names that always require approval.

Python

always_require_approval: Collection[str] | **None**

never_require_approval

A sequence of tool names that never require approval.

Python

never_require_approval: Collection[str] | **None**

HostedMCPTool Class

Represents a MCP tool that is managed and executed by the service.

Create a hosted MCP tool.

Constructor

Python
<code>HostedMCPTool()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>name</code>	The name of the tool.
<code>description</code>	A description of the tool. Default value: None
<code>url</code>	The URL of the tool.
<code>approval_mode</code>	The approval mode for the tool. This can be: "always_require": The tool always requires approval before use. "never_require": The tool never requires approval before use. - A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. Default value: None
<code>allowed_tools</code>	A list of tools that are allowed to use this tool. Default value: None
<code>headers</code>	Headers to include in requests to the tool. Default value: None
<code>additional_properties</code>	Additional properties to include in the tool definition. Default value: None
<code>**kwargs</code>	Additional keyword arguments to pass to the base class.

Examples

Python

```
from agent_framework import HostedMCPTool

# Create a basic MCP tool
mcp_tool = HostedMCPTool(
    name="my_mcp_tool",
    url="https://example.com/mcp",
)

# With approval mode and allowed tools
mcp_tool_with_approval = HostedMCPTool(
    name="my_mcp_tool",
    description="My MCP tool",
    url="https://example.com/mcp",
    approval_mode="always_require",
    allowed_tools=["tool1", "tool2"],
    headers={"Authorization": "Bearer token"},
)

# With specific approval mode
mcp_tool_specific = HostedMCPTool(
    name="my_mcp_tool",
    url="https://example.com/mcp",
    approval_mode={
        "always_require_approval": ["dangerous_tool"],
        "never_require_approval": ["safe_tool"],
    },
)
```

Methods

 Expand table

<code>__init__</code>	Create a hosted MCP tool.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with `'_'`).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Create a hosted MCP tool.

Python

```
__init__(*, name: str, description: str | None = None, url: AnyUrl | str, approval_mode: Literal['always_require', 'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None = None, headers: dict[str, str] | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
name Required*	<code>str</code>
description Required*	<code>str</code> <code>None</code>
url Required*	<xref:pydantic.networks.AnyUrl> <code>str</code>
approval_mode Required*	<code>Literal['always_require', 'never_require']</code> <code>agent_framework._tools.HostedMCPSpecificApproval</code> <code>None</code>

Name	Description
allowed_tools Required*	Collection[str] None
headers Required*	dict[str, str] None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
name	The name of the tool.
description	A description of the tool. Default value: None
url	The URL of the tool.
approval_mode	The approval mode for the tool. This can be: "always_require": The tool always requires approval before use. "never_require": The tool never requires approval before use. - A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
headers	Headers to include in requests to the tool. Default value: None
additional_properties	Additional properties to include in the tool definition. Default value: None
**kwargs	Additional keyword arguments to pass to the base class.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

HostedVectorStoreContent Class

Represents a hosted vector store content.

Initializes a HostedVectorStoreContent instance.

Constructor

Python
<code>HostedVectorStoreContent()</code>

Parameters

 Expand table

Name	Description
<code>vector_store_id</code> Required*	The identifier of the hosted vector store.

Keyword-Only Parameters

 Expand table

Name	Description
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content. Default value: None
<code>**kwargs</code>	Any additional keyword arguments.

Examples

Python
<pre>from agent_framework import HostedVectorStoreContent # Create hosted vector store content</pre>

```
vs_content = HostedVectorStoreContent(vector_store_id="vs-xyz789")
print(vs_content.vector_store_id)  # "vs-xyz789"
```

Methods

 Expand table

<code>__init__</code>	Initializes a HostedVectorStoreContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a HostedVectorStoreContent instance.

Python

```
__init__(vector_store_id: str, *, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
vector_store_id Required*	str The identifier of the hosted vector store.
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: - Simple injection: <code>{"<type>": {"<parameter>": value}}</code> - Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> - Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code>

Name	Description
	Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`vector_store_id`

The identifier of the hosted vector store.

type

The type of content, which is always "hosted_vector_store" for this class.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

HostedWebSearchTool Class

Represents a web search tool that can be specified to an AI service to enable it to perform web searches.

Initialize a HostedWebSearchTool.

Constructor

Python
<code>HostedWebSearchTool()</code>

Parameters

 Expand table

Name	Description
<code>description</code>	Default value: None
<code>additional_properties</code>	Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
<code>description</code>	A description of the tool.
<code>additional_properties</code>	Additional properties associated with the tool (e.g., {"user_location": {"city": "Seattle", "country": "US"}}).
<code>**kwargs</code>	Additional keyword arguments to pass to the base class. if additional_properties is not provided, any kwargs will be added to additional_properties.

Examples

Python

```

from agent_framework import HostedWebSearchTool

# Create a basic web search tool
search_tool = HostedWebSearchTool()

# With location context
search_tool_with_location = HostedWebSearchTool(
    description="Search the web for information",
    additional_properties={"user_location": {"city": "Seattle", "country": "US"}},
)

```

Methods

[Expand table](#)

<code>__init__</code>	Initialize a HostedWebSearchTool.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize a HostedWebSearchTool.

Python

```
__init__(description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
description	str None Default value: None
additional_properties	dict[str, Any] None Default value: None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
description	A description of the tool.
additional_properties	Additional properties associated with the tool (e.g., {"user_location": {"city": "Seattle", "country": "US"}}).
**kwargs	Additional keyword arguments to pass to the base class. if additional_properties is not provided, any kwargs will be added to additional_properties.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code>

Name	Description
	Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

InMemoryCheckpointStorage Class

In-memory checkpoint storage for testing and development.

Initialize the memory storage.

Constructor

Python
<code>InMemoryCheckpointStorage()</code>

Methods

[Expand table](#)

<code>__init__</code>	Initialize the memory storage.
<code>__new__</code>	
<code>delete_checkpoint</code>	Delete a checkpoint by ID.
<code>list_checkpoint_ids</code>	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
<code>list_checkpoints</code>	List checkpoint objects. If workflow_id is provided, filter by that workflow.
<code>load_checkpoint</code>	Load a checkpoint by ID.
<code>save_checkpoint</code>	Save a checkpoint and return its ID.

`__init__`

Initialize the memory storage.

Python
<code>__init__()</code> -> <code>None</code>

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

delete_checkpoint

Delete a checkpoint by ID.

Python
<code>async delete_checkpoint(checkpoint_id: str) -> bool</code>

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	str

Returns

[Expand table](#)

Type	Description
bool	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

Python
<code>async list_checkpoint_ids(workflow_id: str None = None) -> list[str]</code>

Parameters

[Expand table](#)

Name	Description
workflow_id	<code>str</code> <code>None</code> Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>list[str]</code>	

list_checkpoints

List checkpoint objects. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoints(workflow_id: str | None = None) -> list[WorkflowCheckpoint]
```

Parameters

[Expand table](#)

Name	Description
workflow_id	<code>str</code> <code>None</code> Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>list[WorkflowCheckpoint]</code>	

load_checkpoint

Load a checkpoint by ID.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	str

Returns

[Expand table](#)

Type	Description
WorkflowCheckpoint None	

save_checkpoint

Save a checkpoint and return its ID.

Python

```
async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str
```

Parameters

[Expand table](#)

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

[Expand table](#)

Type	Description
str	

InProcRunnerContext Class

In-process execution context for local execution and optional checkpointing.

Initialize the in-process execution context.

Constructor

Python
<code>InProcRunnerContext()</code>

Parameters

[Expand table](#)

Name	Description
<code>checkpoint_storage</code>	Optional storage to enable checkpointing. Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the in-process execution context.
<code>__new__</code>	
<code>add_event</code>	Add an event to the context immediately. Events are enqueued so runners can stream them in real time instead of waiting for superstep boundaries.
<code>add_request_info_event</code>	Add a RequestInfoEvent to the context and track it for correlation.
<code>apply_checkpoint</code>	Apply a checkpoint to the current context, mutating its state.
<code>clear_runtime_checkpoint_storage</code>	Clear runtime checkpoint storage override. This is called automatically by workflow execution methods after a run completes, ensuring runtime storage doesn't leak across runs.
<code>create_checkpoint</code>	

drain_events	Drain all currently queued events without blocking for new ones.
drain_messages	
get_pending_request_info_events	Get the mapping of request IDs to their corresponding RequestInfoEvent.
has_checkpointing	
has_events	
has_messages	
is_streaming	Check if the workflow is in streaming mode.
load_checkpoint	
next_event	Wait for and return the next event. Used by the runner to interleave event emission with ongoing iteration work.
reset_for_new_run	Reset the context for a new workflow run. This clears messages, events, and resets streaming flag. Runtime checkpoint storage is NOT cleared here as it's managed at the workflow level.
send_message	
send_request_info_response	Send a response correlated to a pending request.
set_runtime_checkpoint_storage	Set runtime checkpoint storage to override build-time configuration.
set_streaming	Set whether agents should stream incremental updates.
set_workflow_id	

`__init__`

Initialize the in-process execution context.

Python

```
__init__(checkpoint_storage: CheckpointStorage | None = None)
```

Parameters

[Expand table](#)

Name	Description
<code>checkpoint_storage</code>	CheckpointStorage None Optional storage to enable checkpointing. Default value: None

`__new__`

Python
<code>__new__(**kwargs)</code>

`add_event`

Add an event to the context immediately.

Events are enqueued so runners can stream them in real time instead of waiting for superstep boundaries.

Python
<code>async add_event(event: WorkflowEvent) -> None</code>

Parameters

[Expand table](#)

Name	Description
<code>event</code> Required*	WorkflowEvent

Returns

[Expand table](#)

Type	Description
None	

`add_request_info_event`

Add a RequestInfoEvent to the context and track it for correlation.

Python

```
async add_request_info_event(event: RequestInfoEvent) -> None
```

Parameters

[Expand table](#)

Name	Description
event Required*	RequestInfoEvent The RequestInfoEvent to be added.

Returns

[Expand table](#)

Type	Description
None	

apply_checkpoint

Apply a checkpoint to the current context, mutating its state.

Python

```
async apply_checkpoint(checkpoint: WorkflowCheckpoint) -> None
```

Parameters

[Expand table](#)

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

[Expand table](#)

Type	Description
None	

clear_runtime_checkpoint_storage

Clear runtime checkpoint storage override.

This is called automatically by workflow execution methods after a run completes, ensuring runtime storage doesn't leak across runs.

Python
<code>clear_runtime_checkpoint_storage() -> None</code>

Returns

[Expand table](#)

Type	Description
None	

create_checkpoint

Python
<code>async create_checkpoint(shared_state: SharedState, iteration_count: int, metadata: dict[str, Any] None = None) -> str</code>

Parameters

[Expand table](#)

Name	Description
<code>shared_state</code> Required*	<code>SharedState</code>
<code>iteration_count</code> Required*	<code>int</code>
<code>metadata</code>	<code>dict[str, Any] None</code> Default value: None

Returns

 Expand table

Type	Description
<code>str</code>	

drain_events

Drain all currently queued events without blocking for new ones.

Python

```
async drain_events() -> list[WorkflowEvent]
```

Returns

 Expand table

Type	Description
<code>list[WorkflowEvent]</code>	

drain_messages

Python

```
async drain_messages() -> dict[str, list[Message]]
```

Returns

 Expand table

Type	Description
<code>dict[str, list[Message]]</code>	

get_pending_request_info_events

Get the mapping of request IDs to their corresponding RequestInfoEvent.

Python

```
async get_pending_request_info_events() -> dict[str, RequestInfoEvent]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, RequestInfoEvent]</code>	A dictionary mapping request IDs to their corresponding RequestInfoEvent.

has_checkpointing

Python

```
has_checkpointing() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	

has_events

Python

```
async has_events() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	

has_messages

Python

```
async has_messages() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	

is_streaming

Check if the workflow is in streaming mode.

Python

```
is_streaming() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if streaming mode is enabled, False otherwise.

load_checkpoint

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

[Expand table](#)

Name	Description
<code>checkpoint_id</code> Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
WorkflowCheckpoint None	

next_event

Wait for and return the next event.

Used by the runner to interleave event emission with ongoing iteration work.

Python
<pre>async next_event() -> WorkflowEvent</pre>

Returns

[Expand table](#)

Type	Description
WorkflowEvent	

reset_for_new_run

Reset the context for a new workflow run.

This clears messages, events, and resets streaming flag. Runtime checkpoint storage is NOT cleared here as it's managed at the workflow level.

Python
<pre>reset_for_new_run() -> None</pre>

Returns

[Expand table](#)

Type	Description
None	

send_message

Python

```
async send_message(message: Message) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	Message

Returns

[Expand table](#)

Type	Description
None	

send_request_info_response

Send a response correlated to a pending request.

Python

```
async send_request_info_response(request_id: str, response: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
request_id Required*	str The ID of the original request.
response Required*	Any The response data to be sent.

Returns

[Expand table](#)

Type	Description
None	

set_runtime_checkpoint_storage

Set runtime checkpoint storage to override build-time configuration.

Python
<pre>set_runtime_checkpoint_storage(storage: CheckpointStorage) -> None</pre>

Parameters

[Expand table](#)

Name	Description
storage Required*	CheckpointStorage The checkpoint storage to use for this run.

Returns

[Expand table](#)

Type	Description
None	

set_streaming

Set whether agents should stream incremental updates.

Python
<pre>set_streaming(streaming: bool) -> None</pre>

Parameters

[Expand table](#)

Name	Description
streaming Required*	<code>bool</code> True for streaming mode (run_stream), False for non-streaming (run).

Returns

 Expand table

Type	Description
<code>None</code>	

set_workflow_id

Python
<code>set_workflow_id(workflow_id: str) -> <code>None</code></code>

Parameters

 Expand table

Name	Description
workflow_id Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>None</code>	

MCPStdioTool Class

MCP tool for connecting to stdio-based MCP servers.

This class connects to MCP servers that communicate via standard input/output, typically used for local processes.

Initialize the MCP stdio tool.

 Note

The arguments are used to create a StdioServerParameters object, which is then used to create a stdio client. See `mcp.client.stdio.stdio_client` and `mcp.client.stdio.stdio_server_parameters` for more details.

Constructor

Python
<code>MCPStdioTool()</code>

Parameters

 Expand table

Name	Description
<code>name</code> Required*	The name of the tool.
<code>command</code> Required*	The command to run the MCP server.

Keyword-Only Parameters

 Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: "always_require": The tool always requires approval before use. "never_require": The tool never requires approval before use. - A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
args	The arguments to pass to the command. Default value: None
env	The environment variables to set for the command. Default value: None
encoding	The encoding to use for the command output. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the stdio client.

Examples

Python

```
from agent_framework import MCPStdioTool, ChatAgent

# Create an MCP stdio tool
mcp_tool = MCPStdioTool(
    name="filesystem",
    command="npx",
    args=["-y", "@modelcontextprotocol/server-filesystem", "/tmp"],
    description="File system operations",
)

# Use with a chat agent
async with mcp_tool:
    agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
    response = await agent.run("List files in the directory")
```

Methods

 Expand table

<code>__init__</code>	Initialize the MCP stdio tool. ⓘ Note The arguments are used to create a StdioServerParameters object, which is then used to create a stdio client. See <code>mcp.client.stdio.stdio_client</code> and <code>mcp.client.stdio.stdio_server_parameters</code> for more details.
<code>__new__</code>	
<code>call_tool</code>	Call a tool with the given arguments.
<code>close</code>	Disconnect from the MCP server. Closes the connection and cleans up resources.
<code>connect</code>	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
<code>get_mcp_client</code>	Get an MCP stdio client.
<code>get_prompt</code>	Call a prompt with the given arguments.
<code>load_prompts</code>	Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

ⓘ Note

Subclass MCPTool and override this function if you want to adapt the behavior.

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

ⓘ Note

If you want to extend this behavior, you can subclass MCPTool and override

this function. If you want to keep the default behavior, make sure to call `super().message_handler(message)`.

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

ⓘ Note

This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time,

so overriding it is the best way to customize this behavior.

`__init__`

Initialize the MCP stdio tool.

ⓘ **Note**

The arguments are used to create a `StdioServerParameters` object, which is then used to create a stdio client. See `mcp.client.stdio.stdio_client` and `mcp.client.stdio.stdio_server_parameters` for more details.

Python

```
__init__(name: str, command: str, *, load_tools: bool = True, load_prompts: bool = True, request_timeout: int | None = None, session: ClientSession | None = None, description: str | None = None, approval_mode: Literal['always_require', 'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None = None, args: list[str] | None = None, env: dict[str, str] | None = None, encoding: str | None = None, chat_client: ChatClientProtocol | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
name Required*	<code>str</code> The name of the tool.
command Required*	<code>str</code> The command to run the MCP server.
load_tools Required*	<code>bool</code>
load_prompts Required*	<code>bool</code>
request_timeout Required*	<code>int</code> <code>None</code>
session Required*	<xref:mcp.client.session.ClientSession> <code>None</code>

Name	Description
description Required*	str None
approval_mode Required*	Literal ['always_require', 'never_require'] agent_framework._tools.HostedMCPSpecificApproval None
allowed_tools Required*	Collection [str] None
args Required*	list [str] None
env Required*	dict [str, str] None
encoding Required*	str None
chat_client Required*	ChatClientProtocol None
additional_properties Required*	dict [str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None

Name	Description
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
args	The arguments to pass to the command. Default value: None
env	The environment variables to set for the command. Default value: None
encoding	The encoding to use for the command output. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the stdio client.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

call_tool

Call a tool with the given arguments.

Python

```
async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]
```

Parameters

 Expand table

Name	Description
tool_name Required*	<code>str</code> The name of the tool to call.
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Arguments to pass to the tool.

Returns

 Expand table

Type	Description
<code>list[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent]</code>	A list of content items returned by the tool.

Exceptions

 Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python
<code>async close() -> None</code>

Returns

[\[\]](#) Expand table

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

Python
<code>async connect() -> None</code>

Returns

[\[\]](#) Expand table

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP stdio client.

Python

```
get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]
```

Returns

[Expand table](#)

Type	Description
contextlib._AsyncGeneratorContextManager [Any , None]	An async context manager for the stdio client transport.

get_prompt

Call a prompt with the given arguments.

Python

```
async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]
```

Parameters

[Expand table](#)

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>kwargs</code>	Arguments to pass to the prompt.

Returns

[Expand table](#)

Type	Description
<code>list[ChatMessage]</code>	A list of chat messages returned by the prompt.

Exceptions

[Expand table](#)

Type	Description
<code>ToolExecutionException</code>	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

Python
<pre>async load_prompts() -> None</pre>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

Python
<pre>async load_tools() -> None</pre>

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

Note

Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

[Expand table](#)

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

[Expand table](#)

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

ⓘ **Note**

If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call `super().message_handler(message)`.

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] | ServerNotification | Exception) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder> [<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] <xref:mcp.types.ServerNotification> Exception The message from the MCP server (request responder, notification, or exception).

Returns

[Expand table](#)

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

ⓘ Note

This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params: CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

[Expand table](#)

Name	Description
context Required*	<xref:mcp.shared.context.RequestContext> [<xref:mcp.client.session.ClientSession>, Any , Any] The request context from the MCP server.
params Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

 Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MCPStreamableHTTPTool Class

MCP tool for connecting to HTTP-based MCP servers.

This class connects to MCP servers that communicate via streamable HTTP/SSE.

Initialize the MCP streamable HTTP tool.

 Note

The arguments are used to create a streamable HTTP client.

See `mcp.client.streamable_http.streamablehttp_client` for more details.

Any extra arguments passed to the constructor will be passed to the streamable HTTP client constructor.

Constructor

Python
<code>MCPStreamableHTTPTool()</code>

Parameters

 Expand table

Name	Description
<code>name</code> Required*	The name of the tool.
<code>url</code> Required*	The URL of the MCP server.

Keyword-Only Parameters

 Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: "always_require": The tool always requires approval before use. "never_require": The tool never requires approval before use. - A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
headers	The headers to send with the request. Default value: None
timeout	The timeout for the request. Default value: None
sse_read_timeout	The timeout for reading from the SSE stream. Default value: None
terminate_on_close	Close the transport when the MCP client is terminated. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the SSE client.

Examples

Python

```
from agent_framework import MCPStreamableHTTPTool, ChatAgent

# Create an MCP HTTP tool
mcp_tool = MCPStreamableHTTPTool(
    name="web-api",
    url="https://api.example.com/mcp",
    headers={"Authorization": "Bearer token"},
    description="Web API operations",
)

# Use with a chat agent
async with mcp_tool:
    agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
    response = await agent.run("Fetch data from the API")
```

Methods

 Expand table

<code>__init__</code>	Initialize the MCP streamable HTTP tool. ⓘ Note The arguments are used to create a streamable HTTP client. See <code>mcp.client.streamable_http.streamablehttp_client</code> for more details. Any extra arguments passed to the constructor will be passed to the streamable HTTP client constructor.
<code>__new__</code>	
<code>call_tool</code>	Call a tool with the given arguments.
<code>close</code>	Disconnect from the MCP server. Closes the connection and cleans up resources.
<code>connect</code>	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
<code>get_mcp_client</code>	Get an MCP streamable HTTP client.

<code>get_prompt</code>	Call a prompt with the given arguments.
<code>load_prompts</code>	Load prompts from the MCP server. Retrieves available prompts from the connected MCP server and converts them into <code>AIFunction</code> instances.
<code>load_tools</code>	Load tools from the MCP server. Retrieves available tools from the connected MCP server and converts them into <code>AIFunction</code> instances.
<code>logging_callback</code>	Callback function for logging. This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params. Note Subclass <code>MCPTool</code> and override this function if you want to adapt the behavior.
<code>message_handler</code>	Handle messages from the MCP server. By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received. Note If you want to extend this behavior, you can subclass <code>MCPTool</code> and override this function. If you want to keep the default behavior, make sure to call <code>super().message_handler(message)</code>.
<code>sampling_callback</code>	Callback function for sampling. This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses. Note This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time,

so overriding it is the best way to customize this behavior.

`__init__`

Initialize the MCP streamable HTTP tool.

ⓘ Note

The arguments are used to create a streamable HTTP client.

See `mcp.client.streamable_http.streamablehttp_client` for more details.

Any extra arguments passed to the constructor will be passed to the streamable HTTP client constructor.

Python

```
__init__(name: str, url: str, *, load_tools: bool = True, load_prompts: bool = True, request_timeout: int | None = None, session: ClientSession | None = None, description: str | None = None, approval_mode: Literal['always_require', 'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None = None, headers: dict[str, Any] | None = None, timeout: float | None = None, sse_read_timeout: float | None = None, terminate_on_close: bool | None = None, chat_client: ChatClientProtocol | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
name Required*	<code>str</code> The name of the tool.
url Required*	<code>str</code> The URL of the MCP server.
load_tools Required*	<code>bool</code>
load_prompts Required*	<code>bool</code>

Name	Description
request_timeout Required*	int None
session Required*	<xref:mcp.client.session.ClientSession> None
description Required*	str None
approval_mode Required*	Literal ['always_require', 'never_require'] agent_framework_tools.HostedMCPSpecificApproval None
allowed_tools Required*	Collection [str] None
headers Required*	dict [str , Any] None
timeout Required*	float None
sse_read_timeout Required*	float None
terminate_on_close Required*	bool None
chat_client Required*	ChatClientProtocol None
additional_properties Required*	dict [str , Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True

Name	Description
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: "always_require": The tool always requires approval before use. "never_require": The tool never requires approval before use. - A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
headers	The headers to send with the request. Default value: None
timeout	The timeout for the request. Default value: None
sse_read_timeout	The timeout for reading from the SSE stream. Default value: None
terminate_on_close	Close the transport when the MCP client is terminated. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the SSE client.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

call_tool

Call a tool with the given arguments.

Python
<pre>async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent]</pre>

Parameters

[Expand table](#)

Name	Description
tool_name Required*	<code>str</code> The name of the tool to call.
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Arguments to pass to the tool.

Returns

[Expand table](#)

Type	Description
list [TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent]	A list of content items returned by the tool.

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python
<pre>async close() -> None</pre>

Returns

[Expand table](#)

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

Python
<pre>async connect() -> None</pre>

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP streamable HTTP client.

Python
<code>get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:contextlib._AsyncGeneratorContextManager>[Any, None]</code>	An async context manager for the streamable HTTP client transport.

get_prompt

Call a prompt with the given arguments.

Python
<code>async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]</code>

Parameters

[Expand table](#)

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Arguments to pass to the prompt.

Returns

[Expand table](#)

Type	Description
list[ChatMessage]	A list of chat messages returned by the prompt.

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_prompts() -> None
```

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

Python
<pre>async load_tools() -> None</pre>

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

ⓘ **Note**

Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

[Expand table](#)

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

[Expand table](#)

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

ⓘ **Note**

If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call

```
super().message_handler(message).
```

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] |  
ServerNotification | Exception) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder> [<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] <xref:mcp.types.ServerNotification> Exception The message from the MCP server (request responder, notification, or exception).

Returns

[Expand table](#)

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

 Note

This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params:
CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

 Expand table

Name	Description
context Required*	<xref:mcp.shared.context.RequestContext> [<xref:mcp.client.session.ClientSession>, Any , Any] The request context from the MCP server.
params Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

 Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MCPWebsocketTool Class

MCP tool for connecting to WebSocket-based MCP servers.

This class connects to MCP servers that communicate via WebSocket.

Initialize the MCP WebSocket tool.

 Note

The arguments are used to create a WebSocket client.

See `mcp.client.websocket.websocket_client` for more details.

Any extra arguments passed to the constructor will be passed to the
WebSocket client constructor.

Constructor

Python
<code>MCPWebsocketTool()</code>

Parameters

 Expand table

Name	Description
<code>name</code> Required*	The name of the tool.
<code>url</code> Required*	The URL of the MCP server.

Keyword-Only Parameters

 Expand table

Name	Description
<code>load_tools</code>	Whether to load tools from the MCP server. Default value: True
<code>load_prompts</code>	Whether to load prompts from the MCP server. Default value: True
<code>request_timeout</code>	The default timeout in seconds for all requests. Default value: None
<code>session</code>	The session to use for the MCP connection. Default value: None
<code>description</code>	The description of the tool. Default value: None
<code>approval_mode</code>	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
<code>allowed_tools</code>	A list of tools that are allowed to use this tool. Default value: None
<code>additional_properties</code>	Additional properties. Default value: None
<code>chat_client</code>	The chat client to use for sampling. Default value: None
<code>kwargs</code>	Any extra arguments to pass to the WebSocket client.

Examples

Python

```
from agent_framework import MCPWebSocketTool, ChatAgent

# Create an MCP WebSocket tool
mcp_tool = MCPWebSocketTool(
    name="realtime-service", url="wss://service.example.com/mcp",
    description="Real-time service operations"
)
```



```
# Use with a chat agent
async with mcp_tool:
    agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
    response = await agent.run("Connect to the real-time service")
```

Methods

 Expand table

<code>__init__</code>	Initialize the MCP WebSocket tool. ⓘ Note The arguments are used to create a WebSocket client. See <code>mcp.client.websocket.websocket_client</code> for more details. Any extra arguments passed to the constructor will be passed to the WebSocket client constructor.
<code>__new__</code>	
<code>call_tool</code>	Call a tool with the given arguments.
<code>close</code>	Disconnect from the MCP server. Closes the connection and cleans up resources.
<code>connect</code>	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
<code>get_mcp_client</code>	Get an MCP WebSocket client.
<code>get_prompt</code>	Call a prompt with the given arguments.
<code>load_prompts</code>	Load prompts from the MCP server. Retrieves available prompts from the connected MCP server and converts them into <code>AIFunction</code> instances.
<code>load_tools</code>	Load tools from the MCP server. Retrieves available tools from the connected MCP server and converts them into <code>AIFunction</code> instances.
<code>logging_callback</code>	Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

ⓘ **Note**

Subclass MCPTool and override this function if you want to adapt the behavior.

`message_handler` Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

ⓘ **Note**

If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call `super().message_handler(message)`.

`sampling_callback` Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

ⓘ **Note**

This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

`__init__`

Initialize the MCP WebSocket tool.

ⓘ **Note**

The arguments are used to create a WebSocket client.

See `mcp.client.websocket.websocket_client` for more details.

Any extra arguments passed to the constructor will be passed to the `WebSocket` client constructor.

Python

```
__init__(name: str, url: str, *, load_tools: bool = True, load_prompts: bool = True, request_timeout: int | None = None, session: ClientSession | None = None, description: str | None = None, approval_mode: Literal['always_require', 'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None = None, chat_client: ChatClientProtocol | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
name Required*	<code>str</code> The name of the tool.
url Required*	<code>str</code> The URL of the MCP server.
load_tools Required*	<code>bool</code>
load_prompts Required*	<code>bool</code>
request_timeout Required*	<code>int</code> <code>None</code>
session Required*	<code><xref:mcp.client.session.ClientSession></code> <code>None</code>
description Required*	<code>str</code> <code>None</code>
approval_mode Required*	<code>Literal['always_require', 'never_require']</code> <code>agent_framework_tools.HostedMCPSpecificApproval</code> <code>None</code>
allowed_tools Required*	<code>Collection[str]</code> <code>None</code>
chat_client	<code>ChatClientProtocol</code> <code>None</code>

Name	Description
Required*	
additional_properties Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys <i>always_require_approval</i> or <i>never_require_approval</i>, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
chat_client	The chat client to use for sampling. Default value: None

Name	Description
<code>kwargs</code>	Any extra arguments to pass to the WebSocket client.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`call_tool`

Call a tool with the given arguments.

Python
<pre>async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent]</pre>

Parameters

[Expand table](#)

Name	Description
<code>tool_name</code> Required*	<code>str</code> The name of the tool to call.
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Arguments to pass to the tool.

Returns

[Expand table](#)

Type	Description
list[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent]	A list of content items returned by the tool.

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python
<code>async close() -> None</code>

Returns

[Expand table](#)

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

Python

```
async connect() -> None
```

Returns

[Expand table](#)

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP WebSocket client.

Python

```
get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]
```

Returns

[Expand table](#)

Type	Description
<xref:contextlib._AsyncGeneratorContextManager>[Any, None]	An async context manager for the WebSocket client transport.

get_prompt

Call a prompt with the given arguments.

Python

```
async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]
```

Parameters

[Expand table](#)

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Arguments to pass to the prompt.

Returns

[Expand table](#)

Type	Description
list[ChatMessage]	A list of chat messages returned by the prompt.

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_prompts() -> None
```

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_tools() -> None
```

Returns

 Expand table

Type	Description
None	

Exceptions

[Expand table](#)

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

ⓘ Note

Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

[Expand table](#)

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

[Expand table](#)

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

ⓘ **Note**

If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call `super().message_handler(message)`.

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] | ServerNotification | Exception) -> None
```

Parameters

[⌵](#) Expand table

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder> [<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] <xref:mcp.types.ServerNotification> Exception The message from the MCP server (request responder, notification, or exception).

Returns

[⌵](#) Expand table

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

ⓘ **Note**

This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params: CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

 Expand table

Name	Description
context Required*	<xref:mcp.shared.context.RequestContext> [<xref:mcp.client.session.ClientSession>, Any , Any] The request context from the MCP server.
params Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

 Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MagenticBuilder Class

Fluent builder for creating Magentic One multi-agent orchestration workflows.

Magentic One workflows use an LLM-powered manager to coordinate multiple agents through dynamic task planning, progress tracking, and adaptive replanning. The manager creates plans, selects agents, monitors progress, and determines when to replan or complete.

The builder provides a fluent API for configuring participants, the manager, optional plan review, checkpointing, and event callbacks.

Human-in-the-loop Support: Magentic provides specialized HITL mechanisms via:

- `.with_plan_review()` - Review and approve/revise plans before execution
- `.with_human_input_on_stall()` - Intervene when workflow stalls
- Tool approval via *FunctionApprovalRequestContent* - Approve individual tool calls

These emit *MagenticHumanInterventionRequest* events that provide structured decision options (APPROVE, REVISE, CONTINUE, REPLAN, GUIDANCE) appropriate for Magentic's planning-based orchestration.

Usage:

Python

```
from agent_framework import MagenticBuilder, StandardMagenticManager
from azure.ai.projects.aio import AIProjectClient

# Create manager with LLM client
project_client = AIProjectClient.from_connection_string(...)
chat_client = project_client.inference.get_chat_completions_client()

# Build Magentic workflow with agents
workflow = (
    MagenticBuilder()
    .participants(researcher=research_agent, writer=writing_agent,
coder=coding_agent)
    .with_standard_manager(chat_client=chat_client, max_round_count=20,
max_stall_count=3)
    .with_plan_review(enable=True)
    .with_checkpointing(checkpoint_storage)
    .build()
)

# Execute workflow
```

```
async for message in workflow.run("Research and write article about AI agents"):
    print(message.text)
```

With custom manager:

Python

```
# Create custom manager subclass
class MyCustomManager(MagenticManagerBase):
    async def plan(self, context: MagenticContext) -> ChatMessage:
        # Custom planning logic
        ...

manager = MyCustomManager()
workflow = MagenticBuilder().participants(agent1=agent1,
agent2=agent2).with_standard_manager(manager).build()
```

Constructor

Python

MagenticBuilder()

Methods

 Expand table

__init__	
__new__	
build	Build a Magentic workflow with the orchestrator and all agent executors.
participants	Add participant agents or executors to the Magentic workflow. Participants are the agents that will execute tasks under the manager's direction. Each participant should have distinct capabilities that complement the team. The manager will select which participant to invoke based on the current plan and progress state. Usage: Python <pre>workflow = (MagenticBuilder()</pre>

```

        .participants(
            researcher=research_agent, writer=writing_agent,
            coder=coding_agent, reviewer=review_agent
        )
        .with_standard_manager(agent=manager_agent)
        .build()
    )

```

-[Notes]-

- Participant names become part of the manager's context for selection
- Agent descriptions (if available) are extracted and provided to the manager
- Can be called multiple times to add participants incrementally

start_with	Build a Magentic workflow and return a wrapper with convenience methods.
start_with_message	Build a Magentic workflow and return a wrapper with convenience methods for ChatMessage tasks.
start_with_string	Build a Magentic workflow and return a wrapper with convenience methods for string tasks.

with_checkpointing	Enable workflow state persistence using the provided checkpoint storage. Checkpointing allows workflows to be paused, resumed across process restarts, or recovered after failures. The entire workflow state including conversation history, task ledgers, and progress is persisted at key points.
------------------------------------	--

Usage:

Python

```

from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent)
    .with_checkpointing(storage)
    .build()
)

# First run
thread_id = "task-123"
async for msg in workflow.run("task",
thread_id=thread_id):
    print(msg.text)

# Resume from checkpoint
async for msg in workflow.run("continue",

```

```
thread_id=thread_id):  
    print(msg.text)
```

-[Notes]-

- Checkpoints are created after each significant state transition
- Thread ID must be consistent across runs to resume properly
- Storage implementations may have different persistence guarantees

`with_human_input_on_stall` Enable human intervention when the workflow detects a stall.

When enabled, instead of automatically replanning when the workflow detects that agents are not making progress or are stuck in a loop, the workflow will pause and emit a `MagenticStallInterventionRequest` event. A human can then decide to continue, trigger replanning, or provide guidance.

This is useful for:

- Workflows where automatic replanning may not resolve the issue
- Scenarios requiring human judgment about workflow direction
- Debugging stuck workflows with human insight
- Complex tasks where human guidance can help agents get back on track

When stall detection triggers (based on `max_stall_count`), instead of calling `_reset_and_replan` automatically, the workflow will:

1. Emit a `MagenticHumanInterventionRequest` with `kind=STALL`
2. Wait for human response via `send_responses_streaming`
3. Take action based on the human's decision (continue, replan, or guidance)

Usage:

Python

```
workflow = (  
    MagenticBuilder()  
    .participants(agent1=agent1)  
    .with_standard_manager(agent=manager_agent,  
max_stall_count=3)  
    .with_human_input_on_stall(enable=True)  
    .build()  
)  
  
# During execution, handle human intervention requests  
async for event in workflow.run_stream("task"):  
    if isinstance(event, RequestInfoEvent):  
        if event.request_type is  
MagenticHumanInterventionRequest:  
            request = event.data  
            if request.kind ==  
MagenticHumanInterventionKind.STALL:
```



```

        print(f"Workflow stalled:
{request.stall_reason}")
        reply = MagenticHumanInterventionReply(

decision=MagenticHumanInterventionDecision.GUIDANCE,
        comments="Focus on completing the
current step first",
        )
        responses = {event.request_id: reply}
        async for ev in
workflow.send_responses_streaming(responses):
            ...

```

with_plan_review

Enable or disable human-in-the-loop plan review before task execution.

When enabled, the workflow will pause after the manager generates the initial plan and emit a `MagenticHumanInterventionRequest` event with `kind=PLAN_REVIEW`. A human reviewer can then approve, request revisions, or reject the plan. The workflow continues only after approval.

This is useful for:

- High-stakes tasks requiring human oversight
- Validating the manager's understanding of requirements
- Catching hallucinations or unrealistic plans early
- Educational scenarios where learners review AI planning

Usage:

Python

```

workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent)
    .with_plan_review(enable=True)
    .build()
)

# During execution, handle plan review
async for event in workflow.run_stream("task"):
    if isinstance(event, RequestInfoEvent):
        request = event.data
        if isinstance(request,
MagenticHumanInterventionRequest):
            if request.kind ==
MagenticHumanInterventionKind.PLAN_REVIEW:
                # Review plan and respond
                reply =
MagenticHumanInterventionReply(decision=MagenticHumanInterve
ntionDecision.APPROVE)

```

```
        await
        workflow.send_responses({event.request_id: reply})
```

with_standard_manager

Configure the workflow manager for task planning and agent coordination.

The manager is responsible for creating plans, selecting agents, tracking progress, and deciding when to replan or complete. This method supports two usage patterns:

1. **Provide existing manager:** Pass a pre-configured manager instance (custom or standard) for full control over behavior
2. **Auto-create with agent:** Pass an agent to automatically create a StandardMagneticManager that uses the agent's configured instructions and options (temperature, seed, etc.)

Usage with agent (recommended):

Python

```
from agent_framework import ChatAgent, ChatOptions
from agent_framework.openai import OpenAIChatClient

# Configure manager agent with specific options and
instructions
manager_agent = ChatAgent(
    name="Coordinator",
    chat_client=OpenAIChatClient(model_id="gpt-4o"),
    chat_options=ChatOptions(temperature=0.3, seed=42),
    instructions="Be concise and focus on accuracy",
)

workflow = (
    MagneticBuilder()
    .participants(agent1=agent1, agent2=agent2)
    .with_standard_manager(
        agent=manager_agent,
        max_round_count=20,
        max_stall_count=3,
    )
    .build()
)
```

Usage with custom manager:

Python

```
class MyManager(MagneticManagerBase):
    async def plan(self, context: MagneticContext) ->
    ChatMessage:
        # Custom planning logic
        return ChatMessage(role=Role.ASSISTANT,
```

```
text="...")

manager = MyManager()
workflow =
MagenticBuilder().participants(agent1=agent1).with_standard_
manager(manager).build()
```

Usage with prompt customization:

Python

```
workflow = (
    MagenticBuilder()
    .participants(coder=coder_agent,
reviewer=reviewer_agent)
    .with_standard_manager(
        agent=manager_agent,
        task_ledger_plan_prompt="Create a detailed step-
by-step plan...",
        progress_ledger_prompt="Assess progress and
decide next action...",
        max_stall_count=2,
    )
    .build()
)
```

-[Notes]-

- StandardMagenticManager uses structured LLM calls for all decisions
- Custom managers can implement alternative selection strategies
- Prompt templates support Jinja2-style variable substitution
- Stall detection helps prevent infinite loops in stuck scenarios
- The agent's instructions are used as system instructions for all manager prompts

__init__

Python

```
__init__() -> None
```

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

build

Build a Magentic workflow with the orchestrator and all agent executors.

Python
<code>build() -> Workflow</code>

Returns

 Expand table

Type	Description
Workflow	

participants

Add participant agents or executors to the Magentic workflow.

Participants are the agents that will execute tasks under the manager's direction. Each participant should have distinct capabilities that complement the team. The manager will select which participant to invoke based on the current plan and progress state.

Usage:

Python
<pre>workflow = (MagenticBuilder() .participants(researcher=research_agent, writer=writing_agent, coder=coding_agent, reviewer=review_agent))</pre>

```
        .with_standard_manager(agent=manager_agent)
        .build()
    )
```

-[Notes]-

- Participant names become part of the manager's context for selection
- Agent descriptions (if available) are extracted and provided to the manager
- Can be called multiple times to add participants incrementally

Python

```
participants(**participants: AgentProtocol | Executor) -> Self
```

Parameters

 Expand table

Name	Description
**participants Required*	AgentProtocol Executor Named agents or executors to add to the workflow. Names should be descriptive of the agent's role (e.g., researcher=research_agent). Accepts BaseAgent instances or custom Executor implementations.

Returns

 Expand table

Type	Description
Self	Self for method chaining

start_with

Build a Magentic workflow and return a wrapper with convenience methods.

Python

```
start_with(task: str | ChatMessage) -> MagenticWorkflow
```

Parameters

[Expand table](#)

Name	Description
task Required*	str ChatMessage The task description as a string or ChatMessage.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_magentic.M agenticWorkflow>	A MagenticWorkflow wrapper that provides convenience methods.

start_with_message

Build a Magentic workflow and return a wrapper with convenience methods for ChatMessage tasks.

Python
<pre>start_with_message(task: ChatMessage) -> MagenticWorkflow</pre>

Parameters

[Expand table](#)

Name	Description
task Required*	ChatMessage The task as a ChatMessage.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_magentic.M agenticWorkflow>	A MagenticWorkflow wrapper that provides convenience methods.

start_with_string

Build a Magentic workflow and return a wrapper with convenience methods for string tasks.

Python

```
start_with_string(task: str) -> MagenticWorkflow
```

Parameters

 Expand table

Name	Description
task Required*	<code>str</code> The task description as a string.

Returns

 Expand table

Type	Description
<code><xref:agent_framework_workflows_magentic.MagenticWorkflow></code>	A MagenticWorkflow wrapper that provides convenience methods for starting with strings.

with_checkpointing

Enable workflow state persistence using the provided checkpoint storage.

Checkpointing allows workflows to be paused, resumed across process restarts, or recovered after failures. The entire workflow state including conversation history, task ledgers, and progress is persisted at key points.

Usage:

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent)
    .with_checkpointing(storage)
    .build()
)
```

```
# First run
thread_id = "task-123"
async for msg in workflow.run("task", thread_id=thread_id):
    print(msg.text)

# Resume from checkpoint
async for msg in workflow.run("continue", thread_id=thread_id):
    print(msg.text)
```

-[Notes]-

- Checkpoints are created after each significant state transition
- Thread ID must be consistent across runs to resume properly
- Storage implementations may have different persistence guarantees

Python

with_checkpointing(checkpoint_storage: CheckpointStorage) -> MagenticBuilder

Parameters

 Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage Storage backend for checkpoints (e.g., InMemoryCheckpointStorage, FileCheckpointStorage, or custom implementations)

Returns

 Expand table

Type	Description
MagenticBuilder	Self for method chaining

with_human_input_on_stall

Enable human intervention when the workflow detects a stall.

When enabled, instead of automatically replanning when the workflow detects that agents are not making progress or are stuck in a loop, the workflow will pause and emit a

MagenticStallInterventionRequest event. A human can then decide to continue, trigger replanning, or provide guidance.

This is useful for:

- Workflows where automatic replanning may not resolve the issue
- Scenarios requiring human judgment about workflow direction
- Debugging stuck workflows with human insight
- Complex tasks where human guidance can help agents get back on track

When stall detection triggers (based on `max_stall_count`), instead of calling `_reset_and_replan` automatically, the workflow will:

1. Emit a `MagenticHumanInterventionRequest` with `kind=STALL`
2. Wait for human response via `send_responses_streaming`
3. Take action based on the human's decision (continue, replan, or guidance)

Usage:

Python

```
workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent, max_stall_count=3)
    .with_human_input_on_stall(enable=True)
    .build()
)

# During execution, handle human intervention requests
async for event in workflow.run_stream("task"):
    if isinstance(event, RequestInfoEvent):
        if event.request_type is MagenticHumanInterventionRequest:
            request = event.data
            if request.kind == MagenticHumanInterventionKind.STALL:
                print(f"Workflow stalled: {request.stall_reason}")
                reply = MagenticHumanInterventionReply(
                    decision=MagenticHumanInterventionDecision.GUIDANCE,
                    comments="Focus on completing the current step first",
                )
                responses = {event.request_id: reply}
                async for ev in workflow.send_responses_streaming(responses):
                    ...
```

Python

```
with_human_input_on_stall(enable: bool = True) -> MagenticBuilder
```

Parameters

 Expand table

Name	Description
enable	bool Whether to enable stall intervention (default True) Default value: True

Returns

 Expand table

Type	Description
MagneticBuilder	Self for method chaining

- See also
- [<xref:agent_framework.MagneticHumanInterventionRequest>](#): Unified request type
 - [MagneticHumanInterventionDecision](#): Decision options
 - [with_standard_manager](#): Configure max_stall_count for stall detection

with_plan_review

Enable or disable human-in-the-loop plan review before task execution.

When enabled, the workflow will pause after the manager generates the initial plan and emit a `MagneticHumanInterventionRequest` event with `kind=PLAN_REVIEW`. A human reviewer can then approve, request revisions, or reject the plan. The workflow continues only after approval.

This is useful for:

- High-stakes tasks requiring human oversight
- Validating the manager's understanding of requirements
- Catching hallucinations or unrealistic plans early
- Educational scenarios where learners review AI planning

Usage:

```
Python
```

```

workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent)
    .with_plan_review(enable=True)
    .build()
)

# During execution, handle plan review
async for event in workflow.run_stream("task"):
    if isinstance(event, RequestInfoEvent):
        request = event.data
        if isinstance(request, MagenticHumanInterventionRequest):
            if request.kind == MagenticHumanInterventionKind.PLAN_REVIEW:
                # Review plan and respond
                reply =
MagenticHumanInterventionReply(decision=MagenticHumanInterventionDecision.APPROVE)
                await workflow.send_responses({event.request_id: reply})

```

Python

with_plan_review(enable: bool = True) -> MagenticBuilder

Parameters

[Expand table](#)

Name	Description
enable	bool Whether to require plan review (default True) Default value: True

Returns

[Expand table](#)

Type	Description
MagenticBuilder	Self for method chaining

See
also

- [<xref:agent_framework.MagenticHumanInterventionRequest>](#): Event emitted for review
- [<xref:agent_framework.MagenticHumanInterventionReply>](#): Response to send back

- [MagenticHumanInterventionDecision](#): APPROVE/REVISE options

with_standard_manager

Configure the workflow manager for task planning and agent coordination.

The manager is responsible for creating plans, selecting agents, tracking progress, and deciding when to replan or complete. This method supports two usage patterns:

1. **Provide existing manager:** Pass a pre-configured manager instance (custom or standard) for full control over behavior
2. **Auto-create with agent:** Pass an agent to automatically create a StandardMagenticManager that uses the agent's configured instructions and options (temperature, seed, etc.)

Usage with agent (recommended):

Python

```
from agent_framework import ChatAgent, ChatOptions
from agent_framework.openai import OpenAIChatClient

# Configure manager agent with specific options and instructions
manager_agent = ChatAgent(
    name="Coordinator",
    chat_client=OpenAIChatClient(model_id="gpt-4o"),
    chat_options=ChatOptions(temperature=0.3, seed=42),
    instructions="Be concise and focus on accuracy",
)

workflow = (
    MagenticBuilder()
    .participants(agent1=agent1, agent2=agent2)
    .with_standard_manager(
        agent=manager_agent,
        max_round_count=20,
        max_stall_count=3,
    )
    .build()
)
```

Usage with custom manager:

Python

```

class MyManager(MagenticManagerBase):
    async def plan(self, context: MagenticContext) -> ChatMessage:
        # Custom planning logic
        return ChatMessage(role=Role.ASSISTANT, text="...")

manager = MyManager()
workflow =
MagenticBuilder().participants(agent1=agent1).with_standard_manager(manager).build
()

```

Usage with prompt customization:

Python

```

workflow = (
    MagenticBuilder()
    .participants(coder=coder_agent, reviewer=reviewer_agent)
    .with_standard_manager(
        agent=manager_agent,
        task_ledger_plan_prompt="Create a detailed step-by-step plan...",
        progress_ledger_prompt="Assess progress and decide next action...",
        max_stall_count=2,
    )
    .build()
)

```

-[Notes]-

- StandardMagenticManager uses structured LLM calls for all decisions
- Custom managers can implement alternative selection strategies
- Prompt templates support Jinja2-style variable substitution
- Stall detection helps prevent infinite loops in stuck scenarios
- The agent's instructions are used as system instructions for all manager prompts

Python

```

with_standard_manager(manager: MagenticManagerBase | None = None, *, agent:
AgentProtocol | None = None, task_ledger: _MagenticTaskLedger | None = None,
task_ledger_facts_prompt: str | None = None, task_ledger_plan_prompt: str | None =
None, task_ledger_full_prompt: str | None = None, task_ledger_facts_update_prompt:
str | None = None, task_ledger_plan_update_prompt: str | None = None,
progress_ledger_prompt: str | None = None, final_answer_prompt: str | None = None,

```

```
max_stall_count: int = 3, max_reset_count: int | None = None, max_round_count: int | None = None) -> Self
```

Parameters

 Expand table

Name	Description
manager	MagenticManagerBase None Pre-configured manager instance (StandardMagenticManager or custom MagenticManagerBase subclass). If provided, all other arguments are ignored. Default value: None
agent Required*	AgentProtocol None Agent instance for generating plans and decisions. The agent's configured instructions and options (temperature, seed, etc.) will be applied.
task_ledger Required*	<xref:agent_framework_workflows_magentic_MagenticTaskLedger> None Optional custom task ledger implementation for specialized prompting or structured output requirements
task_ledger_facts_prompt Required*	str None Custom prompt template for extracting facts from task description
task_ledger_plan_prompt Required*	str None Custom prompt template for generating initial plan
task_ledger_full_prompt Required*	str None Custom prompt template for complete task ledger (facts + plan combined)
task_ledger_facts_update_prompt Required*	str None Custom prompt template for updating facts based on agent progress
task_ledger_plan_update_prompt Required*	str None Custom prompt template for replanning when needed
progress_ledger_prompt Required*	str None Custom prompt template for assessing progress and determining next actions
final_answer_prompt Required*	str None Custom prompt template for synthesizing final response when task is complete

Name	Description
<code>max_stall_count</code> Required*	<code>int</code> Maximum consecutive rounds without progress before triggering replan (default 3). Set to 0 to disable stall detection.
<code>max_reset_count</code> Required*	<code>int None</code> Maximum number of complete resets allowed before failing. None means unlimited resets.
<code>max_round_count</code> Required*	<code>int None</code> Maximum total coordination rounds before stopping with partial result. None means unlimited rounds.

Keyword-Only Parameters

 Expand table

Name	Description
<code>agent</code>	Default value: None
<code>task_ledger</code>	Default value: None
<code>task_ledger_facts_prompt</code>	Default value: None
<code>task_ledger_plan_prompt</code>	Default value: None
<code>task_ledger_full_prompt</code>	Default value: None
<code>task_ledger_facts_update_prompt</code>	Default value: None
<code>task_ledger_plan_update_prompt</code>	Default value: None
<code>progress_ledger_prompt</code>	Default value: None
<code>final_answer_prompt</code>	Default value: None
<code>max_stall_count</code>	Default value: 3
<code>max_reset_count</code>	Default value: None
<code>max_round_count</code>	Default value: None

Returns

 Expand table

Type	Description
<code>Self</code>	Self for method chaining

Exceptions

 Expand table

Type	Description
ValueError	If manager is None and agent is not provided.

MagenticContext Class

Context for the Magentic manager.

Constructor

Python
<code>MagenticContext()</code>

Parameters

 Expand table

Name	Description
<code>task</code> Required*	
<code>chat_history</code>	Default value: <factory>
<code>participant_descriptions</code>	Default value: <factory>
<code>round_count</code>	Default value: 0
<code>stall_count</code>	Default value: 0
<code>reset_count</code>	Default value: 0

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	
<code>from_json</code>	
<code>reset</code>	Reset the context.

This will clear the chat history and reset the stall count. This will not reset the task, round count, or participant descriptions.

[to_dict](#)

[to_json](#)

`__init__`

Python

```
__init__(task: agent_framework._types.ChatMessage, chat_history:
list[agent_framework._types.ChatMessage] = <factory>, participant_descriptions:
dict[str, str] = <factory>, round_count: int = 0, stall_count: int = 0,
reset_count: int = 0) -> None
```

Parameters

[Expand table](#)

Name	Description
task Required*	ChatMessage
chat_history	list[ChatMessage] Default value: <factory>
participant_descriptions	dict[str, str] Default value: <factory>
round_count	int Default value: 0
stall_count	int Default value: 0
reset_count	int Default value: 0

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

clone

Python
<code>clone(*, deep: bool = True) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>deep</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>deep</code>	Default value: True

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

from_dict

Python
<code>from_dict(data: dict[str, Any]) -> MagenticContext</code>

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>MagenticContext</code>	

from_json

Python
<pre>from_json(raw: str) -> TModel</pre>

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

reset

Reset the context.

This will clear the chat history and reset the stall count. This will not reset the task, round count, or participant descriptions.

Python

```
reset() -> None
```

Returns

 Expand table

Type	Description
<code>None</code>	

to_dict

Python

```
to_dict() -> dict[str, Any]
```

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

to_json

Python

```
to_json() -> str
```

Returns

 Expand table

Type	Description
<code>str</code>	

Attributes

chat_history

Python

```
chat_history: list[ChatMessage]
```

participant_descriptions

Python

```
participant_descriptions: dict[str, str]
```

reset_count

Python

```
reset_count: int = 0
```

round_count

Python

```
round_count: int = 0
```

stall_count

Python

```
stall_count: int = 0
```

task

Python

```
task: ChatMessage
```

MagenticHumanInterventionDecision Enum

Decision options for human intervention responses.

Fields

 Expand table

APPROVE
CONTINUE
GUIDANCE
REJECT
REPLAN
REVISE

MagenticHumanInterventionKind Enum

The kind of human intervention being requested.

Fields

 Expand table

PLAN_REVIEW
STALL
TOOL_APPROVAL

MagenticManagerBase Class

Base class for the Magentic One manager.

Constructor

Python
<code>MagenticManagerBase()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>max_stall_count</code>	Default value: 3
<code>max_reset_count</code>	Default value: None
<code>max_round_count</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>create_progress_ledger</code>	Create a progress ledger.
<code>on_checkpoint_restore</code>	Restore runtime state from checkpoint data.
<code>on_checkpoint_save</code>	Serialize runtime state for checkpointing.
<code>plan</code>	Create a plan for the task.
<code>prepare_final_answer</code>	Prepare the final answer.
<code>replan</code>	Replan for the task.

`__init__`

Python

```
__init__(*, max_stall_count: int = 3, max_reset_count: int | None = None,
max_round_count: int | None = None) -> None
```

Parameters

 Expand table

Name	Description
max_stall_count Required*	<code>int</code>
max_reset_count Required*	<code>int</code> <code>None</code>
max_round_count Required*	<code>int</code> <code>None</code>

Keyword-Only Parameters

 Expand table

Name	Description
max_stall_count	Default value: 3
max_reset_count	Default value: None
max_round_count	Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

__new__

Python

```
__new__(**kwargs)
```

create_progress_ledger

Create a progress ledger.

Python

```
abstract async create_progress_ledger(magentic_context: MagenticContext) ->
_MagenticProgressLedger
```

Parameters

[Expand table](#)

Name	Description
magentic_context Required*	MagenticContext

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_magentic_MagenticProgressLedger>	

on_checkpoint_restore

Restore runtime state from checkpoint data.

Python

```
on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
state Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
None	

on_checkpoint_save

Serialize runtime state for checkpointing.

Python
<code>on_checkpoint_save() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
dict[str, Any]	

plan

Create a plan for the task.

Python
<code>abstract async plan(magentic_context: MagenticContext) -> ChatMessage</code>

Parameters

[Expand table](#)

Name	Description
<code>magentic_context</code> Required*	MagenticContext

Returns

[Expand table](#)

Type	Description
ChatMessage	

prepare_final_answer

Prepare the final answer.

Python
<pre>abstract async prepare_final_answer(magentic_context: MagenticContext) -> ChatMessage</pre>

Parameters

[Expand table](#)

Name	Description
magentic_context Required*	MagenticContext

Returns

[Expand table](#)

Type	Description
ChatMessage	

replan

Replan for the task.

Python
<pre>abstract async replan(magentic_context: MagenticContext) -> ChatMessage</pre>

Parameters

[Expand table](#)

Name	Description
magnetic_context Required*	MagneticContext

Returns

 Expand table

Type	Description
ChatMessage	

ManagerDirectiveModel Class

Pydantic model for structured manager directive output.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model ManagerDirectiveModel

Keyword-Only Parameters

 Expand table

Name	Description
next_agent	Default value: None
message	
finish	Default value: False
final_response	Default value: None

Methods

 Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
__new__	

construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
dict	
from_orm	
json	
model_construct	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][object.<i>dict</i>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.
model_json_schema	Generates a JSON schema for a model class.

<code>model_parametrized_name</code>	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i>. This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises `[ValidationError][pydantic_core.ValidationError]` if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

 Expand table

Name	Description
data Required*	Any

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

construct

Python
<code>construct(_fields_set: set[str] None = None, **values: Any) -> Self</code>

Parameters

 Expand table

Name	Description
_fields_set	set[str] None Default value: None
values Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python
<pre>dict(*, include: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, exclude: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool

Name	Description
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_orm

Python
from_orm(obj: Any) -> Self

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

 Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

 Expand table

Name	Description
_fields_set	set[str] None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

 Expand table

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [*dict*][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <code>to_python</code> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int</code> <code>None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool</code> <code>None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <code>round_trip</code> parameter instead.
round_trip Required*	<code>bool</code> If <code>True</code> , dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. <code>False</code> /"none" ignores them, <code>True</code> /"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .

Name	Description
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
                  schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
                  Literal['validation', 'serialization'] = 'validation', *, union_format:
                  Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>by_alias</code>	<code>bool</code> Whether to use attribute aliases or not. Default value: True
<code>ref_template</code>	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
<code>union_format</code> Required*	<code>Literal['any_of', 'primitive_type_array']</code> The format to use when combining schemas from unions together. Can be one of: <code>'any_of'</code>: Use the anyOf keyword to combine schemas (the default). <code>'primitive_type_array'</code>: Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
<code>schema_generator</code>	<code>type[<xref:pydantic.json_schema.GenerateJsonSchema>]</code> To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
<code>mode</code>	<code>Literal['validation', 'serialization']</code> The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>union_format</code>	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
<code>params</code> Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model</i> [<i>str</i> , <i>int</i>], the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<code>model_post_init(context: Any, /) -> None</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	

Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

 Expand table

Name	Description
force Required*	<code>bool</code> Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	<code>bool</code> Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<code><xref:MappingNamespace> None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

 Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
<code>_types_namespace</code>	Default value: <code>None</code>

Returns

[↗](#) Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[↗](#) Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.
by_alias	<code>bool</code> <code>None</code>

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>strict</code>	Default value: None
<code>extra</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>by_name</code>	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

[Expand table](#)

Name	Description
<code>path</code> Required*	str <xref:Path>
<code>content_type</code> Required*	str None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj	Any

Name	Description
Required*	

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

[↗](#) Expand table

Name	Description
<code>by_alias</code>	bool Default value: True
<code>ref_template</code>	str Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

[↗](#) Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
kwargs Required*	Any

Keyword-Only Parameters

[↗](#) Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
localns Required*	Any

Returns

 Expand table

Type	Description
None	

validate

Python
<pre>validate(value: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
value Required*	Any

Returns

 Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

final_response

Final answer to the user when finish is true

Python
<code>field final_response: str None = None</code>

finish

Whether the task is complete

Python
<code>field finish: bool = False</code>

message

Instruction for the selected participant

Python

```
field message: str = ''
```

next_agent

Name of the participant who should act next (null when finish is true)

Python

```
field next_agent: str | None = None
```

ManagerSelectionRequest Class

Request sent to manager agent for next speaker selection.

This dataclass packages the full conversation state and task context for the manager agent to analyze and make a speaker selection decision.

Constructor

Python
<code>ManagerSelectionRequest()</code>

Parameters

 Expand table

Name	Description
task Required*	
participants Required*	
conversation Required*	
round_index Required*	
metadata	Default value: None

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>to_dict</code>	Convert to dictionary for serialization.

__init__

Python

```
__init__(task: ChatMessage, participants: dict[str, str], conversation:
list[ChatMessage], round_index: int, metadata: dict[str, Any] | None = None) ->
None
```

Parameters

[Expand table](#)

Name	Description
task Required*	ChatMessage
participants Required*	dict[str, str]
conversation Required*	list[ChatMessage]
round_index Required*	int
metadata	dict[str, Any] None Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

to_dict

Convert to dictionary for serialization.

Python

```
to_dict() -> dict[str, Any]
```

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

Attributes

task

Original user task message

Python

```
task: ChatMessage
```

participants

Mapping of participant names to their descriptions

Python

```
participants: dict[str, str]
```

conversation

Full conversation history including all messages

Python

```
conversation: list[ChatMessage]
```

round_index

Number of manager selection rounds completed so far

Python
round_index: int

metadata

Optional metadata for extensibility

Python
metadata: dict[str, Any] None = None

ManagerSelectionResponse Class

Response from manager agent with speaker selection decision.

The manager agent must produce this structure (or compatible dict/JSON) to communicate its decision back to the orchestrator.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model ManagerSelectionResponse

Keyword-Only Parameters

 Expand table

Name	Description
selected_participant	Default value: None
instruction	Default value: None
finish	Default value: False
final_message	Default value: None

Methods

 Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.
--------------------------	--

	<i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
__new__	
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
dict	
from_dict	Create from dictionary representation.
from_orm	
get_final_message_as_chat_message	Convert final_message string to ChatMessage if present.
json	
model_construct	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][object.<i>dict</i>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties] [functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" model_dump

	Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModel.s.
model_post_init	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

 Expand table

Name	Description
data Required*	Any

Returns

 Expand table

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use `model_copy` instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

`copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self`

Parameters

[Expand table](#)

Name	Description
<code>include</code> Required*	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code> Optional set or mapping specifying which fields to include in the copied model.
<code>exclude</code> Required*	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code> Optional set or mapping specifying which fields to exclude in the copied model.

Name	Description
update Required*	<code>Dict[str, Any] None</code> Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	<code>bool</code> If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
<code>Self</code>	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_dict

Create from dictionary representation.

Python

```
static from_dict(data: dict[str, Any]) -> ManagerSelectionResponse
```

Parameters

[Expand table](#)

Name	Description
data Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
ManagerSelectionResponse	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

get_final_message_as_chat_message

Convert final_message string to ChatMessage if present.

Python
<code>get_final_message_as_chat_message() -> ChatMessage None</code>

Returns

[↗](#) Expand table

Type	Description
ChatMessage None	

json

Python
<code>json(*, include: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, exclude: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str</code>

Parameters

[↗](#) Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None

Name	Description
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>
exclude_defaults Required*	<code>bool</code>
exclude_none Required*	<code>bool</code>
encoder Required*	<code>Callable[[Any], Any]</code> <code>None</code>
models_as_dict Required*	<code>bool</code>
dumps_kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: <code>None</code>
exclude	Default value: <code>None</code>
by_alias	Default value: <code>False</code>
exclude_unset	Default value: <code>False</code>
exclude_defaults	Default value: <code>False</code>
exclude_none	Default value: <code>False</code>
encoder	Default value: <code>PydanticUndefined</code>
models_as_dict	Default value: <code>PydanticUndefined</code>

Returns

[Expand table](#)

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	set[str] None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
<code>values</code> Required*	Any Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.***dict***] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python
<code>model_copy(*, update: Mapping[str, Any] None = None, deep: bool = False) -> Self</code>

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
mode Required*	Literal ['json', 'python'] str The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set [int] set [str] Mapping [int, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool Mapping [str, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool None A set of fields to include in the output.
exclude Required*	set [int] set [str] Mapping [int, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool Mapping [str, set [int]] set [str] Mapping [int, IncEx] bool Mapping [str, IncEx] bool bool None

Name	Description
	A set of fields to exclude from the output.
context Required*	Any None Additional context to pass to the serializer.
by_alias Required*	bool None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	bool Literal ['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	Callable [[Any], Any] None A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None

Name	Description
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```

```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	int None Indentation to use in the JSON output. If <i>None</i> is passed, the output will be compact.
ensure_ascii Required*	bool If <i>True</i> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <i>False</i> (the default), these characters will be output as-is.
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None Field(s) to include in the JSON output.
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None Field(s) to exclude from the JSON output.
context Required*	Any None Additional context to pass to the serializer.
by_alias Required*	bool None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip	bool

Name	Description
Required*	If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
                  schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
                  Literal['validation', 'serialization'] = 'validation', *, union_format:
                  Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: <code>True</code>
ref_template	str The reference template. Default value: <code>DEFAULT_REF_TEMPLATE</code>
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>]

Name	Description
	To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: <code>GenerateJsonSchema</code>
mode	<code>Literal['validation', 'serialization']</code> The mode in which to generate the schema. Default value: <code>'validation'</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: <code>'any_of'</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	

Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
    _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True

Name	Description
<code>_parent_namespace_depth</code>	Default value: 2
<code>_types_namespace</code>	Default value: None

Returns

 Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal['allow', 'ignore', 'forbid']</code> <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.

Name	Description
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	<code>str bytes bytearray</code> The JSON data to validate.
strict Required*	<code>bool None</code> Whether to enforce types strictly.
extra Required*	<code>Literal['allow', 'ignore', 'forbid'] None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	<code>Any None</code> Extra variables to pass to the validator.
by_alias Required*	<code>bool None</code> Whether to use the field's alias when validating against the provided input data.
by_name Required*	<code>bool None</code> Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias	bool None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	<code>bool</code> <code>None</code> Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>strict</code>	Default value: <code>None</code>
<code>extra</code>	Default value: <code>None</code>
<code>context</code>	Default value: <code>None</code>
<code>by_alias</code>	Default value: <code>None</code>
<code>by_name</code>	Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>path</code> Required*	<code>str</code> <code><xref:Path></code>

Name	Description
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

parse_obj

Python
<pre>parse_obj(obj: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

[↗](#) Expand table

Name	Description
<code>by_alias</code>	bool Default value: True
<code>ref_template</code>	str Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
by_alias Required*	bool
ref_template Required*	str
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**kwargs: Any) -> None
```


Parameters

 Expand table

Name	Description
localns Required*	Any

Returns

 Expand table

Type	Description
None	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

 Expand table

Name	Description
value Required*	Any

Returns

 Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

 Expand table

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

 Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

selected_participant

Name of participant to speak next (None = finish conversation)

Python

```
field selected_participant: str | None = None
```

instruction

Optional instruction to provide to the selected participant

Python

```
field instruction: str | None = None
```

finish

Whether the conversation should be completed

Python

```
field finish: bool = False
```

final_message

Optional final message string when finishing conversation (will be converted to ChatMessage)

Python

```
field final_message: str | None = None
```

Message Class

A class representing a message in the workflow.

Constructor

Python
Message()

Parameters

 Expand table

Name	Description
data Required*	
source_id Required*	
target_id	Default value: None
type	Default value: MessageType.STANDARD
trace_contexts	Default value: None
source_span_ids	Default value: None
original_request	Default value: None

Methods

 Expand table

__init__	
__new__	
from_dict	Create a Message from a dictionary.
to_dict	Convert the Message to a dictionary for serialization.

`__init__`

Python

```
__init__(data: Any, source_id: str, target_id: str | None = None, type:
MessageType = MessageType.STANDARD, trace_contexts: list[dict[str, str]] | None =
None, source_span_ids: list[str] | None = None, original_request: Any = None) ->
None
```

Parameters

[Expand table](#)

Name	Description
data Required*	Any
source_id Required*	str
target_id	str None Default value: None
type	<xref:agent_framework._workflows._runner_context.MessageType> Default value: MessageType.STANDARD
trace_contexts	list[dict[str, str]] None Default value: None
source_span_ids	list[str] None Default value: None
original_request	Any Default value: None

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

from_dict

Create a Message from a dictionary.

Python

```
static from_dict(data: dict[str, Any]) -> Message
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>Message</code>	

to_dict

Convert the Message to a dictionary for serialization.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Attributes

source_span_id

Get the first source span ID for backward compatibility.

trace_context

Get the first trace context for backward compatibility.

data

Python

data: Any

original_request

Python

original_request: Any = None

source_id

Python

source_id: str

source_span_ids

Python

source_span_ids: list[str] | None = None

target_id

Python

target_id: str | None = None

trace_contexts

Python

```
trace_contexts: list[dict[str, str]] | None = None
```

type

Python

```
type: MessageType = 'standard'
```


OrchestrationState Class

Unified state container for orchestrator checkpointing.

This dataclass standardizes checkpoint serialization across all three group chat patterns while allowing pattern-specific extensions via metadata.

Common attributes cover shared orchestration concerns (task, conversation, round tracking). Pattern-specific state goes in the metadata dict.

Constructor

Python
<code>OrchestrationState()</code>

Parameters

[Expand table](#)

Name	Description
<code>conversation</code>	Default value: <factory>
<code>round_index</code>	Default value: 0
<code>metadata</code>	Default value: <factory>
<code>task</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>from_dict</code>	Deserialize from checkpointed dict.
<code>to_dict</code>	Serialize to dict for checkpointing.

`__init__`

Python

```
__init__(conversation: list[agent_framework._types.ChatMessage] = <factory>,
round_index: int = 0, metadata: dict[str, typing.Any] = <factory>, task:
agent_framework._types.ChatMessage | None = None) -> None
```

Parameters

 Expand table

Name	Description
conversation	<code>list[ChatMessage]</code> Default value: <factory>
round_index	<code>int</code> Default value: 0
metadata	<code>dict[str, Any]</code> Default value: <factory>
task	<code>ChatMessage None</code> Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Deserialize from checkpointed dict.

Python

```
from_dict(data: dict[str, Any]) -> OrchestrationState
```

Parameters

 Expand table

Name	Description
data Required*	<code>dict[str, Any]</code> Checkpoint data with encoded conversation

Returns

 Expand table

Type	Description
<code>OrchestrationState</code>	Restored OrchestrationState instance

to_dict

Serialize to dict for checkpointing.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dict with encoded conversation and metadata for persistence

Attributes

conversation

Full conversation history (all messages)

Python
<code>conversation: list[ChatMessage]</code>

round_index

Number of coordination rounds completed (0 if not tracked)

Python

```
round_index: int = 0
```

metadata

Extensible dict for pattern-specific state

Python

```
metadata: dict[str, Any]
```

task

Optional primary task/question being orchestrated

Python

```
task: ChatMessage | None = None
```

RequestInfoEvent Class

Event triggered when a workflow executor requests external information.

Initialize the request info event.

Constructor

Python
<code>RequestInfoEvent()</code>

Parameters

 Expand table

Name	Description
<code>request_id</code> Required*	Unique identifier for the request.
<code>source_executor_id</code> Required*	ID of the executor that made the request.
<code>request_data</code> Required*	The data associated with the request.
<code>response_type</code> Required*	Expected type of the response.

Methods

 Expand table

<code>__init__</code>	Initialize the request info event.
<code>__new__</code>	
<code>from_dict</code>	Create a RequestInfoEvent from a dictionary.
<code>to_dict</code>	Convert the request info event to a dictionary for serialization.

`__init__`

Initialize the request info event.

Python

```
__init__(request_id: str, source_executor_id: str, request_data: Any, response_type: type[Any])
```

Parameters

 Expand table

Name	Description
request_id Required*	str Unique identifier for the request.
source_executor_id Required*	str ID of the executor that made the request.
request_data Required*	Any The data associated with the request.
response_type Required*	type[Any] Expected type of the response.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create a RequestInfoEvent from a dictionary.

Python

```
static from_dict(data: dict[str, Any]) -> RequestInfoEvent
```

Parameters

 Expand table

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code>RequestInfoEvent</code>	

to_dict

Convert the request info event to a dictionary for serialization.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

RequestInfoInterceptor Class

Internal executor that pauses workflow for human input before agent runs.

This executor is inserted into the workflow graph by builders when `.with_request_info()` is called. It intercepts `AgentExecutorRequest` messages BEFORE the agent runs and pauses the workflow via `ctx.request_info()` with an `AgentInputRequest`.

When a response is received, the response handler injects the input as a user message into the conversation and forwards the request to the agent.

The optional `agent_filter` parameter allows limiting which agents trigger the pause. If the target agent's ID is not in the filter set, the request is forwarded without pausing.

Initialize the request info interceptor executor.

Constructor

Python
<code>RequestInfoInterceptor()</code>

Parameters

 Expand table

Name	Description
<code>executor_id</code>	ID for this executor. If None, generates a unique ID using the format "request_info_interceptor-<uuid4>". Default value: None
<code>agent_filter</code>	Optional set of agent/executor IDs to filter on. If provided, only requests to these agents trigger a pause. If None (default), all requests trigger a pause. Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the request info interceptor executor.
---------------------------------------	---

__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters. • Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using <code>@handler</code> decorator.
from_dict	
from_json	
handle_input_response	Handle the human input and forward the modified request to the agent. Injects the response as a user message into the conversation and forwards the modified request to the agent. TODO: Consider having each orchestration implement its own response handler for more specialized behavior.
intercept_agent_request	Intercept request before agent runs and pause for human input. Pauses the workflow and emits a <code>RequestInfoEvent</code> with the current conversation for steering. If an agent filter is configured and this agent is not in the filter, the request is forwarded without pausing.
intercept_concurrent_requests	Intercept requests before concurrent agents run. This handler is used by <code>ConcurrentBuilder</code> to get human input before all parallel agents execute.
intercept_conversation	Intercept conversation before agent runs (used by <code>SequentialBuilder</code>). <code>SequentialBuilder</code> passes <code>list[ChatMessage]</code> directly to agents. This handler intercepts that flow and pauses for human input.
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
on_checkpoint_save	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

<code>to_dict</code>	Serialize executor definition for workflow topology export.
<code>to_json</code>	

`__init__`

Initialize the request info interceptor executor.

Python

```
__init__(executor_id: str | None = None, agent_filter: set[str] | None = None) -> None
```

Parameters

 Expand table

Name	Description
<code>executor_id</code>	<code>str</code> <code>None</code> ID for this executor. If <code>None</code> , generates a unique ID using the format "request_info_interceptor- <code><uuid4></code> ". Default value: <code>None</code>
<code>agent_filter</code>	<code>set[str]</code> <code>None</code> Optional set of agent/executor IDs to filter on. If provided, only requests to these agents trigger a pause. If <code>None</code> (default), all requests trigger a pause. Default value: <code>None</code>

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

can_handle

Check if the executor can handle a given message type.

Python

```
can_handle(message: Message) -> bool
```

Parameters

 Expand table

Name	Description
message Required*	Message The message to check.

Returns

 Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

 Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using `@handler` decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.

Name	Description
<code>trace_contexts</code>	<code>list[dict[str, str]] None</code> Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
<code>source_span_ids</code>	<code>list[str] None</code> Optional source span IDs from multiple sources for linking. Default value: None

Returns

[Expand table](#)

Type	Description
<code>None</code>	An awaitable that resolves to the result of the execution.

from_dict

Python
<pre>from_dict(data: dict[str, Any]) -> TModel</pre>

Parameters

[Expand table](#)

Name	Description
<code>data</code> Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

from_json

--

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	str

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

handle_input_response

Handle the human input and forward the modified request to the agent.

Injects the response as a user message into the conversation and forwards the modified request to the agent.

TODO: Consider having each orchestration implement its own response handler for more specialized behavior.

Python

```
async handle_input_response(original_request: AgentInputRequest, response: str,
ctx: WorkflowContext[AgentExecutorRequest | list[ChatMessage], Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
original_request Required*	AgentInputRequest The AgentInputRequest that triggered the pause

Name	Description
response Required*	str The human input text
ctx Required*	WorkflowContext[AgentExecutorRequest list[ChatMessage], Any] Workflow context for continuing the workflow

Returns

[Expand table](#)

Type	Description
None	

intercept_agent_request

Intercept request before agent runs and pause for human input.

Pauses the workflow and emits a RequestInfoEvent with the current conversation for steering. If an agent filter is configured and this agent is not in the filter, the request is forwarded without pausing.

Python

```
async intercept_agent_request(request: AgentExecutorRequest, ctx:
WorkflowContext[AgentExecutorRequest, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
request Required*	AgentExecutorRequest The request about to be sent to the agent
ctx Required*	WorkflowContext[AgentExecutorRequest, Any] Workflow context for requesting info

Returns

[Expand table](#)

Type	Description
None	

intercept_concurrent_requests

Intercept requests before concurrent agents run.

This handler is used by ConcurrentBuilder to get human input before all parallel agents execute.

Python

```
async intercept_concurrent_requests(requests: list[AgentExecutorRequest], ctx: WorkflowContext[list[AgentExecutorRequest], Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
requests Required*	list[AgentExecutorRequest] List of requests for all concurrent agents
ctx Required*	WorkflowContext[list[AgentExecutorRequest], Any] Workflow context for requesting info

Returns

[Expand table](#)

Type	Description
None	

intercept_conversation

Intercept conversation before agent runs (used by SequentialBuilder).

SequentialBuilder passes list[ChatMessage] directly to agents. This handler intercepts that flow and pauses for human input.

Python


```
async intercept_conversation(messages: list[ChatMessage], ctx:
WorkflowContext[list[ChatMessage], Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
messages Required*	list[ChatMessage] The conversation about to be sent to the agent
ctx Required*	WorkflowContext [list[ChatMessage] , Any] Workflow context for requesting info

Returns

[Expand table](#)

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

[Expand table](#)

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>state</code> Required*	<code>dict[str, Any]</code> The state dictionary that was saved during checkpointing.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

Python

```
to_dict() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Python

```
to_json() -> str
```

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

[Expand table](#)

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via `send_message()`.

Returns

[Expand table](#)

Type	Description
	A list of the output types inferred from the handlers' <code>WorkflowContext[T]</code> annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via `yield_output()`.

Returns

[Expand table](#)

Type	Description
	A list of the workflow output types inferred from handlers' <code>WorkflowContext[T, U]</code> annotations.

Role Class

Describes the intended purpose of a message within a chat interaction.

Properties: **SYSTEM**: The role that instructs or sets the behavior of the AI system. **USER**: The role that provides user input for chat interactions. **ASSISTANT**: The role that provides responses to system-instructed, user-prompted input. **TOOL**: The role that provides additional information and references in response to tool use requests.

Initialize Role with a value.

Constructor

Python
<code>Role()</code>

Parameters

 Expand table

Name	Description
value Required*	The string representation of the role.

Examples

Python
<pre>from agent_framework import Role # Use predefined role constants system_role = Role.SYSTEM user_role = Role.USER assistant_role = Role.ASSISTANT tool_role = Role.TOOL # Create custom role custom_role = Role(value="custom") # Compare roles</pre>

```
print(system_role == Role.SYSTEM) # True
print(system_role.value) # "system"
```

Methods

 Expand table

<code>__init__</code>	Initialize Role with a value.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with '_').
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize Role with a value.

Python

```
__init__(value: str) -> None
```

Parameters

[Expand table](#)

Name	Description
value Required*	<code>str</code> The string representation of the role.

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

value

The string representation of the role.

ASSISTANT

Python

```
ASSISTANT: Role = Role(value='assistant')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

--

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

SYSTEM

Python

```
SYSTEM: Role = Role(value='system')
```

TOOL

Python

```
TOOL: Role = Role(value='tool')
```

USER

Python

```
USER: Role = Role(value='user')
```

Runner Class

A class to run a workflow in Pregel supersteps.

Initialize the runner with edges, shared state, and context.

Constructor

Python
Runner()

Parameters

 Expand table

Name	Description
edge_groups Required*	The edge groups of the workflow.
executors Required*	Map of executor IDs to executor instances.
shared_state Required*	The shared state for the workflow.
ctx Required*	The runner context for the workflow.
max_iterations	The maximum number of iterations to run. Default value: 100
workflow_id	The workflow ID for checkpointing. Default value: None

Methods

 Expand table

__init__	Initialize the runner with edges, shared state, and context.
__new__	

reset_iteration_count	Reset the iteration count to zero.
restore_from_checkpoint	Restore workflow state from a checkpoint.
run_until_convergence	Run the workflow until no more messages are sent.

`__init__`

Initialize the runner with edges, shared state, and context.

Python

```
__init__(edge_groups: Sequence[EdgeGroup], executors: dict[str, Executor],
shared_state: SharedState, ctx: RunnerContext, max_iterations: int = 100,
workflow_id: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
edge_groups Required*	Sequence [<xref:agent_framework_workflows_edge.EdgeGroup>] The edge groups of the workflow.
executors Required*	dict [str , Executor] Map of executor IDs to executor instances.
shared_state Required*	SharedState The shared state for the workflow.
ctx Required*	RunnerContext The runner context for the workflow.
max_iterations	int The maximum number of iterations to run. Default value: 100
workflow_id	str None The workflow ID for checkpointing. Default value: None

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

reset_iteration_count

Reset the iteration count to zero.

Python
<code>reset_iteration_count() -> None</code>

Returns

[Expand table](#)

Type	Description
None	

restore_from_checkpoint

Restore workflow state from a checkpoint.

Python
<code>async restore_from_checkpoint(checkpoint_id: str, checkpoint_storage: CheckpointStorage None = None) -> bool</code>

Parameters

[Expand table](#)

Name	Description
checkpoint_id Required*	<code>str</code> The ID of the checkpoint to restore from

Name	Description
<code>checkpoint_storage</code>	CheckpointStorage None Optional storage to load checkpoints from when the runner context itself is not configured with checkpointing. Default value: None

Returns

[Expand table](#)

Type	Description
bool	True if restoration was successful, False otherwise

run_until_convergence

Run the workflow until no more messages are sent.

Python
<pre>async run_until_convergence() -> AsyncGenerator[WorkflowEvent, None]</pre>

Returns

[Expand table](#)

Type	Description
AsyncGenerator[WorkflowEvent, None]	

Attributes

context

Get the workflow context.

RunnerContext Class

Protocol for the execution context used by the runner.

A single context that supports messaging, events, and optional checkpointing. If checkpoint storage is not configured, checkpoint methods may raise.

Constructor

Python
<code>RunnerContext()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>add_event</code>	Add an event to the execution context.
<code>add_request_info_event</code>	Add a RequestInfoEvent to the context and track it for correlation.
<code>apply_checkpoint</code>	Apply a checkpoint to the current context, mutating its state.
<code>clear_runtime_checkpoint_storage</code>	Clear runtime checkpoint storage override.
<code>create_checkpoint</code>	Create a checkpoint of the current workflow state.
<code>drain_events</code>	Drain all events from the context.
<code>drain_messages</code>	Drain all messages from the context.
<code>get_pending_request_info_events</code>	Get the mapping of request IDs to their corresponding RequestInfoEvent.
<code>has_checkpointing</code>	Check if the context supports checkpointing.
<code>has_events</code>	Check if there are any events in the context.
<code>has_messages</code>	Check if there are any messages in the context.
<code>is_streaming</code>	Check if the workflow is in streaming mode.

<code>load_checkpoint</code>	Load a checkpoint without mutating the current context state.
<code>next_event</code>	Wait for and return the next event emitted by the workflow run.
<code>reset_for_new_run</code>	Reset the context for a new workflow run.
<code>send_message</code>	Send a message from the executor to the context.
<code>send_request_info_response</code>	Send a response correlated to a pending request.
<code>set_runtime_checkpoint_storage</code>	Set runtime checkpoint storage to override build-time configuration.
<code>set_streaming</code>	Set whether agents should stream incremental updates.
<code>set_workflow_id</code>	Set the workflow ID for the context.

`__init__`

Python
<code>__init__(*args, **kwargs)</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`add_event`

Add an event to the execution context.

Python
<code>async add_event(event: WorkflowEvent) -> None</code>

Parameters

[Expand table](#)

Name	Description
<code>event</code> Required*	<code>WorkflowEvent</code> The event to be added.

Returns

 Expand table

Type	Description
None	

add_request_info_event

Add a RequestInfoEvent to the context and track it for correlation.

Python
<pre>async add_request_info_event(event: RequestInfoEvent) -> None</pre>

Parameters

 Expand table

Name	Description
event Required*	RequestInfoEvent The RequestInfoEvent to be added.

Returns

 Expand table

Type	Description
None	

apply_checkpoint

Apply a checkpoint to the current context, mutating its state.

Python
<pre>async apply_checkpoint(checkpoint: WorkflowCheckpoint) -> None</pre>

Parameters

[Expand table](#)

Name	Description
checkpoint Required*	WorkflowCheckpoint The checkpoint whose state is to be applied.

Returns

[Expand table](#)

Type	Description
None	

clear_runtime_checkpoint_storage

Clear runtime checkpoint storage override.

Python

```
clear_runtime_checkpoint_storage() -> None
```

Returns

[Expand table](#)

Type	Description
None	

create_checkpoint

Create a checkpoint of the current workflow state.

Python

```
async create_checkpoint(shared_state: SharedState, iteration_count: int, metadata: dict[str, Any] | None = None) -> str
```

Parameters

[Expand table](#)

Name	Description
shared_state Required*	SharedState The shared state to include in the checkpoint. This is needed to capture the full state of the workflow. The shared state is not managed by the context itself.
iteration_count Required*	int The current iteration count of the workflow.
metadata	dict[str, Any] None Optional metadata to associate with the checkpoint. Default value: None

Returns

[Expand table](#)

Type	Description
str	The ID of the created checkpoint.

drain_events

Drain all events from the context.

Python

```
async drain_events() -> list[WorkflowEvent]
```

Returns

[Expand table](#)

Type	Description
list[WorkflowEvent]	A list of events that were added to the context.

drain_messages

Drain all messages from the context.

Python

```
async drain_messages() -> dict[str, list[Message]]
```


Returns

[Expand table](#)

Type	Description
<code>dict[str, list[Message]]</code>	A dictionary mapping executor IDs to lists of messages.

get_pending_request_info_events

Get the mapping of request IDs to their corresponding RequestInfoEvent.

Python

```
async get_pending_request_info_events() -> dict[str, RequestInfoEvent]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, RequestInfoEvent]</code>	A dictionary mapping request IDs to their corresponding RequestInfoEvent.

has_checkpointing

Check if the context supports checkpointing.

Python

```
has_checkpointing() -> bool
```

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if checkpointing is supported, False otherwise.

has_events

Check if there are any events in the context.

Python

```
async has_events() -> bool
```

Returns

[Expand table](#)

Type	Description
bool	True if there are events, False otherwise.

has_messages

Check if there are any messages in the context.

Python

```
async has_messages() -> bool
```

Returns

[Expand table](#)

Type	Description
bool	True if there are messages, False otherwise.

is_streaming

Check if the workflow is in streaming mode.

Python

```
is_streaming() -> bool
```

Returns

[Expand table](#)

Type	Description
bool	True if streaming mode is enabled, False otherwise.

load_checkpoint

Load a checkpoint without mutating the current context state.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

 Expand table

Name	Description
checkpoint_id Required*	<code>str</code> The ID of the checkpoint to load.

Returns

 Expand table

Type	Description
<code>WorkflowCheckpoint</code> <code>None</code>	The loaded checkpoint, or None if it does not exist.

next_event

Wait for and return the next event emitted by the workflow run.

Python

```
async next_event() -> WorkflowEvent
```

Returns

 Expand table

Type	Description
<code>WorkflowEvent</code>	

reset_for_new_run

Reset the context for a new workflow run.

Python
<code>reset_for_new_run()</code> -> <code>None</code>

Returns

 Expand table

Type	Description
<code>None</code>	

send_message

Send a message from the executor to the context.

Python
<code>async send_message(message: Message)</code> -> <code>None</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	<code>Message</code> The message to be sent.

Returns

 Expand table

Type	Description
<code>None</code>	

send_request_info_response

Send a response correlated to a pending request.

--

Python

```
async send_request_info_response(request_id: str, response: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
request_id Required*	str The ID of the original request.
response Required*	Any The response data to be sent.

Returns

[Expand table](#)

Type	Description
None	

set_runtime_checkpoint_storage

Set runtime checkpoint storage to override build-time configuration.

Python

```
set_runtime_checkpoint_storage(storage: CheckpointStorage) -> None
```

Parameters

[Expand table](#)

Name	Description
storage Required*	CheckpointStorage The checkpoint storage to use for this run.

Returns

[Expand table](#)

Type	Description
None	

set_streaming

Set whether agents should stream incremental updates.

Python
<code>set_streaming(streaming: bool) -> None</code>

Parameters

[Expand table](#)

Name	Description
streaming Required*	<code>bool</code> True for streaming mode (run_stream), False for non-streaming (run).

Returns

[Expand table](#)

Type	Description
None	

set_workflow_id

Set the workflow ID for the context.

Python
<code>set_workflow_id(workflow_id: str) -> None</code>

Parameters

[Expand table](#)

Name	Description
<code>workflow_id</code> Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>None</code>	

SequentialBuilder Class

High-level builder for sequential agent/executor workflows with shared context.

- *participants(...)* accepts a list of AgentProtocol (recommended) or Executor instances
- *register_participants(...)* accepts a list of factories for AgentProtocol (recommended) or Executor factories
- Executors must define a handler that consumes list[ChatMessage] and sends out a list[ChatMessage]
- The workflow wires participants in order, passing a list[ChatMessage] down the chain
- Agents append their assistant messages to the conversation
- Custom executors can transform/summarize and return a list[ChatMessage]
- The final output is the conversation produced by the last participant

Usage:

Python

```
from agent_framework import SequentialBuilder

# With agent instances
workflow = SequentialBuilder().participants([agent1, agent2,
summarizer_exec]).build()

# With agent factories
workflow = (
    SequentialBuilder().register_participants([create_agent1, create_agent2,
create_summarizer_exec]).build()
)

# Enable checkpoint persistence
workflow = SequentialBuilder().participants([agent1,
agent2]).with_checkpointing(storage).build()

# Enable request info for mid-workflow feedback (pauses before each agent)
workflow = SequentialBuilder().participants([agent1,
agent2]).with_request_info().build()

# Enable request info only for specific agents
workflow = (
    SequentialBuilder()
    .participants([agent1, agent2, agent3])
    .with_request_info(agents=[agent2]) # Only pause before agent2
```



```
.build()  
)
```

Constructor

Python
SequentialBuilder()

Methods

 Expand table

__init__	
__new__	
build	Build and validate the sequential workflow. Wiring pattern: • <code>_InputToConversation</code> normalizes the initial input into <code>list[ChatMessage]</code> • For each participant in order: ◦ If Agent (or AgentExecutor): pass conversation to the agent, then optionally route through a request info interceptor, then convert response to conversation via <code>_ResponseToConversation</code> ◦ Else (custom Executor): pass conversation directly to the executor • <code>_EndWithConversation</code> yields the final conversation and the workflow becomes idle
participants	Define the ordered participants for this sequential workflow. Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Raises if empty or duplicates are provided for clarity.
register_participants	Register participant factories for this sequential workflow.
with_checkpointing	Enable checkpointing for the built workflow using the provided storage.
with_request_info	Enable request info before agents run in the workflow. When enabled, the workflow pauses before each agent runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the agent responds. The caller provides input via the standard response_handler/request_info pattern. Example:

Python

```
# Pause before all agents
workflow = SequentialBuilder().participants([a1,
a2]).with_request_info().build()

# Pause only before specific agents
workflow = (
    SequentialBuilder()
    .participants([drafter, reviewer, finalizer])
    .with_request_info(agents=[reviewer]) # Only pause before
reviewer
    .build()
)
```

__init__

Python

```
__init__() -> None
```

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

build

Build and validate the sequential workflow.

Wiring pattern:

- `_InputToConversation` normalizes the initial input into `list[ChatMessage]`
- For each participant in order:

- If Agent (or AgentExecutor): pass conversation to the agent, then optionally route through a request info interceptor, then convert response to conversation via `_ResponseToConversation`
- Else (custom Executor): pass conversation directly to the executor
- `_EndWithConversation` yields the final conversation and the workflow becomes idle

Python

```
build() -> Workflow
```

Returns

[Expand table](#)

Type	Description
Workflow	

participants

Define the ordered participants for this sequential workflow.

Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances.

Raises if empty or duplicates are provided for clarity.

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) ->
SequentialBuilder
```

Parameters

[Expand table](#)

Name	Description
participants Required*	Sequence[AgentProtocol Executor]

Returns

[Expand table](#)

Type	Description
SequentialBuilder	

register_participants

Register participant factories for this sequential workflow.

Python
<pre>register_participants(participant_factories: Sequence[Callable[[], AgentProtocol Executor]]) -> SequentialBuilder</pre>

Parameters

[Expand table](#)

Name	Description
participant_factories Required*	Sequence[Callable[[], AgentProtocol Executor]]

Returns

[Expand table](#)

Type	Description
SequentialBuilder	

with_checkpointing

Enable checkpointing for the built workflow using the provided storage.

Python
<pre>with_checkpointing(checkpoint_storage: CheckpointStorage) -> SequentialBuilder</pre>

Parameters

[Expand table](#)

Name	Description
<code>checkpoint_storage</code> Required*	CheckpointStorage

Returns

 Expand table

Type	Description
SequentialBuilder	

with_request_info

Enable request info before agents run in the workflow.

When enabled, the workflow pauses before each agent runs, emitting a `RequestInfoEvent` that allows the caller to review the conversation and optionally inject guidance before the agent responds. The caller provides input via the standard `response_handler/request_info` pattern.

Example:

Python

```
# Pause before all agents
workflow = SequentialBuilder().participants([a1,
a2]).with_request_info().build()

# Pause only before specific agents
workflow = (
    SequentialBuilder()
    .participants([drafter, reviewer, finalizer])
    .with_request_info(agents=[reviewer]) # Only pause before reviewer
    .build()
)
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None =
None) -> SequentialBuilder
```

Parameters

[Expand table](#)

Name	Description
agents Required*	Sequence [str AgentProtocol Executor] None Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every agent.

Keyword-Only Parameters

[Expand table](#)

Name	Description
agents	Default value: None

Returns

[Expand table](#)

Type	Description
<xref:self>	The builder instance for fluent chaining.

SharedState Class

A class to manage shared state in a workflow.

SharedState provides thread-safe access to workflow state data that needs to be shared across executors during workflow execution.

Reserved Keys: The following keys are reserved for internal framework use and should not be modified by user code:

- `_executor_state`: Stores executor state for checkpointing (managed by Runner)

Warning

Do not use keys starting with underscore (`_`) as they may be reserved for internal framework operations.

Initialize the shared state.

Constructor

Python
<code>SharedState()</code>

Methods

 Expand table

<code>__init__</code>	Initialize the shared state.
<code>__new__</code>	
<code>clear</code>	Clear the entire shared state.
<code>delete</code>	Delete a key from the shared state.
<code>delete_within_hold</code>	Delete a key without acquiring the lock (unsafe - use within <code>hold()</code> context).
<code>export_state</code>	Get a serialized copy of the entire shared state.

<code>get</code>	Get a value from the shared state.
<code>get_within_hold</code>	Get a value without acquiring the lock (unsafe - use within <code>hold()</code> context).
<code>has</code>	Check if a key exists in the shared state.
<code>has_within_hold</code>	Check if a key exists without acquiring the lock (unsafe - use within <code>hold()</code> context).
<code>hold</code>	Context manager to hold the shared state lock for multiple operations. Usage: <code>async with shared_state.hold(): await shared_state.set_within_hold("key", value)</code> <code>value = await shared_state.get_within_hold("key")</code>
<code>import_state</code>	Populate the shared state from a serialized state dictionary. This replaces the entire current state with the provided state.
<code>set</code>	Set a value in the shared state.
<code>set_within_hold</code>	Set a value without acquiring the lock (unsafe - use within <code>hold()</code> context).

`__init__`

Initialize the shared state.

Python

```
__init__() -> None
```

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

`clear`

Clear the entire shared state.

Python

```
async clear() -> None
```

Returns

[Expand table](#)

Type	Description
None	

delete

Delete a key from the shared state.

Python

```
async delete(key: str) -> None
```

Parameters

[Expand table](#)

Name	Description
key Required*	str

Returns

[Expand table](#)

Type	Description
None	

delete_within_hold

Delete a key without acquiring the lock (unsafe - use within hold() context).

Python

```
async delete_within_hold(key: str) -> None
```

Parameters

[Expand table](#)

Name	Description
key Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

export_state

Get a serialized copy of the entire shared state.

Python

```
async export_state() -> dict[str, Any]
```

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

get

Get a value from the shared state.

Python

```
async get(key: str) -> Any
```

Parameters

 Expand table

Name	Description
key Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>Any</code>	

get_within_hold

Get a value without acquiring the lock (unsafe - use within hold() context).

Python
<code>async get_within_hold(key: str) -> Any</code>

Parameters

 Expand table

Name	Description
key Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>Any</code>	

has

Check if a key exists in the shared state.

Python

```
async has(key: str) -> bool
```

Parameters

 Expand table

Name	Description
key Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>bool</code>	

has_within_hold

Check if a key exists without acquiring the lock (unsafe - use within hold() context).

Python

```
async has_within_hold(key: str) -> bool
```

Parameters

 Expand table

Name	Description
key Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code>bool</code>	

hold

Context manager to hold the shared state lock for multiple operations.

Usage: `async with shared_state.hold(): await shared_state.set_within_hold("key", value) value = await shared_state.get_within_hold("key")`

Python
<code>hold() -> AsyncIterator[SharedState]</code>

Returns

[Expand table](#)

Type	Description
<code>Asyncclterator[SharedState]</code>	

import_state

Populate the shared state from a serialized state dictionary.

This replaces the entire current state with the provided state.

Python
<code>async import_state(state: dict[str, Any]) -> None</code>

Parameters

[Expand table](#)

Name	Description
state Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

set

Set a value in the shared state.

Python
<pre>async set(key: str, value: Any) -> None</pre>

Parameters

[Expand table](#)

Name	Description
key Required*	<code>str</code>
value Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

set_within_hold

Set a value without acquiring the lock (unsafe - use within hold() context).

Python
<pre>async set_within_hold(key: str, value: Any) -> None</pre>

Parameters

 Expand table

Name	Description
key Required*	str
value Required*	Any

Returns

 Expand table

Type	Description
None	

SingleEdgeGroup Class

Convenience wrapper for a solitary edge, keeping the group API uniform.

Create a one-to-one edge group between two executors.

Constructor

Python

```
SingleEdgeGroup()
```

Parameters

[Expand table](#)

Name	Description
source_id Required*	
target_id Required*	
condition	Default value: None

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	Default value: None

Examples

Python

```
group = SingleEdgeGroup("ingest", "validate")
assert group.edges[0].source_id == "ingest"
```


Methods

[Expand table](#)

<code>__init__</code>	Create a one-to-one edge group between two executors.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Hydrate the correct <i>EdgeGroup</i> subclass from serialised state. The method inspects the <i>type</i> field, allocates the corresponding class without executing subclass <i>init</i>, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
<code>from_json</code>	
<code>register</code>	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their <i>name</i>, which must therefore remain stable across versions when persisted workflows are in circulation.
<code>to_dict</code>	Serialise the group metadata and contained edges into primitives. The payload captures each edge through its own <i>to_dict</i> call, enabling round-tripping through formats such as JSON without leaking Python objects.
<code>to_json</code>	

`__init__`

Create a one-to-one edge group between two executors.

Python

```
__init__(source_id: str, target_id: str, condition: Callable[[Any], bool] | None = None, *, id: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>source_id</code> Required*	<code>str</code>

Name	Description
target_id Required*	str
condition	Callable[[Any], bool] None Default value: None
id Required*	str None

Keyword-Only Parameters

[⌵](#) Expand table

Name	Description
id	Default value: None

Returns

[⌵](#) Expand table

Type	Description
None	

Examples

Python

```
group = SingleEdgeGroup("ingest", "validate")
assert group.edges[0].source_id == "ingest"
```

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

 Expand table

Name	Description
deep Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
deep	Default value: True

Returns

 Expand table

Type	Description
<code>Self</code>	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass *init*, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._edge.EdgeGroup></code>	

Examples

Python

```
payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":  
"b"}]}  
group = EdgeGroup.from_dict(payload)  
assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python
<pre>register(subclass: type[EdgeGroup]) -> type[EdgeGroup]</pre>

Parameters

[Expand table](#)

Name	Description
subclass Required*	type [<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

[Expand table](#)

Type	Description
type [<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python
<pre>@EdgeGroup.register class CustomGroup(EdgeGroup): pass assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup</pre>

to_dict

Serialise the group metadata and contained edges into primitives.

The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[\[\]](#) Expand table

Type	Description
<code>dict[str, Any]</code>	

Examples

Python
<pre>group = EdgeGroup([Edge("read", "write")]) snapshot = group.to_dict() assert snapshot["edges"][0]["source_id"] == "read"</pre>

to_json

Python
<code>to_json() -> str</code>

Returns

[\[\]](#) Expand table

Type	Description
<code>str</code>	

Attributes

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

type

Python

type: str

StandardMagenticManager Class

Standard Magentic manager that performs real LLM calls via a ChatAgent.

The manager constructs prompts that mirror the original Magentic One orchestration:

- Facts gathering
- Plan creation
- Progress ledger in JSON
- Facts update and plan update on reset
- Final answer synthesis

Initialize the Standard Magentic Manager.

Constructor

Python
<pre>StandardMagenticManager()</pre>

Parameters

 Expand table

Name	Description
agent Required*	An agent instance to use for LLM calls. The agent's configured options (temperature, seed, instructions, etc.) will be applied.
task_ledger	Optional task ledger for managing task state. Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
task_ledger_facts_prompt	Optional prompt for the task ledger facts. Default value: None
task_ledger_plan_prompt	Optional prompt for the task ledger plan. Default value: None

Name	Description
<code>task_ledger_full_prompt</code>	Optional prompt for the full task ledger. Default value: None
<code>task_ledger_facts_update_prompt</code>	Optional prompt for updating task ledger facts. Default value: None
<code>task_ledger_plan_update_prompt</code>	Optional prompt for updating task ledger plan. Default value: None
<code>progress_ledger_prompt</code>	Optional prompt for the progress ledger. Default value: None
<code>final_answer_prompt</code>	Optional prompt for the final answer. Default value: None
<code>max_stall_count</code>	Maximum number of stalls allowed. Default value: 3
<code>max_reset_count</code>	Maximum number of resets allowed. Default value: None
<code>max_round_count</code>	Maximum number of rounds allowed. Default value: None
<code>progress_ledger_retry_count</code>	Maximum number of retries for the progress ledger. Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the Standard Magentic Manager.
<code>__new__</code>	
<code>create_progress_ledger</code>	Use the model to produce a JSON progress ledger based on the conversation so far. Adds lightweight retries with backoff for transient parse issues and avoids selecting a non-existent "unknown" agent. If there are no participants, a clear error is raised.
<code>on_checkpoint_restore</code>	
<code>on_checkpoint_save</code>	
<code>plan</code>	Create facts and plan using the model, then render a combined task ledger as a single assistant message.

prepare_final_answer	Ask the model to produce the final answer addressed to the user.
replan	Update facts and plan when stalling or looping has been detected.

`__init__`

Initialize the Standard Magentic Manager.

Python

```
__init__(agent: AgentProtocol, task_ledger: _MagenticTaskLedger | None = None, *,
task_ledger_facts_prompt: str | None = None, task_ledger_plan_prompt: str | None =
None, task_ledger_full_prompt: str | None = None, task_ledger_facts_update_prompt:
str | None = None, task_ledger_plan_update_prompt: str | None = None,
progress_ledger_prompt: str | None = None, final_answer_prompt: str | None = None,
max_stall_count: int = 3, max_reset_count: int | None = None, max_round_count: int
| None = None, progress_ledger_retry_count: int | None = None) -> None
```

Parameters

 Expand table

Name	Description
agent Required*	AgentProtocol An agent instance to use for LLM calls. The agent's configured options (temperature, seed, instructions, etc.) will be applied.
task_ledger	<xref:agent_framework._workflows._magentic._MagenticTaskLedger> None Optional task ledger for managing task state. Default value: None
task_ledger_facts_prompt Required*	str None
task_ledger_plan_prompt Required*	str None
task_ledger_full_prompt Required*	str None
task_ledger_facts_update_prompt Required*	str None
task_ledger_plan_update_prompt Required*	str None
progress_ledger_prompt Required*	str None

Name	Description
final_answer_prompt Required*	str None
max_stall_count Required*	int
max_reset_count Required*	int None
max_round_count Required*	int None
progress_ledger_retry_count Required*	int None

Keyword-Only Parameters

 [Expand table](#)

Name	Description
task_ledger_facts_prompt	Optional prompt for the task ledger facts. Default value: None
task_ledger_plan_prompt	Optional prompt for the task ledger plan. Default value: None
task_ledger_full_prompt	Optional prompt for the full task ledger. Default value: None
task_ledger_facts_update_prompt	Optional prompt for updating task ledger facts. Default value: None
task_ledger_plan_update_prompt	Optional prompt for updating task ledger plan. Default value: None
progress_ledger_prompt	Optional prompt for the progress ledger. Default value: None
final_answer_prompt	Optional prompt for the final answer. Default value: None
max_stall_count	Maximum number of stalls allowed. Default value: 3
max_reset_count	Maximum number of resets allowed. Default value: None
max_round_count	Maximum number of rounds allowed. Default value: None

Name	Description
progress_ledger_retry_count	Maximum number of retries for the progress ledger. Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

create_progress_ledger

Use the model to produce a JSON progress ledger based on the conversation so far.

Adds lightweight retries with backoff for transient parse issues and avoids selecting a non-existent "unknown" agent. If there are no participants, a clear error is raised.

Python
<code>async create_progress_ledger(magentic_context: MagenticContext) -> _MagenticProgressLedger</code>

Parameters

[Expand table](#)

Name	Description
magentic_context Required*	MagenticContext

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows._magentic._MagenticProgressLedger>	

on_checkpoint_restore

Python
on_checkpoint_restore(state: dict[str, Any]) -> None

Parameters

[Expand table](#)

Name	Description
state Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
None	

on_checkpoint_save

Python
on_checkpoint_save() -> dict[str, Any]

Returns

[Expand table](#)

Type	Description
dict[str, Any]	

plan

Create facts and plan using the model, then render a combined task ledger as a single assistant message.

Python

```
async plan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

 Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

 Expand table

Type	Description
ChatMessage	

prepare_final_answer

Ask the model to produce the final answer addressed to the user.

Python

```
async prepare_final_answer(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

 Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

 Expand table

Type	Description
ChatMessage	

replan

Update facts and plan when stalling or looping has been detected.

Python

```
async replan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

 Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

 Expand table

Type	Description
ChatMessage	

Attributes

task_ledger

Python

```
task_ledger: _MagenticTaskLedger | None
```


SubWorkflowRequestMessage Class

Message sent from a sub-workflow to an executor in the parent workflow to request information.

This message wraps a RequestInfoEvent emitted by the executor in the sub-workflow.

Constructor

Python
<code>SubWorkflowRequestMessage()</code>

Parameters

 Expand table

Name	Description
<code>source_event</code> Required*	
<code>executor_id</code> Required*	

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>create_response</code>	Validate and wrap response data into a SubWorkflowResponseMessage. Validation ensures the response data type matches the expected type from the original request.

`__init__`

Python
<code>__init__(source_event: RequestInfoEvent, executor_id: str) -> None</code>

Parameters

 Expand table

Name	Description
source_event Required*	RequestInfoEvent
executor_id Required*	str

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

create_response

Validate and wrap response data into a SubWorkflowResponseMessage.

Validation ensures the response data type matches the expected type from the original request.

Python
<code>create_response(data: Any) -> SubWorkflowResponseMessage</code>

Parameters

 Expand table

Name	Description
data	Any

Name	Description
Required*	

Returns

[Expand table](#)

Type	Description
SubWorkflowResponseMessage	

Attributes

source_event

The original RequestInfoEvent emitted by the sub-workflow executor.

Python
<code>source_event: RequestInfoEvent</code>

executor_id

The ID of the WorkflowExecutor in the parent workflow that is responsible for this sub-workflow. This can be used to ensure that the response is sent back to the correct sub-workflow instance.

Python
<code>executor_id: str</code>

SubWorkflowResponseMessage Class

Message sent from a parent workflow to a sub-workflow via WorkflowExecutor to provide requested information.

This message wraps the response data along with the original RequestInfoEvent emitted by the sub-workflow executor.

Constructor

Python
<code>SubWorkflowResponseMessage()</code>

Parameters

 Expand table

Name	Description
<code>data</code> Required*	
<code>source_event</code> Required*	

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__(data: Any, source_event: RequestInfoEvent) -> None</code>

Parameters

[Expand table](#)

Name	Description
data Required*	Any
source_event Required*	RequestInfoEvent

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

data

The response data to the original request.

Python
<code>data: Any</code>

source_event

The original RequestInfoEvent emitted by the sub-workflow executor.

Python
<code>source_event: RequestInfoEvent</code>

SuperStepCompletedEvent Class

Event triggered when a superstep ends.

Initialize the superstep event.

Constructor

Python
<code>SuperStepCompletedEvent()</code>

Parameters

[Expand table](#)

Name	Description
iteration Required*	The number of the superstep (1-based index).
data	Optional data associated with the superstep event. Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the superstep event.
<code>__new__</code>	

`__init__`

Initialize the superstep event.

Python
<code>__init__(iteration: int, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
iteration Required [*]	int The number of the superstep (1-based index).
data	Any None Optional data associated with the superstep event. Default value: None

`__new__`

Python

```
__new__(**kwargs)
```

SuperStepStartedEvent Class

Event triggered when a superstep starts.

Initialize the superstep event.

Constructor

Python
<code>SuperStepStartedEvent()</code>

Parameters

[Expand table](#)

Name	Description
<code>iteration</code> Required*	The number of the superstep (1-based index).
<code>data</code>	Optional data associated with the superstep event. Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the superstep event.
<code>__new__</code>	

`__init__`

Initialize the superstep event.

Python
<code>__init__(iteration: int, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
iteration Required*	int The number of the superstep (1-based index).
data	Any None Optional data associated with the superstep event. Default value: None

`__new__`

Python
<code>__new__(**kwargs)</code>

SwitchCaseEdgeGroup Class

Fan-out variant that mimics a traditional switch/case control flow.

Each case inspects the message payload and decides whether it should handle the message. Exactly one case-or the default branch-returns a target at runtime, preserving single-dispatch semantics.

Configure a switch/case routing structure for a single source executor.

Constructor

Python
<code>SwitchCaseEdgeGroup()</code>

Parameters

 Expand table

Name	Description
source_id Required*	Identifier of the executor producing the message to be routed.
cases Required*	list [<xref:agent_framework._workflows._edge.SwitchCaseEdgeGroupCase agent_framework._workflows._edge.SwitchCaseEdgeGroupDefault>] Ordered sequence of case descriptors concluding with a <i>SwitchCaseEdgeGroupDefault</i> . Ordering matters because the runtime evaluates each branch sequentially until one matches.
id Required*	Optional explicit identifier for the edge group.
Examples Required*	Python <pre>cases = [SwitchCaseEdgeGroupCase(lambda payload: payload["kind"] == "csv", target_id="process_csv"), SwitchCaseEdgeGroupDefault(target_id="process_default"),] group = SwitchCaseEdgeGroup("router", cases)</pre>

Name	Description
	<pre>encoded = group.to_dict() assert encoded["cases"][0]["type"] == "Case"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Configure a switch/case routing structure for a single source executor.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Hydrate the correct <i>EdgeGroup</i> subclass from serialised state. The method inspects the <i>type</i> field, allocates the corresponding class without executing subclass <i>init</i>, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
<code>from_json</code>	
<code>register</code>	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their <i>name</i>, which must therefore remain stable across versions when persisted workflows are in circulation.
<code>to_dict</code>	Serialise the switch-case group, capturing all case descriptors. Each case is converted using <i>encode_value</i> to respect dataclass semantics as well as any nested serialisable structures.
<code>to_json</code>	

`__init__`

Configure a switch/case routing structure for a single source executor.

Python

```
__init__(source_id: str, cases: Sequence[SwitchCaseEdgeGroupCase |
SwitchCaseEdgeGroupDefault], *, id: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
source_id Required*	str Identifier of the executor producing the message to be routed.
cases Required*	Sequence[SwitchCaseEdgeGroupCase SwitchCaseEdgeGroupDefault] Ordered sequence of case descriptors concluding with a <i>SwitchCaseEdgeGroupDefault</i> . Ordering matters because the runtime evaluates each branch sequentially until one matches.
id Required*	str None Optional explicit identifier for the edge group.
Examples Required*	Python <pre>cases = [SwitchCaseEdgeGroupCase(lambda payload: payload["kind"] == "csv", target_id="process_csv"), SwitchCaseEdgeGroupDefault(target_id="process_default"),] group = SwitchCaseEdgeGroup("router", cases) encoded = group.to_dict() assert encoded["cases"][0]["type"] == "Case"</pre>

Keyword-Only Parameters

 Expand table

Name	Description
id	Default value: None

Returns

 Expand table

Type	Description
None	

__new__

Python
__new__(**kwargs)

clone

Python
clone(*, deep: bool = True) -> Self

Parameters

[🔗](#) Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

[🔗](#) Expand table

Name	Description
deep	Default value: True

Returns

[🔗](#) Expand table

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass *init*, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

 Expand table

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code><xref:agent_framework_workflows_edge.EdgeGroup></code>	

Examples

Python

```
payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id": "b"}]}
group = EdgeGroup.from_dict(payload)
assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

[Expand table](#)

Name	Description
subclass Required*	<code>type[<xref:agent_framework._workflows._edge.EdgeGroup>]</code>

Returns

[Expand table](#)

Type	Description
<code>type[<xref:agent_framework._workflows._edge.EdgeGroup>]</code>	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass

assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the switch-case group, capturing all case descriptors.

Each case is converted using *encode_value* to respect dataclass semantics as well as any nested serialisable structures.

Python

```
to_dict() -> dict[str, Any]
```

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

Examples

Python

```
group = SwitchCaseEdgeGroup(
    "router",
    [
        SwitchCaseEdgeGroupCase(lambda _: True, target_id="handler"),
        SwitchCaseEdgeGroupDefault(target_id="fallback"),
    ],
)
```



```
)
snapshot = group.to_dict()
assert len(snapshot["cases"]) == 2
```

to_json

Python

```
to_json() -> str
```

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

selection_func

Expose the runtime callable used to select active fan-out targets.

When no selection function was supplied the property returns *None*, signalling that all targets must receive the payload.

Examples

Python

```
group = FanOutEdgeGroup("source", ["x", "y"], selection_func=None)
assert group.selection_func is None
```

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

target_ids

Return a shallow copy of the configured downstream executor ids.

The list is defensively copied to prevent callers from mutating the internal state while still providing deterministic ordering.

Examples

Python

```
group = FanOutEdgeGroup("node", ["alpha", "beta"])
assert group.target_ids == ["alpha", "beta"]
```

cases

Python

```
cases: list[SwitchCaseEdgeGroupCase | SwitchCaseEdgeGroupDefault]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

selection_func_name

Python

```
selection_func_name: str | None
```

type

Python

```
type: str
```

SwitchCaseEdgeGroupCase Class

Persistable description of a single conditional branch in a switch-case.

Unlike the runtime *Case* object this serialisable variant stores only the target identifier and a descriptive name for the predicate. When the underlying callable is unavailable during deserialisation we substitute a proxy placeholder that fails loudly, ensuring the missing dependency is immediately visible.

Record the routing metadata for a conditional case branch.

Constructor

Python
<code>SwitchCaseEdgeGroupCase()</code>

Parameters

 Expand table

Name	Description
condition Required*	Optional live predicate. When omitted we fall back to a placeholder that raises at runtime to highlight missing registrations.
target_id Required*	<code>str</code> Identifier of the executor that should handle messages when the predicate succeeds.
condition_name Required*	<code><xref:agent_framework.str None></code> Human-friendly label for the predicate used for diagnostics and on-disk persistence.
Examples Required*	Python <pre>case = SwitchCaseEdgeGroupCase(lambda payload: payload["type"] == "csv", target_id="csv_handler") assert case.condition_name == "<lambda>"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>condition_name</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Record the routing metadata for a conditional case branch.
<code>__new__</code>	
<code>clone</code>	
<code>from_dict</code>	Instantiate a case from its serialised dictionary payload.
<code>from_json</code>	
<code>to_dict</code>	Serialise the case metadata without the executable predicate.
<code>to_json</code>	

`__init__`

Record the routing metadata for a conditional case branch.

Python

```
__init__(condition: Callable[[Any], bool] | None, target_id: str, *,  
condition_name: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
condition Required*	<code>Callable[[Any], bool] None</code> Optional live predicate. When omitted we fall back to a placeholder that raises at runtime to highlight missing registrations.
target_id Required*	<code>str</code> Identifier of the executor that should handle messages when the predicate succeeds.
condition_name	<code>str None</code>

Name	Description
Required*	Human-friendly label for the predicate used for diagnostics and on-disk persistence.
Examples Required*	Python <pre>case = SwitchCaseEdgeGroupCase(lambda payload: payload["type"] == "csv", target_id="csv_handler") assert case.condition_name == "<lambda>"</pre>

Keyword-Only Parameters

[Expand table](#)

Name	Description
condition_name	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<pre>__new__(**kwargs)</pre>

clone

Python
<pre>clone(*, deep: bool = True) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
deep Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

from_dict

Instantiate a case from its serialised dictionary payload.

Python
<code>from_dict(data: dict[str, Any]) -> SwitchCaseEdgeGroupCase</code>

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>SwitchCaseEdgeGroupCase</code>	

Examples

Python

```
payload = {"target_id": "handler", "condition_name": "is_ready"}
case = SwitchCaseEdgeGroupCase.from_dict(payload)
assert case.target_id == "handler"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

 Expand table

Name	Description
<code>raw</code> Required*	<code>str</code>

Returns

 Expand table

Type	Description
<code><xref:agent_framework_workflows_model_utils.TModel></code>	

to_dict

Serialise the case metadata without the executable predicate.

Python

```
to_dict() -> dict[str, Any]
```


Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

Examples

Python

```
case = SwitchCaseEdgeGroupCase(lambda _: True, target_id="handler")
assert case.to_dict()["target_id"] == "handler"
```

to_json

Python

```
to_json() -> str
```

Returns

 Expand table

Type	Description
<code>str</code>	

Attributes

condition

Return the predicate associated with this case.

The placeholder installed during deserialisation raises a *RuntimeError* when invoked so that workflow authors are forced to provide the missing callable explicitly.

Examples

Python

```
case = SwitchCaseEdgeGroupCase(None, target_id="missing",  
condition_name="needs_registration")  
guard = case.condition  
try:  
    guard({})  
except RuntimeError:  
    pass
```

condition_name

Python

condition_name: str | **None**

target_id

Python

target_id: str

type

Python

type: str

SwitchCaseEdgeGroupDefault Class

Persistable descriptor for the fallback branch of a switch-case group.

The default branch is guaranteed to exist and is invoked when every other case predicate fails to match the payload.

Point the default branch toward the given executor identifier.

Constructor

Python
<code>SwitchCaseEdgeGroupDefault()</code>

Parameters

 Expand table

Name	Description
<code>target_id</code> Required*	

Examples

Python
<pre>fallback = SwitchCaseEdgeGroupDefault(target_id="dead_letter") assert fallback.target_id == "dead_letter"</pre>

Methods

 Expand table

<code>__init__</code>	Point the default branch toward the given executor identifier.
<code>__new__</code>	
<code>clone</code>	

<code>from_dict</code>	Recreate the default branch from its persisted form.
<code>from_json</code>	
<code>to_dict</code>	Serialise the default branch metadata for persistence or logging.
<code>to_json</code>	

`__init__`

Point the default branch toward the given executor identifier.

Python
<pre>__init__(target_id: str) -> None</pre>

Parameters

[Expand table](#)

Name	Description
<code>target_id</code> Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

Examples

Python
<pre>fallback = SwitchCaseEdgeGroupDefault(target_id="dead_letter") assert fallback.target_id == "dead_letter"</pre>

`__new__`

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

[Expand table](#)

Name	Description
deep Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

from_dict

Recreate the default branch from its persisted form.

Python

```
from_dict(data: dict[str, Any]) -> SwitchCaseEdgeGroupDefault
```

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>SwitchCaseEdgeGroupDefault</code>	

Examples

Python

```
payload = {"target_id": "dead_letter", "type": "Default"}
fallback = SwitchCaseEdgeGroupDefault.from_dict(payload)
assert fallback.target_id == "dead_letter"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

to_dict

Serialise the default branch metadata for persistence or logging.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Examples

Python
<pre> fallback = SwitchCaseEdgeGroupDefault("dead_letter") assert fallback.to_dict()["type"] == "Default" </pre>

to_json

Python
<code>to_json() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

target_id

Python
target_id: str

type

Python
type: str

TextContent Class

Represents text content in a chat.

Initializes a TextContent instance.

Constructor

Python
<code>TextContent()</code>

Parameters

 Expand table

Name	Description
<code>text</code> Required*	The text content represented by this instance.

Keyword-Only Parameters

 Expand table

Name	Description
<code>additional_properties</code>	Optional additional properties associated with the content. Default value: None
<code>raw_representation</code>	Optional raw representation of the content. Default value: None
<code>annotations</code>	Optional annotations associated with the content. Default value: None
<code>**kwargs</code>	Any additional keyword arguments.

Examples

Python
<pre>from agent_framework import TextContent</pre>

```

# Create basic text content
text = TextContent(text="Hello, world!")
print(text.text) # "Hello, world!"

# Concatenate text content
text1 = TextContent(text="Hello, ")
text2 = TextContent(text="world!")
combined = text1 + text2
print(combined.text) # "Hello, world!"

```

Methods

[Expand table](#)

<code>__init__</code>	Initializes a <code>TextContent</code> instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a `TextContent` instance.

Python

```
__init__(text: str, *, additional_properties: dict[str, Any] | None = None,
raw_representation: Any | None = None, annotations: Sequence[CitationAnnotation |
MutableMapping[str, Any]] | None = None, **kwargs: Any)
```

Parameters

 Expand table

Name	Description
text Required*	<code>str</code> The text content represented by this instance.
additional_properties Required*	<code>dict[str, Any] None</code>
raw_representation Required*	<code>Any None</code>
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
**kwargs	Any additional keyword arguments.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: Simple injection: <code>{"<type>": {"<parameter>": value}}</code>

Name	Description
	- Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> - Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}
```

```

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value	<code>str</code>

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

text

The text content represented by this instance.

type

The type of content, which is always "text" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

TextReasoningContent Class

Represents text reasoning content in a chat.

Remarks: This class and *TextContent* are superficially similar, but distinct.

Initializes a TextReasoningContent instance.

Constructor

Python
<code>TextReasoningContent()</code>

Parameters

 Expand table

Name	Description
text Required*	The text content represented by this instance.

Keyword-Only Parameters

 Expand table

Name	Description
protected_data	This property is used to store data from a provider that should be roundtripped back to the provider but that is not intended for human consumption. It is often encrypted or otherwise redacted information that is only intended to be sent back to the provider and not displayed to the user. It's possible for a <code>TextReasoningContent</code> to contain only <i>protected_data</i> and have an empty <i>text</i> property. This data also may be associated with the corresponding <i>text</i>, acting as a validation signature for it. Note that whereas <i>text</i> can be provider agnostic, <i>protected_data</i> is provider-specific, and is likely to only be understood by the provider that created it. The data is often represented as a more complex object, so it should be serialized to a string before storing so that the whole object is easily serializable without loss. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```
from agent_framework import TextReasoningContent

# Create reasoning content
reasoning = TextReasoningContent(text="Let me think step by step...")
print(reasoning.text) # "Let me think step by step..."

# Concatenate reasoning content
reasoning1 = TextReasoningContent(text="First, ")
reasoning2 = TextReasoningContent(text="second, ")
combined = reasoning1 + reasoning2
print(combined.text) # "First, second, "
```

Methods

 Expand table

<code>__init__</code>	Initializes a TextReasoningContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available

	through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code> . All the same serialization rules apply as in <code>to_dict()</code> , including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a `TextReasoningContent` instance.

Python

```
__init__(text: str | None, *, protected_data: str | None = None,
additional_properties: dict[str, Any] | None = None, raw_representation: Any |
None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]]
| None = None, **kwargs: Any)
```

Parameters

[Expand table](#)

Name	Description
text Required*	<code>str None</code> The text content represented by this instance.
protected_data Required*	<code>str None</code>
additional_properties Required*	<code>dict[str, Any] None</code>
raw_representation Required*	<code>Any None</code>
annotations Required*	<code>Sequence[CitationAnnotation MutableMapping[str, Any]] None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

Name	Description
protected_data	This property is used to store data from a provider that should be roundtripped back to the provider but that is not intended for human consumption. It is often encrypted or otherwise redacted information that is only intended to be sent back to the provider and not displayed to the user. It's possible for a <code>TextReasoningContent</code> to contain only <i>protected_data</i> and have an empty <i>text</i> property. This data also may be associated with the corresponding <i>text</i>, acting as a validation signature for it. Note that whereas <i>text</i> can be provider agnostic, <i>protected_data</i> is provider-specific, and is likely to only be understood by the provider that created it. The data is often represented as a more complex object, so it should be serialized to a string before storing so that the whole object is easily serializable without loss. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
**kwargs	Any additional keyword arguments.

`__new__`

Python

```
__new__(**kwargs)
```

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: <code>None</code>

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCIass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None

Name	Description
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str]</code> <code>None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

text

The text content represented by this instance.

type

The type of content, which is always "text_reasoning" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

TextSpanRegion Class

Represents a region of text that has been annotated.

Initialize TextSpanRegion.

Constructor

Python
<code>TextSpanRegion()</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>start_index</code>	The start index of the text span. Default value: None
<code>end_index</code>	The end index of the text span. Default value: None
<code>**kwargs</code>	Additional keyword arguments.

Examples

Python
<pre>from agent_framework import TextSpanRegion # Create a text span region region = TextSpanRegion(start_index=0, end_index=10) print(region.type) # "text_span"</pre>

Methods

[Expand table](#)

<code>__init__</code>	Initialize TextSpanRegion.
-----------------------	----------------------------

<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize TextSpanRegion.

Python

```
__init__(*, start_index: int | None = None, end_index: int | None = None,
**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
start_index Required*	<code>int</code> <code>None</code>

Name	Description
<code>end_index</code> Required*	<code>int</code> <code>None</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>start_index</code>	The start index of the text span. Default value: <code>None</code>
<code>end_index</code>	The end index of the text span. Default value: <code>None</code>
<code>**kwargs</code>	Additional keyword arguments.

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

from_dict()

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
<code>from_json()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCIass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```


Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python
<code>DEFAULT_EXCLUDE: ClassVar[set[str]] = {}</code>

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ToolMode Class

Defines if and how tools are used in a chat request.

Initialize ToolMode.

Constructor

Python
<code>ToolMode()</code>

Parameters

 Expand table

Name	Description
<code>mode</code>	The tool mode - "auto", "required", or "none". Default value: none

Keyword-Only Parameters

 Expand table

Name	Description
<code>required_function_name</code>	Optional function name for required mode. Default value: None

Examples

Python
<pre>from agent_framework import ToolMode # Use predefined tool modes auto_mode = ToolMode.AUTO # Model decides when to use tools required_mode = ToolMode.REQUIRED_ANY # Model must use a tool none_mode = ToolMode.NONE # No tools allowed # Require a specific function specific_mode = ToolMode.REQUIRED(function_name="get_weather")</pre>

```
print(specific_mode.required_function_name) # "get_weather"

# Compare modes
print(auto_mode == "auto") # True
```

Methods

 Expand table

REQUIRED	Returns a ToolMode that requires the specified function to be called.
__init__	Initialize ToolMode.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
serialize_model	Serializes the ToolMode to just the mode string.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).
to_json	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

REQUIRED

Returns a ToolMode that requires the specified function to be called.



Python

```
REQUIRED(function_name: str | None = None) -> ToolMode
```

Parameters

[Expand table](#)

Name	Description
function_name	str None Default value: None

Returns

[Expand table](#)

Type	Description
ToolMode	

`__init__`

Initialize ToolMode.

Python

```
__init__(mode: Literal['auto', 'required', 'none'] = 'none', *,  
required_function_name: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
mode	Literal ['auto', 'required', 'none'] The tool mode - "auto", "required", or "none". Default value: "none"
required_function_name Required*	str None

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>required_function_name</code>	Optional function name for required mode. Default value: None

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>value</code> Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) -> str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```



```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

serialize_model

Serializes the ToolMode to just the mode string.

Python

```
serialize_model() -> str
```

Returns

[Expand table](#)

Type	Description
str	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

AUTO

Python

```
AUTO: ToolMode = ToolMode(mode='auto')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

NONE

Python

```
NONE: ToolMode = ToolMode(mode='none')
```

REQUIRED_ANY

Python

```
REQUIRED_ANY: ToolMode = ToolMode(mode='required')
```

ToolProtocol Class

Represents a generic tool.

This protocol defines the interface that all tools must implement to be compatible with the agent framework. It is implemented by various tool classes such as HostedMCPTool, HostedWebSearchTool, and AIFunction's. A AIFunction is usually created by the *ai_function* decorator.

Since each connector needs to parse tools differently, users can pass a dict to specify a service-specific tool when no abstraction is available.

Constructor

Python
<code>ToolProtocol()</code>

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code><code>__init__</code>(*args, **kwargs)</code>

`__new__`

Python
<code><code>__new__</code>(**kwargs)</code>

Attributes

additional_properties

Additional properties associated with the tool.

Python

additional_properties: dict[str, Any] | **None**

description

A description of the tool, suitable for use in describing the purpose to a model.

Python

description: str

name

The name of the tool.

Python

name: str

TypeCompatibilityError Class

Exception raised when type incompatibility is detected between connected executors.

Constructor

Python
TypeCompatibilityError()

Parameters

 Expand table

Name	Description
source_executor_id Required*	
target_executor_id Required*	
source_types Required*	
target_types Required*	

Methods

 Expand table

__init__	
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.traceback to tb and return self.

[__init__](#)

Python

```
__init__(source_executor_id: str, target_executor_id: str, source_types:
list[type[Any]], target_types: list[type[Any]])
```

Parameters

 Expand table

Name	Description
source_executor_id Required*	<code>str</code>
target_executor_id Required*	<code>str</code>
source_types Required*	<code>list[type[Any]]</code>
target_types Required*	<code>list[type[Any]]</code>

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.traceback to tb and return self.

Python

```
with_traceback()
```

Attributes

args

UriContent Class

Represents a URI content.

 Important

This is used for content that is identified by a URI, such as an image or a file.

For (binary) data URIs, use `DataContent` instead.

Initializes a `UriContent` instance.

Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.

Constructor

Python
<code>UriContent()</code>

Parameters

 Expand table

Name	Description
<code>uri</code> Required*	The URI of the content.
<code>media_type</code> Required*	The media type of the content.

Keyword-Only Parameters

 Expand table

Name	Description
<code>annotations</code>	Optional annotations associated with the content. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Examples

Python

```
from agent_framework import UriContent

# Create URI content for an image
image_uri = UriContent(
    uri="https://example.com/image.png",
    media_type="image/png",
)

# Create URI content for a document
doc_uri = UriContent(
    uri="https://example.com/document.pdf",
    media_type="application/pdf",
)

# Check if it's an image
if image_uri.has_top_level_media_type("image"):
    print("This is an image URI")
```

Methods

 Expand table

<code>__init__</code>	Initializes a UriContent instance. Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use <i>DataContent</i> instead.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports

	three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>has_top_level_media_type</code>	Returns a boolean indicating if the media type has the specified top-level media type.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a `UriContent` instance.

Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.

Python

```
__init__(uri: str, media_type: str, *, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
<code>uri</code> Required*	<code>str</code> The URI of the content.
<code>media_type</code> Required*	<code>str</code> The media type of the content.

Name	Description
annotations Required*	Sequence[CitationAnnotation MutableMapping[str, Any]] None
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
**kwargs	Any additional keyword arguments.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any] None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: - Simple injection: <code>{"<type>": {"<parameter>": value}}</code> - Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code>

Name	Description
	Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

has_top_level_media_type

Returns a boolean indicating if the media type has the specified top-level media type.

Python

```
has_top_level_media_type(top_level_media_type: Literal['application', 'audio', 'image', 'text']) -> bool
```

Parameters

 Expand table

Name	Description
top_level_media_type Required*	<code>Literal['application', 'audio', 'image', 'text']</code> The top-level media type to check for, allowed values: "image", "text", "application", "audio".

Returns

 Expand table

Type	Description
<code>bool</code>	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<pre>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[↗](#) Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[↗](#) Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

uri

The URI of the content, e.g., '<https://example.com/image.png>'.

media_type

The media type of the content, e.g., 'image/png', 'application/json', etc.

type

The type of content, which is always "uri" for this class.

annotations

Optional annotations associated with the content.

Python
<code>annotations: list[CitationAnnotation] None</code>

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```


UsageContent Class

Represents usage information associated with a chat request and response.

Initializes a UsageContent instance.

Constructor

Python
<pre>UsageContent()</pre>

Parameters

 Expand table

Name	Description
details Required*	

Keyword-Only Parameters

 Expand table

Name	Description
annotations	Default value: None
additional_properties	Default value: None
raw_representation	Default value: None

Examples

Python
<pre>from agent_framework import UsageContent, UsageDetails # Create usage content usage = UsageContent(details=UsageDetails(input_token_count=100,</pre>

```

        output_token_count=50,
        total_token_count=150,
    ),
)
print(usage.details.total_token_count)  # 150

```

Methods

 Expand table

<code>__init__</code>	Initializes a UsageContent instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes a UsageContent instance.

Python

```

__init__(details: UsageDetails | MutableMapping[str, Any], *, annotations:
Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None,
additional_properties: dict[str, Any] | None = None, raw_representation: Any |
None = None, **kwargs: Any) -> None

```

Parameters

 Expand table

Name	Description
details Required*	UsageDetails MutableMapping[str, Any]
annotations Required*	Sequence[CitationAnnotation MutableMapping[str, Any]] None
additional_properties Required*	dict[str, Any] None
raw_representation Required*	Any None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
annotations	Default value: None
additional_properties	Default value: None
raw_representation	Default value: None

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: Simple injection: <code>{"<type>": {"<parameter>": value}}</code>

Name	Description
	- Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> - Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}
```

```

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value	<code>str</code>

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

details

The usage information, including input and output token counts, and any additional counts.

type

The type of content, which is always "usage" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',  
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

UsageDetails Class

Provides usage details about a request/response.

Initializes the UsageDetails instance.

Constructor

Python
<pre>UsageDetails()</pre>

Parameters

 Expand table

Name	Description
<code>input_token_count</code>	The number of tokens in the input. Default value: None
<code>output_token_count</code>	The number of tokens in the output. Default value: None
<code>total_token_count</code>	The total number of tokens used to produce the response. Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
<code>**kwargs</code>	Additional token counts, can be set by passing keyword arguments. They can be retrieved through the <i>additional_counts</i> property.

Examples

Python
<pre>from agent_framework import UsageDetails # Create usage details</pre>

```

usage = UsageDetails(
    input_token_count=100,
    output_token_count=50,
    total_token_count=150,
)
print(usage.total_token_count)  # 150

# With additional counts
usage = UsageDetails(
    input_token_count=100,
    output_token_count=50,
    total_token_count=150,
    reasoning_tokens=25,
)
print(usage.additional_counts["reasoning_tokens"])  # 25

# Combine usage details
usage1 = UsageDetails(input_token_count=100, output_token_count=50)
usage2 = UsageDetails(input_token_count=200, output_token_count=100)
combined = usage1 + usage2
print(combined.input_token_count)  # 300

```

Methods

[Expand table](#)

<code>__init__</code>	Initializes the UsageDetails instance.
<code>__new__</code>	
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>to_dict</code>	Convert the UsageDetails instance to a dictionary.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initializes the UsageDetails instance.

Python

```
__init__(input_token_count: int | None = None, output_token_count: int | None = None, total_token_count: int | None = None, **kwargs: int) -> None
```

Parameters

 Expand table

Name	Description
<code>input_token_count</code>	<code>int None</code> The number of tokens in the input. Default value: None
<code>output_token_count</code>	<code>int None</code> The number of tokens in the output. Default value: None
<code>total_token_count</code>	<code>int None</code> The total number of tokens used to produce the response. Default value: None
<code>kwargs</code> Required*	<code>int</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>**kwargs</code>	Additional token counts, can be set by passing keyword arguments. They can be retrieved through the <i>additional_counts</i> property.

Returns

 Expand table

Type	Description
<code>None</code>	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```



```

    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}

```

```

context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the UsageDetails instance to a dictionary.

--

Python

```
to_dict(*, exclude_none: bool = True, exclude: set[str] | None = None) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude_none Required*	bool
exclude Required*	set[str] None

Keyword-Only Parameters

 Expand table

Name	Description
exclude_none	Whether to exclude None values from the output. Default value: True
exclude	Set of field names to exclude from the output. Default value: None

Returns

 Expand table

Type	Description
dict[str, Any]	Dictionary representation of the UsageDetails instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

additional_counts

Represents well-known additional counts for usage. This is not an exhaustive list.

Remarks: To make it possible to avoid collisions between similarly-named, but unrelated, additional counts between different AI services, any keys not explicitly defined here should be prefixed with the name of the AI service, e.g., "openai." or "azure.". The separator "." was chosen because it cannot be a legal character in a JSON key.

Over time additional counts may be added to the base class.

input_token_count

The number of tokens in the input.

output_token_count

The number of tokens in the output.

total_token_count

The total number of tokens used to produce the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_extra_counts'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ValidationTypeEnum Enum

Enumeration of workflow validation types.

Fields

 Expand table

EDGE_DUPLICATION
EXECUTOR_DUPLICATION
GRAPH_CONNECTIVITY
HANDLER_OUTPUT_ANNOTATION
INTERCEPTOR_CONFLICT
TYPE_COMPATIBILITY

Workflow Class

A graph-based execution engine that orchestrates connected executors.

Overview

A workflow executes a directed graph of executors connected via edge groups using a Pregel-like model, running in supersteps until the graph becomes idle. Workflows are created using the `WorkflowBuilder` class - do not instantiate this class directly.

Execution Model

Executors run in synchronized supersteps where each executor:

- Is invoked when it receives messages from connected edge groups
- Can send messages to downstream executors via `ctx.send_message()`
- Can yield workflow-level outputs via `ctx.yield_output()`
- Can emit custom events via `ctx.add_event()`

Messages between executors are delivered at the end of each superstep and are not visible in the event stream. Only workflow-level events (outputs, custom events) and status events are observable to callers.

Input/Output Types

Workflow types are discovered at runtime by inspecting:

- Input types: From the start executor's input types
- Output types: Union of all executors' workflow output types Access these via the `input_types` and `output_types` properties.

Execution Methods

The workflow provides two primary execution APIs, each supporting multiple scenarios:

- `run()`: Execute to completion, returns `WorkflowRunResult` with all events
- `run_stream()`: Returns async generator yielding events as they occur

Both methods support:

- Initial workflow runs: Provide *message* parameter
- Checkpoint restoration: Provide *checkpoint_id* (and optionally *checkpoint_storage*)
- HIL continuation: Provide *responses* to continue after RequestInfoExecutor requests
- Runtime checkpointing: Provide *checkpoint_storage* to enable/override checkpointing for this run

State Management

Workflow instances contain states and states are preserved across calls to *run* and *run_stream*. To execute multiple independent runs, create separate Workflow instances via WorkflowBuilder.

External Input Requests

Executors within a workflow can request external input using *ctx.request_info()*:

1. Executor calls *ctx.request_info()* to request input
2. Executor implements *response_handler()* to process the response
3. Requests are emitted as RequestInfoEvent instances in the event stream
4. Workflow enters IDLE_WITH_PENDING_REQUESTS state
5. Caller handles requests and provides responses via the *send_responses* or *send_responses_streaming* methods
6. Responses are routed to the requesting executors and response handlers are invoked

Checkpointing

Checkpointing can be configured at build time or runtime:

Build-time (via WorkflowBuilder): `workflow = WorkflowBuilder().with_checkpointing(storage).build()`

Runtime (via run/run_stream parameters): `result = await workflow.run(message, checkpoint_storage=runtime_storage)`

When enabled, checkpoints are created at the end of each superstep, capturing:

- Executor states
- Messages in transit
- Shared state Workflows can be paused and resumed across process restarts using checkpoint storage.

Composition

Workflows can be nested using WorkflowExecutor, which wraps a child workflow as an executor. The nested workflow's input/output types become part of the WorkflowExecutor's types. When invoked, the WorkflowExecutor runs the nested workflow to completion and processes its outputs.

Initialize the workflow with a list of edges.

Constructor

Python
<code>Workflow()</code>

Parameters

 Expand table

Name	Description
edge_groups Required*	A list of EdgeGroup instances that define the workflow edges.
executors Required*	A dictionary mapping executor IDs to Executor instances.
start_executor Required*	The starting executor for the workflow.
runner_context Required*	The RunnerContext instance to be used during workflow execution.
max_iterations	The maximum number of iterations the workflow will run for convergence. Default value: 100
name	Optional human-readable name for the workflow. Default value: None
description	Optional description of what the workflow does. Default value: None
kwargs Required*	Additional keyword arguments. Unused in this implementation.

Methods

<code>__init__</code>	Initialize the workflow with a list of edges.
<code>__new__</code>	
<code>as_agent</code>	Create a WorkflowAgent that wraps this workflow. The returned agent converts standard agent inputs (strings, ChatMessage, or lists of these) into a list[ChatMessage] that is passed to the workflow's start executor. This conversion happens in WorkflowAgent._normalize_messages() which transforms: • str -> [ChatMessage(role=USER, text=str)] • ChatMessage -> [ChatMessage] • list[str ChatMessage] -> list[ChatMessage] (with string elements converted) The workflow's start executor must accept list[ChatMessage] as an input type, otherwise initialization will fail with a ValueError.
<code>clone</code>	
<code>from_dict</code>	
<code>from_json</code>	
<code>get_executors_list</code>	Get the list of executors in the workflow.
<code>get_start_executor</code>	Get the starting executor of the workflow.
<code>run</code>	Run the workflow to completion and return all events. Unified non-streaming interface supporting initial runs and checkpoint restoration.
<code>run_stream</code>	Run the workflow and stream events. Unified streaming interface supporting initial runs and checkpoint restoration.
<code>send_responses</code>	Send responses back to the workflow.
<code>send_responses_streaming</code>	Send responses back to the workflow and stream the events generated by the workflow.
<code>to_dict</code>	Serialize the workflow definition into a JSON-ready dictionary.
<code>to_json</code>	Serialize the workflow definition to JSON.

`__init__`

Initialize the workflow with a list of edges.

Python

```
__init__(edge_groups: list[EdgeGroup], executors: dict[str, Executor],
start_executor: Executor, runner_context: RunnerContext, max_iterations: int =
DEFAULT_MAX_ITERATIONS, name: str | None = None, description: str | None = None,
**kwargs: Any)
```

Parameters

 Expand table

Name	Description
edge_groups Required*	list [<xref:agent_framework_workflows_edge.EdgeGroup>] A list of EdgeGroup instances that define the workflow edges.
executors Required*	dict [str , Executor] A dictionary mapping executor IDs to Executor instances.
start_executor Required*	Executor The starting executor for the workflow.
runner_context Required*	RunnerContext The RunnerContext instance to be used during workflow execution.
max_iterations	int The maximum number of iterations the workflow will run for convergence. Default value: DEFAULT_MAX_ITERATIONS
name	str None Optional human-readable name for the workflow. Default value: None
description	str None Optional description of what the workflow does. Default value: None
kwargs Required*	Any Additional keyword arguments. Unused in this implementation.

__new__

Python

```
__new__(**kwargs)
```

as_agent

Create a `WorkflowAgent` that wraps this workflow.

The returned agent converts standard agent inputs (strings, `ChatMessage`, or lists of these) into a `list[ChatMessage]` that is passed to the workflow's start executor. This conversion happens in `WorkflowAgent._normalize_messages()` which transforms:

- `str` -> `[ChatMessage(role=USER, text=str)]`
- `ChatMessage` -> `[ChatMessage]`
- `list[str | ChatMessage]` -> `list[ChatMessage]` (with string elements converted)

The workflow's start executor must accept `list[ChatMessage]` as an input type, otherwise initialization will fail with a `ValueError`.

Python

```
as_agent(name: str | None = None) -> WorkflowAgent
```

Parameters

[Expand table](#)

Name	Description
<code>name</code>	<code>str None</code> Optional name for the agent. If <code>None</code> , a default name will be generated. Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
<code>WorkflowAgent</code>	A <code>WorkflowAgent</code> instance that wraps this workflow.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the workflow's start executor cannot handle <code>list[ChatMessage]</code> input.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

 Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
deep	Default value: True

Returns

 Expand table

Type	Description
Self	

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

 Expand table

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

from_json

Python
<code>from_json(raw: str) -> TModel</code>

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

get_executors_list

Get the list of executors in the workflow.

Python
<code>get_executors_list() -> list[Executor]</code>

Returns

[Expand table](#)

Type	Description
<code>list[Executor]</code>	

get_start_executor

Get the starting executor of the workflow.

Python
<pre>get_start_executor() -> Executor</pre>

Returns

[Expand table](#)

Type	Description
<code>Executor</code>	The starting executor instance.

run

Run the workflow to completion and return all events.

Unified non-streaming interface supporting initial runs and checkpoint restoration.

Python
<pre>async run(message: Any None = None, *, checkpoint_id: str None = None, checkpoint_storage: CheckpointStorage None = None, include_status_events: bool = False, **kwargs: Any) -> WorkflowRunResult</pre>

Parameters

[Expand table](#)

Name	Description
message	<code>Any None</code>

Name	Description
	Initial message for the start executor. Required for new workflow runs, should be None when resuming from checkpoint. Default value: None
checkpoint_id Required*	str None ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. When resuming, checkpoint_storage must be provided (either at build time or runtime) to load the checkpoint.
checkpoint_storage Required*	CheckpointStorage None Runtime checkpoint storage with two behaviors: • With checkpoint_id: Used to load and restore the specified checkpoint • Without checkpoint_id: Enables checkpointing for this run, overriding build-time configuration
include_status_events Required*	bool Whether to include WorkflowStatusEvent instances in the result list.
kwargs Required*	Any Additional keyword arguments to pass through to agent invocations. These are stored in SharedState and accessible in @ai_function tools via the >> <<kwargs parameter.

Keyword-Only Parameters

[Expand table](#)

Name	Description
checkpoint_id	Default value: None
checkpoint_storage	Default value: None
include_status_events	Default value: False

Returns

[Expand table](#)

Type	Description
WorkflowRunResu	A WorkflowRunResult instance containing events generated during workflow

Type	Description
It	execution.

Exceptions

 Expand table

Type	Description
ValueError	If both message and checkpoint_id are provided, or if neither is provided.
ValueError	If checkpoint_id is provided but no checkpoint storage is available (neither at build time nor runtime).
RuntimeError	If checkpoint restoration fails.

Examples

Initial run:

Python

```
result = await workflow.run("start message")
outputs = result.get_outputs()
```

With custom context for ai_functions:

Python

```
result = await workflow.run(
    "analyze data",
    custom_data={"endpoint": "https://api.example.com"},
    user_token={"user": "alice"},
)
```

Enable checkpointing at runtime:

Python

```
storage = FileCheckpointStorage("./checkpoints")
result = await workflow.run("start", checkpoint_storage=storage)
```

Resume from checkpoint (storage provided at build time):

Python

```
result = await workflow.run(checkpoint_id="cp_123")
```

Resume from checkpoint (storage provided at runtime):

Python

```
storage = FileCheckpointStorage("./checkpoints")
result = await workflow.run(checkpoint_id="cp_123", checkpoint_storage=storage)
```

run_stream

Run the workflow and stream events.

Unified streaming interface supporting initial runs and checkpoint restoration.

Python

```
async run_stream(message: Any | None = None, *, checkpoint_id: str | None = None,
checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) ->
AsyncIterable[WorkflowEvent]
```

Parameters

[Expand table](#)

Name	Description
message	Any None Initial message for the start executor. Required for new workflow runs, should be None when resuming from checkpoint. Default value: None
checkpoint_id Required*	str None ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. When resuming, checkpoint_storage must be provided (either at build time or runtime) to load the checkpoint.
checkpoint_storage Required*	CheckpointStorage None Runtime checkpoint storage with two behaviors: • With checkpoint_id: Used to load and restore the specified checkpoint • Without checkpoint_id: Enables checkpointing for this run, overriding

Name	Description
	build-time configuration
kwargs Required*	Any Additional keyword arguments to pass through to agent invocations. These are stored in SharedState and accessible in @ai_function tools via the >><<kwargs parameter.

Keyword-Only Parameters

[Expand table](#)

Name	Description
checkpoint_id	Default value: None
checkpoint_storage	Default value: None

Returns

[Expand table](#)

Type	Description
AsyncIterable [WorkflowEvent]	

Exceptions

[Expand table](#)

Type	Description
ValueError	If both message and checkpoint_id are provided, or if neither is provided.
ValueError	If checkpoint_id is provided but no checkpoint storage is available (neither at build time nor runtime).
RuntimeError	If checkpoint restoration fails.

Examples

Initial run:

```
Python
```

```
async for event in workflow.run_stream("start message"):
    process(event)
```

With custom context for ai_functions:

Python

```
async for event in workflow.run_stream(
    "analyze data",
    custom_data={"endpoint": "https://api.example.com"},
    user_token={"user": "alice"},
):
    process(event)
```

Enable checkpointing at runtime:

Python

```
storage = FileCheckpointStorage("./checkpoints")
async for event in workflow.run_stream("start", checkpoint_storage=storage):
    process(event)
```

Resume from checkpoint (storage provided at build time):

Python

```
async for event in workflow.run_stream(checkpoint_id="cp_123"):
    process(event)
```

Resume from checkpoint (storage provided at runtime):

Python

```
storage = FileCheckpointStorage("./checkpoints")
async for event in workflow.run_stream(checkpoint_id="cp_123",
checkpoint_storage=storage):
    process(event)
```

send_responses

Send responses back to the workflow.

Python

```
async send_responses(responses: dict[str, Any]) -> WorkflowRunResult
```

Parameters

[Expand table](#)

Name	Description
responses Required*	<code>dict[str, Any]</code> A dictionary where keys are request IDs and values are the corresponding response data.

Returns

[Expand table](#)

Type	Description
<code>WorkflowRunResult</code>	A WorkflowRunResult instance containing a list of events generated during the workflow execution.

send_responses_streaming

Send responses back to the workflow and stream the events generated by the workflow.

Python

```
async send_responses_streaming(responses: dict[str, Any]) ->  
AsyncIterable[WorkflowEvent]
```

Parameters

[Expand table](#)

Name	Description
responses Required*	<code>dict[str, Any]</code> The responses to be sent back to the workflow, where keys are request IDs and values are the corresponding response data.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[WorkflowEvent]</code>	

to_dict

Serialize the workflow definition into a JSON-ready dictionary.

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

to_json

Serialize the workflow definition to JSON.

Python
<code>to_json() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Attributes

`graph_signature_hash`

`input_types`

Get the input types of the workflow.

The input types are the list of input types of the start executor.

Returns

 Expand table

Type	Description
	A list of input types that the workflow can accept.

output_types

Get the output types of the workflow.

The output types are the list of all workflow output types from executors that have workflow output types.

Returns

 Expand table

Type	Description
	A list of output types that the workflow can produce.

WorkflowAgent Class

An *Agent* subclass that wraps a workflow and exposes it as an agent.

Initialize the WorkflowAgent.

Constructor

Python
<code>WorkflowAgent()</code>

Parameters

[Expand table](#)

Name	Description
<code>workflow</code> Required*	The workflow to wrap as an agent.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>id</code>	Unique identifier for the agent. If None, will be generated. Default value: None
<code>name</code>	Optional name for the agent. Default value: None
<code>description</code>	Optional description of the agent. Default value: None
<code>**kwargs</code>	Additional keyword arguments passed to BaseAgent.

Methods

[Expand table](#)

<code>__init__</code>	Initialize the WorkflowAgent.
<code>__new__</code>	
<code>as_tool</code>	Create an AIFunction tool that wraps this agent.
<code>deserialize_thread</code>	Deserialize a thread from its serialized state.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_new_thread</code>	Return a new AgentThread instance that is compatible with the agent.
<code>merge_updates</code>	Merge streaming updates into a single AgentRunResponse. Behavior: - Group updates by <code>response_id</code>; within each <code>response_id</code>, group by <code>message_id</code> and keep a dangling bucket for updates without <code>message_id</code>. - Convert each group (per message and dangling) into an intermediate AgentRunResponse via <code>AgentRunResponse.from_agent_run_response_updates</code>, then sort by <code>created_at</code> and merge. - Append messages from updates without any <code>response_id</code> at the end (global dangling), while aggregating metadata.
<code>run</code>	Get a response from the workflow agent (non-streaming). This method collects all streaming updates and merges them into a single response.
<code>run_stream</code>	Stream response updates from the workflow agent.
<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with <code>'_'</code>).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the WorkflowAgent.

Python

```
__init__(workflow: Workflow, *, id: str | None = None, name: str | None = None, description: str | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
workflow Required*	Workflow The workflow to wrap as an agent.
id Required*	str None
name Required*	str None
description Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	Unique identifier for the agent. If None, will be generated. Default value: None
name	Optional name for the agent. Default value: None

Name	Description
description	Optional description of the agent. Default value: None
**kwargs	Additional keyword arguments passed to BaseAgent.

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

as_tool

Create an AIFunction tool that wraps this agent.

Python
<pre>as_tool(*, name: str None = None, description: str None = None, arg_name: str = 'task', arg_description: str None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] Callable[[AgentRunResponseUpdate], Awaitable[None]] None = None) -> AIFunction[BaseModel, str]</pre>

Parameters

 Expand table

Name	Description
name Required*	str None
description Required*	str None
arg_name	str

Name	Description
Required*	
arg_description Required*	str None
stream_callback Required*	Callable [[AgentRunResponseUpdate], None] Callable [[AgentRunResponseUpdate], Awaitable [None]] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

[Expand table](#)

Type	Description
AIFunction [<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

[Expand table](#)

Type	Description
TypeError	If the agent does not implement AgentProtocol.

Type	Description
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
from agent_framework import ChatAgent

# Create an agent
agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

# Convert the agent to a tool
research_tool = agent.as_tool()

# Use the tool with another agent
coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

 Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

[↗](#) Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[↗](#) Expand table

Name	Description
value Required*	

Parameters

[↗](#) Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```



```

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python
<pre>get_new_thread(**kwargs: Any) -> AgentThread</pre>

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

merge_updates

Merge streaming updates into a single AgentRunResponse.

Behavior:

- Group updates by response_id; within each response_id, group by message_id and keep a dangling bucket for updates without message_id.
- Convert each group (per message and dangling) into an intermediate AgentRunResponse via AgentRunResponse.from_agent_run_response_updates, then sort by created_at and merge.
- Append messages from updates without any response_id at the end (global dangling), while aggregating metadata.

Python

```
static merge_updates(updates: list[AgentRunResponseUpdate], response_id: str) -> AgentRunResponse
```

Parameters

 Expand table

Name	Description
updates Required*	list[AgentRunResponseUpdate] The list of AgentRunResponseUpdate objects to merge.
response_id Required*	str The response identifier to set on the returned AgentRunResponse.

Returns

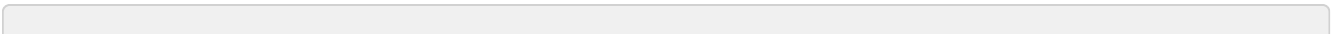
 Expand table

Type	Description
AgentRunResponse	An AgentRunResponse with messages in processing order and aggregated metadata.

run

Get a response from the workflow agent (non-streaming).

This method collects all streaming updates and merges them into a single response.



Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, checkpoint_id: str | None = None, checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

[Expand table](#)

Name	Description
messages	<code>str ChatMessage list[str] list[ChatMessage] None</code> The message(s) to send to the workflow. Required for new runs, should be None when resuming from checkpoint. Default value: None
thread Required*	<code>AgentThread None</code>
checkpoint_id Required*	<code>str None</code>
checkpoint_storage Required*	<code>CheckpointStorage None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
thread	The conversation thread. If None, a new thread will be created. Default value: None
checkpoint_id	ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. Default value: None
checkpoint_storage	Runtime checkpoint storage. When provided with <code>checkpoint_id</code> , used to load and restore the checkpoint. When provided without <code>checkpoint_id</code> , enables checkpointing for this run. Default value: None
**kwargs	

Name	Description
	Additional keyword arguments passed through to underlying workflow and ai_function tools.

Returns

[Expand table](#)

Type	Description
AgentRunResponse	The final workflow response as an AgentRunResponse.

run_stream

Stream response updates from the workflow agent.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] |
None = None, *, thread: AgentThread | None = None, checkpoint_id: str | None =
None, checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) ->
AsyncIterable[AgentRunResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the workflow. Required for new runs, should be None when resuming from checkpoint. Default value: None
thread Required*	AgentThread None
checkpoint_id Required*	str None
checkpoint_storage Required*	CheckpointStorage None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
thread	The conversation thread. If None, a new thread will be created. Default value: None
checkpoint_id	ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. Default value: None
checkpoint_storage	Runtime checkpoint storage. When provided with checkpoint_id, used to load and restore the checkpoint. When provided without checkpoint_id, enables checkpointing for this run. Default value: None
**kwargs	Additional keyword arguments passed through to underlying workflow and ai_function tools.

Returns

[Expand table](#)

Type	Description
AsyncIterable [AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```


Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

`display_name`

Returns the display name of the agent.

This is the name if present, otherwise the id.

pending_requests

workflow

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

REQUEST_INFO_FUNCTION_NAME

Python

```
REQUEST_INFO_FUNCTION_NAME: ClassVar[str] = 'request_info'
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[Middleware] | None
```

RequestInfoFunctionArgs Class

RequestInfoFunctionArgs(request_id: str, data: Any)

Constructor

Python
RequestInfoFunctionArgs()

Parameters

 Expand table

Name	Description
request_id Required*	
data Required*	

Methods

 Expand table

__init__
__new__
from_dict
from_json
to_dict
to_json

`__init__`

Python
<code>__init__(request_id: str, data: Any) -> None</code>

Parameters

 Expand table

Name	Description
request_id Required*	str
data Required*	Any

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

from_dict

Python
<code>from_dict(payload: dict[str, Any]) -> RequestInfoFunctionArgs</code>

Parameters

 Expand table

Name	Description
payload Required*	dict[str, Any]

Returns

[Expand table](#)

Type	Description
RequestInfoFunctionArgs	

from_json

Python
<pre>from_json(raw: str) -> RequestInfoFunctionArgs</pre>

Parameters

[Expand table](#)

Name	Description
raw Required*	str

Returns

[Expand table](#)

Type	Description
RequestInfoFunctionArgs	

to_dict

Python
<pre>to_dict() -> dict[str, Any]</pre>

Returns

[Expand table](#)

Type	Description
dict[str, Any]	

to_json

Python

```
to_json() -> str
```

Returns

 Expand table

Type	Description
<code>str</code>	

Attributes

data

Python

```
data: Any
```

request_id

Python

```
request_id: str
```

WorkflowBuilder Class

A builder class for constructing workflows.

This class provides a fluent API for defining workflow graphs by connecting executors with edges and configuring execution parameters. Call `build` to create an immutable `Workflow` instance.

Initialize the `WorkflowBuilder` with an empty list of edges and no starting executor.

Constructor

Python
<code>WorkflowBuilder()</code>

Parameters

 Expand table

Name	Description
<code>max_iterations</code>	Maximum number of iterations for workflow convergence. Default is 100. Default value: 100
<code>name</code>	Optional human-readable name for the workflow. Default value: None
<code>description</code>	Optional description of what the workflow does. Default value: None

Examples

Python
<pre>from typing_extensions import Never from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler class UpperCaseExecutor(Executor): @handler async def process(self, text: str, ctx: WorkflowContext[str]) -> None: await ctx.send_message(text.upper())</pre>


```

class ReverseExecutor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) -> None:
        await ctx.yield_output(text[::-1])

# Build a workflow
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: UpperCaseExecutor(id="upper"), name="UpperCase")
    .register_executor(lambda: ReverseExecutor(id="reverse"), name="Reverse")
    .add_edge("UpperCase", "Reverse")
    .set_start_executor("UpperCase")
    .build()
)

# Run the workflow
events = await workflow.run("hello")
print(events.get_outputs()) # ['OLLEH']

```

Methods

[Expand table](#)

<code>__init__</code>	Initialize the WorkflowBuilder with an empty list of edges and no starting executor.
<code>__new__</code>	
<code>add_agent</code>	Add an agent to the workflow by wrapping it in an AgentExecutor. This method creates an AgentExecutor that wraps the agent with the given parameters and ensures that subsequent uses of the same agent instance in other builder methods (like <code>add_edge</code>, <code>set_start_executor</code>, etc.) will reuse the same wrapped executor. Note: Agents adapt their behavior based on how the workflow is executed: <code>run_stream()</code>: Agents emit incremental <code>AgentRunUpdateEvent</code> events as tokens are produced <code>run()</code>: Agents emit a single <code>AgentRunEvent</code> containing the complete response
<code>add_chain</code>	Add a chain of executors to the workflow. The output of each executor in the chain will be sent to the next executor in the chain. The input types of each executor must be compatible with the output types of the previous executor.

Cycles in the chain are not allowed, meaning an executor cannot appear more than once in the chain.

Note: If executor instances are provided, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

[add_edge](#)

Add a directed edge between two executors.

The output types of the source and the input types of the target must be compatible. Messages sent by the source executor will be routed to the target executor.

Note: If instances are provided for both source and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

[add_fan_in_edges](#)

Add multiple edges from sources to a single target executor.

The edges will be grouped together for synchronized processing, meaning the target executor will only be executed once all source executors have completed.

The target executor will receive a list of messages aggregated from all source executors. Thus the input types of the target executor must be compatible with a list of the output types of the source executors.

Note: If instances are provided for sources and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

[add_fan_out_edges](#)

Add multiple edges to the workflow where messages from the source will be sent to all targets.

The output types of the source and the input types of the targets must be compatible. Messages from the source will be broadcast to all target executors concurrently.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

[add_multi_selection_edge_group](#)

Add an edge group that represents a multi-selection execution model.

The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to multiple target executors based on the provided selection function.

The selection function should take a message and a list of target executor IDs, and return a list of executor IDs indicating which target executors should receive the message.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

`add_switch_case_edge_group`

Add an edge group that represents a switch-case statement.

The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to one of the target executors based on the provided conditions.

Think of this as a switch statement where each target executor corresponds to a case. Each condition function will be evaluated in order, and the first one that returns True will determine which target executor receives the message.

The default case (if provided) will receive messages that fall through all conditions (i.e., no condition matched).

Note: If instances are provided for source and case targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

`build`

Build and return the constructed workflow.

This method performs validation before building the workflow to ensure:

- A starting executor has been set
- All edges connect valid executors
- The graph is properly connected
- Type compatibility between connected executors

`register_agent`

Register an agent factory function for lazy initialization.

This method allows you to register a factory function that creates an agent. The agent will be instantiated and wrapped in an AgentExecutor only when the workflow is built, enabling deferred initialization and potentially reducing startup time.

`register_executor`

Register an executor factory function for lazy initialization.

	This method allows you to register a factory function that creates an executor. The executor will be instantiated only when the workflow is built, enabling deferred initialization and potentially reducing startup time.
<code>set_max_iterations</code>	Set the maximum number of iterations for the workflow. When a workflow contains cycles, this limit prevents infinite loops by capping the total number of executor invocations. The default is 100 iterations.
<code>set_start_executor</code>	Set the starting executor for the workflow. The start executor is the entry point for the workflow. When the workflow is executed, the initial message will be sent to this executor.
<code>with_checkpointing</code>	Enable checkpointing with the specified storage. Checkpointing allows workflows to save their state periodically, enabling pause/resume functionality and recovery from failures. The checkpoint storage implementation determines where checkpoints are persisted.

`__init__`

Initialize the WorkflowBuilder with an empty list of edges and no starting executor.

Python

```
__init__(max_iterations: int = DEFAULT_MAX_ITERATIONS, name: str | None = None,
description: str | None = None)
```

Parameters

 Expand table

Name	Description
<code>max_iterations</code>	<code>int</code> Maximum number of iterations for workflow convergence. Default is 100. Default value: <code>DEFAULT_MAX_ITERATIONS</code>
<code>name</code>	<code>str</code> <code>None</code> Optional human-readable name for the workflow. Default value: <code>None</code>
<code>description</code>	<code>str</code> <code>None</code> Optional description of what the workflow does. Default value: <code>None</code>

__new__

```
Python

__new__(**kwargs)
```

add_agent

Add an agent to the workflow by wrapping it in an AgentExecutor.

This method creates an AgentExecutor that wraps the agent with the given parameters and ensures that subsequent uses of the same agent instance in other builder methods (like add_edge, set_start_executor, etc.) will reuse the same wrapped executor.

Note: Agents adapt their behavior based on how the workflow is executed:

- run_stream(): Agents emit incremental AgentRunUpdateEvent events as tokens are produced
- run(): Agents emit a single AgentRunEvent containing the complete response

```
Python

add_agent(agent: AgentProtocol, agent_thread: Any | None = None, output_response: bool = False, id: str | None = None) -> Self
```

Parameters

 Expand table

Name	Description
agent Required*	AgentProtocol The agent to add to the workflow.
agent_thread	Any None The thread to use for running the agent. If None, a new thread will be created. Default value: None
output_response	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes. Default value: False
id	str None

Name	Description
	A unique identifier for the executor. If None, the agent's name will be used if available. Default value: None

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the provided id or agent name conflicts with an existing executor.

Examples

Python

```
from agent_framework import WorkflowBuilder
from agent_framework_anthropic import AnthropicAgent

# Create an agent
agent = AnthropicAgent(name="writer", model="claude-3-5-sonnet-20241022")

# Add the agent to a workflow
workflow = WorkflowBuilder().add_agent(agent,
output_response=True).set_start_executor(agent).build()
```

add_chain

Add a chain of executors to the workflow.

The output of each executor in the chain will be sent to the next executor in the chain. The input types of each executor must be compatible with the output types of the previous executor.

Cycles in the chain are not allowed, meaning an executor cannot appear more than once in the chain.

Note: If executor instances are provided, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_chain(executors: Sequence[Executor | AgentProtocol | str]) -> Self
```

Parameters

[Expand table](#)

Name	Description
executors Required*	Sequence [Executor AgentProtocol str] A list of executors or registered names of the executor factories to chain together.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class Step1(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())

class Step2(Executor):
    @handler
```

```

    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text[::-1])

class Step3(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(f"Final: {text}")

# Chain executors in sequence
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: Step1(id="step1"), name="step1")
    .register_executor(lambda: Step2(id="step2"), name="step2")
    .register_executor(lambda: Step3(id="step3"), name="step3")
    .add_chain(["step1", "step2", "step3"])
    .set_start_executor("step1")
    .build()
)

```

add_edge

Add a directed edge between two executors.

The output types of the source and the input types of the target must be compatible. Messages sent by the source executor will be routed to the target executor.

Note: If instances are provided for both source and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```

add_edge(source: Executor | AgentProtocol | str, target: Executor | AgentProtocol
| str, condition: Callable[[Any], bool] | None = None) -> Self

```

Parameters

 Expand table

Name	Description
source Required*	Executor AgentProtocol str

Name	Description
target Required*	Executor AgentProtocol str
condition	Callable [[Any], bool] None Default value: None

Returns

 Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class ProcessorA(Executor):
    @handler
    async def process(self, data: str, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(len(data))

class ProcessorB(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(f"Processed {count} characters")

# Connect executors with an edge
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: ProcessorA(id="a"), name="ProcessorA")
    .register_executor(lambda: ProcessorB(id="b"), name="ProcessorB")
    .add_edge("ProcessorA", "ProcessorB")
    .set_start_executor("ProcessorA")
    .build()
)

# With a condition
def only_large_numbers(msg: int) -> bool:
```

```

        return msg > 100

    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: ProcessorA(id="a"), name="ProcessorA")
        .register_executor(lambda: ProcessorB(id="b"), name="ProcessorB")
        .add_edge("ProcessorA", "ProcessorB", condition=only_large_numbers)
        .set_start_executor("ProcessorA")
        .build()
    )

```

add_fan_in_edges

Add multiple edges from sources to a single target executor.

The edges will be grouped together for synchronized processing, meaning the target executor will only be executed once all source executors have completed.

The target executor will receive a list of messages aggregated from all source executors. Thus the input types of the target executor must be compatible with a list of the output types of the source executors.

Note: If instances are provided for sources and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```

add_fan_in_edges(sources: Sequence[Executor | AgentProtocol | str], target:
Executor | AgentProtocol | str) -> Self

```

Parameters

[Expand table](#)

Name	Description
sources Required*	Sequence [Executor AgentProtocol str] A list of source executors or registered names of the source factories for the edges.
target Required*	Executor AgentProtocol str The target executor or registered name of the target factory for the edges.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class Producer(Executor):
    @handler
    async def produce(self, seed: int, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(f"result_{seed}")

class Aggregator(Executor):
    @handler
    async def aggregate(self, results: list[str], ctx: WorkflowContext[Never, str]) -> None:
        combined = ", ".join(results)
        await ctx.yield_output(f"Combined: {combined}")

# Collect results from multiple producers
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: Producer(id="prod_1"), name="Producer1")
    .register_executor(lambda: Producer(id="prod_2"), name="Producer2")
    .register_executor(lambda: Aggregator(id="agg"), name="Aggregator")
    .add_fan_in_edges(["Producer1", "Producer2"], "Aggregator")
    .set_start_executor("Producer1")
    .build()
)
```

add_fan_out_edges

Add multiple edges to the workflow where messages from the source will be sent to all targets.

The output types of the source and the input types of the targets must be compatible. Messages from the source will be broadcast to all target executors concurrently.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_fan_out_edges(source: Executor | AgentProtocol | str, targets:
Sequence[Executor | AgentProtocol | str]) -> Self
```

Parameters

[Expand table](#)

Name	Description
source Required*	Executor AgentProtocol str The source executor or registered name of the source factory for the edges.
targets Required*	Sequence [Executor AgentProtocol str] A list of target executors or registered names of the target factories for the edges.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class DataSource(Executor):
    @handler
    async def generate(self, count: int, ctx: WorkflowContext[str]) -> None:
        for i in range(count):
            await ctx.send_message(f"data_{i}")

class ValidatorA(Executor):
    @handler
```

```

    async def validate(self, data: str, ctx: WorkflowContext) -> None:
        print(f"ValidatorA: {data}")

class ValidatorB(Executor):
    @handler
    async def validate(self, data: str, ctx: WorkflowContext) -> None:
        print(f"ValidatorB: {data}")

# Broadcast to multiple validators
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: DataSource(id="source"), name="DataSource")
    .register_executor(lambda: ValidatorA(id="val_a"), name="ValidatorA")
    .register_executor(lambda: ValidatorB(id="val_b"), name="ValidatorB")
    .add_fan_out_edges("DataSource", ["ValidatorA", "ValidatorB"])
    .set_start_executor("DataSource")
    .build()
)

```

add_multi_selection_edge_group

Add an edge group that represents a multi-selection execution model.

The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to multiple target executors based on the provided selection function.

The selection function should take a message and a list of target executor IDs, and return a list of executor IDs indicating which target executors should receive the message.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```

add_multi_selection_edge_group(source: Executor | AgentProtocol | str, targets:
Sequence[Executor | AgentProtocol | str], selection_func: Callable[[Any,
list[str]], list[str]]) -> Self

```

Parameters

[Expand table](#)

Name	Description
source Required*	Executor AgentProtocol str The source executor or registered name of the source factory for the edge group.
targets Required*	Sequence [Executor AgentProtocol str] A list of target executors or registered names of the target factories for the edges.
selection_func Required*	Callable [[Any , list [str]], list [str]] A function that selects target executors for messages. Takes (message, list [executor_id]) and returns list [executor_id].

Returns

 Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler
from dataclasses import dataclass

@dataclass
class Task:
    priority: str
    data: str

class TaskDispatcher(Executor):
    @handler
    async def dispatch(self, text: str, ctx: WorkflowContext[Task]) -> None:
        priority = "high" if len(text) > 10 else "low"
        await ctx.send_message(Task(priority=priority, data=text))

class WorkerA(Executor):
    @handler
    async def process(self, task: Task, ctx: WorkflowContext) -> None:
        print(f"WorkerA processing: {task.data}")

class WorkerB(Executor):
    @handler
```

```

    async def process(self, task: Task, ctx: WorkflowContext) -> None:
        print(f"WorkerB processing: {task.data}")

# Select workers based on task priority
def select_workers(task: Task, available: list[str]) -> list[str]:
    if task.priority == "high":
        return available # Send to all workers
    return [available[0]] # Send to first worker only

workflow = (
    WorkflowBuilder()
        .register_executor(lambda: TaskDispatcher(id="dispatcher"),
name="TaskDispatcher")
        .register_executor(lambda: WorkerA(id="worker_a"), name="WorkerA")
        .register_executor(lambda: WorkerB(id="worker_b"), name="WorkerB")
        .add_multi_selection_edge_group(
            "TaskDispatcher",
            ["WorkerA", "WorkerB"],
            selection_func=select_workers,
        )
        .set_start_executor("TaskDispatcher")
        .build()
)

```

add_switch_case_edge_group

Add an edge group that represents a switch-case statement.

The output types of the source and the input types of the targets must be compatible.

Messages from the source executor will be sent to one of the target executors based on the provided conditions.

Think of this as a switch statement where each target executor corresponds to a case. Each condition function will be evaluated in order, and the first one that returns True will determine which target executor receives the message.

The default case (if provided) will receive messages that fall through all conditions (i.e., no condition matched).

Note: If instances are provided for source and case targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

```
add_switch_case_edge_group(source: Executor | AgentProtocol | str, cases:
Sequence[Case | Default]) -> Self
```

Parameters

 Expand table

Name	Description
source Required*	Executor AgentProtocol str The source executor or registered name of the source factory for the edge group.
cases Required*	Sequence [Case Default] A list of case objects that determine the target executor for each message.

Returns

 Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext,
handler, Case, Default
from dataclasses import dataclass

@dataclass
class Result:
    score: int

class Evaluator(Executor):
    @handler
    async def evaluate(self, text: str, ctx: WorkflowContext[Result]) -> None:
        await ctx.send_message(Result(score=len(text)))

class HighScoreHandler(Executor):
    @handler
    async def handle(self, result: Result, ctx: WorkflowContext) -> None:
        print(f"High score: {result.score}")
```



```

class LowScoreHandler(Executor):
    @handler
    async def handle(self, result: Result, ctx: WorkflowContext) -> None:
        print(f"Low score: {result.score}")

# Route based on score value
workflow = (
    WorkflowBuilder()
        .register_executor(lambda: Evaluator(id="eval"), name="Evaluator")
        .register_executor(lambda: HighScoreHandler(id="high"),
name="HighScoreHandler")
        .register_executor(lambda: LowScoreHandler(id="low"),
name="LowScoreHandler")
        .add_switch_case_edge_group(
            "Evaluator",
            [
                Case(condition=lambda r: r.score > 10, target="HighScoreHandler"),
                Default(target="LowScoreHandler"),
            ],
        )
        .set_start_executor("Evaluator")
        .build()
)

```

build

Build and return the constructed workflow.

This method performs validation before building the workflow to ensure:

- A starting executor has been set
- All edges connect valid executors
- The graph is properly connected
- Type compatibility between connected executors

Python

build() -> Workflow

Returns

[Expand table](#)

Type	Description
Workflow	An immutable Workflow instance ready for execution.

Exceptions

[Expand table](#)

Type	Description
ValueError	If starting executor is not set.
WorkflowValidationError	If workflow validation fails (includes EdgeDuplicationError , TypeCompatibilityError , and GraphConnectivityError subclasses).

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class MyExecutor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(text.upper())

# Build and execute a workflow
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: MyExecutor(id="executor"), name="MyExecutor")
    .set_start_executor("MyExecutor")
    .build()
)

# The workflow is now immutable and ready to run
events = await workflow.run("hello")
print(events.get_outputs()) # ['HELLO']

# Workflows can be reused multiple times
events2 = await workflow.run("world")
print(events2.get_outputs()) # ['WORLD']
```

register_agent

Register an agent factory function for lazy initialization.

This method allows you to register a factory function that creates an agent. The agent will be instantiated and wrapped in an AgentExecutor only when the workflow is built, enabling

deferred initialization and potentially reducing startup time.

Python

```
register_agent(factory_func: Callable[[], AgentProtocol], name: str, agent_thread: AgentThread | None = None, output_response: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
factory_func Required*	Callable[[], AgentProtocol] A callable that returns an AgentProtocol instance when called.
name Required*	str The name of the registered agent factory. This doesn't have to match the agent's internal name. But it must be unique within the workflow.
agent_thread	AgentThread None The thread to use for running the agent. If None, a new thread will be created when the agent is instantiated. Default value: None
output_response	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes. Default value: False

Returns

[Expand table](#)

Type	Description
Self	

Examples

Python

```
from agent_framework import WorkflowBuilder
from agent_framework_anthropic import AnthropicAgent
```

```
# Build a workflow
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: ..., name="SomeOtherExecutor")
    .register_agent(
        lambda: AnthropicAgent(name="writer", model="claude-3-5-sonnet-
20241022"),
        name="WriterAgent",
        output_response=True,
    )
    .add_edge("SomeOtherExecutor", "WriterAgent")
    .set_start_executor("SomeOtherExecutor")
    .build()
)
```

register_executor

Register an executor factory function for lazy initialization.

This method allows you to register a factory function that creates an executor. The executor will be instantiated only when the workflow is built, enabling deferred initialization and potentially reducing startup time.

Python

```
register_executor(factory_func: Callable[[], Executor], name: str | list[str]) ->
Self
```

Parameters

[Expand table](#)

Name	Description
factory_func Required*	Callable[[], Executor] A callable that returns an Executor instance when called.
name Required*	str list[str] The name(s) of the registered executor factory. This doesn't have to match the executor's ID, but it must be unique within the workflow.

Returns

[Expand table](#)

Type	Description
Self	

Examples

If multiple names are provided, the same factory function will be registered under each name.

Python

```
from agent_framework import WorkflowBuilder, Executor, WorkflowContext, handler

class LoggerExecutor(Executor):
    @handler
    async def log(self, message: str, ctx: WorkflowContext) -> None:
        print(f"Log: {message}")

# Register the same executor factory under multiple names
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: LoggerExecutor(id="logger"), name=["ExecutorA",
"ExecutorB"])
    .set_start_executor("ExecutorA")
    .add_edge("ExecutorA", "ExecutorB")
    .build()
```

set_max_iterations

Set the maximum number of iterations for the workflow.

When a workflow contains cycles, this limit prevents infinite loops by capping the total number of executor invocations. The default is 100 iterations.

Python

```
set_max_iterations(max_iterations: int) -> Self
```

Parameters

 Expand table

Name	Description
max_iterations	int

Name	Description
Required*	The maximum number of iterations the workflow will run for convergence.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class StepA(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[int]) -> None:
        if count < 10:
            await ctx.send_message(count + 1)

class StepB(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(count)

# Set a custom iteration limit for workflow with cycles
workflow = (
    WorkflowBuilder()
    .set_max_iterations(500)
    .register_executor(lambda: StepA(id="step_a"), name="StepA")
    .register_executor(lambda: StepB(id="step_b"), name="StepB")
    .add_edge("StepA", "StepB")
    .add_edge("StepB", "StepA") # Cycle
    .set_start_executor("StepA")
    .build()
)
```

set_start_executor

Set the starting executor for the workflow.

The start executor is the entry point for the workflow. When the workflow is executed, the initial message will be sent to this executor.

Python

```
set_start_executor(executor: Executor | AgentProtocol | str) -> Self
```

Parameters

[Expand table](#)

Name	Description
executor Required*	Executor AgentProtocol str The starting executor, which can be an Executor instance, AgentProtocol instance, or the name of a registered executor factory.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler

class EntryPoint(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())

class Processor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(text)

workflow = (
```

```

WorkflowBuilder()
    .register_executor(lambda: EntryPoint(id="entry"), name="EntryPoint")
    .register_executor(lambda: Processor(id="proc"), name="Processor")
    .add_edge("EntryPoint", "Processor")
    .set_start_executor("EntryPoint")
    .build()
)

```

with_checkpointing

Enable checkpointing with the specified storage.

Checkpointing allows workflows to save their state periodically, enabling pause/resume functionality and recovery from failures. The checkpoint storage implementation determines where checkpoints are persisted.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> Self
```

Parameters

[Expand table](#)

Name	Description
checkpoint_storage Required*	CheckpointStorage The checkpoint storage implementation to use.

Returns

[Expand table](#)

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```

from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler
from agent_framework import FileCheckpointStorage

```



```

class ProcessorA(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())

class ProcessorB(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(text)

# Enable checkpointing with file-based storage
storage = FileCheckpointStorage("./checkpoints")
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: ProcessorA(id="proc_a"), name="ProcessorA")
    .register_executor(lambda: ProcessorB(id="proc_b"), name="ProcessorB")
    .add_edge("ProcessorA", "ProcessorB")
    .set_start_executor("ProcessorA")
    .with_checkpointing(storage)
    .build()
)

# Run with checkpoint saving
events = await workflow.run("input")

```

WorkflowCheckpoint Class

Represents a complete checkpoint of workflow state.

Checkpoints capture the full execution state of a workflow at a specific point, enabling workflows to be paused and resumed.

 Note

The `shared_state` dict may contain reserved keys managed by the framework.

See `SharedState` class documentation for details on reserved keys.

Constructor

Python
<code>WorkflowCheckpoint()</code>

Parameters

 Expand table

Name	Description
<code>checkpoint_id</code>	Default value: <factory>
<code>workflow_id</code>	
<code>timestamp</code>	Default value: <factory>
<code>messages</code>	Default value: <factory>
<code>shared_state</code>	Default value: <factory>
<code>pending_request_info_events</code>	Default value: <factory>
<code>iteration_count</code>	Default value: 0
<code>metadata</code>	Default value: <factory>
<code>version</code>	Default value: 1.0

Methods

[Expand table](#)

__init__
__new__
from_dict
to_dict

[__init__](#)

Python

```
__init__(checkpoint_id: str = <factory>, workflow_id: str = '', timestamp: str = <factory>, messages: dict[str, list[dict[str, typing.Any]]] = <factory>, shared_state: dict[str, typing.Any] = <factory>, pending_request_info_events: dict[str, dict[str, typing.Any]] = <factory>, iteration_count: int = 0, metadata: dict[str, typing.Any] = <factory>, version: str = '1.0') -> None
```

Parameters

[Expand table](#)

Name	Description
checkpoint_id	str Default value: <factory>
workflow_id Required*	str
timestamp	str Default value: <factory>
messages	dict[str, list[dict[str, Any]]] Default value: <factory>
shared_state	dict[str, Any] Default value: <factory>
pending_request_info_events	dict[str, dict[str, Any]] Default value: <factory>
iteration_count	int Default value: 0

Name	Description
metadata	<code>dict[str, Any]</code> Default value: <factory>
version	<code>str</code> Default value: 1.0

Returns

[Expand table](#)

Type	Description
<code>None</code>	

`__new__`

Python
<code>__new__(**kwargs)</code>

`from_dict`

Python
<code>from_dict(data: Mapping[str, Any]) -> WorkflowCheckpoint</code>

Parameters

[Expand table](#)

Name	Description
<code>data</code> Required*	<code>Mapping[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>WorkflowCheckpoint</code>	

to_dict

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

Attributes

checkpoint_id

Unique identifier for this checkpoint

Python
<code>checkpoint_id: str</code>

iteration_count

Current iteration number when checkpoint was created

Python
<code>iteration_count: int</code>

messages

Messages exchanged between executors

Python
<code>messages: dict[str, list[dict[str, Any]]]</code>

metadata

Additional metadata (e.g., superstep info, graph signature)

Python

```
metadata: dict[str, Any]
```

pending_request_info_events

Python

```
pending_request_info_events: dict[str, dict[str, Any]]
```

shared_state

Complete shared state including user data and executor states. Executor states are stored under the reserved key '_executor_state'.

Python

```
shared_state: dict[str, Any]
```

timestamp

ISO 8601 timestamp when checkpoint was created

Python

```
timestamp: str
```

version

Checkpoint format version

Python

```
version: str
```

workflow_id

Identifier of the workflow this checkpoint belongs to

Python

```
workflow_id: str
```

WorkflowCheckpointSummary Class

Human-readable summary of a workflow checkpoint.

Constructor

Python
<code>WorkflowCheckpointSummary()</code>

Parameters

 Expand table

Name	Description
<code>checkpoint_id</code> Required*	
<code>timestamp</code> Required*	
<code>iteration_count</code> Required*	
<code>targets</code> Required*	
<code>executor_ids</code> Required*	
<code>status</code> Required*	
<code>pending_request_info_events</code> Required*	

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python

```
__init__(checkpoint_id: str, timestamp: str, iteration_count: int, targets: list[str], executor_ids: list[str], status: str, pending_request_info_events: list[RequestInfoEvent]) -> None
```

Parameters

 Expand table

Name	Description
checkpoint_id Required*	<code>str</code>
timestamp Required*	<code>str</code>
iteration_count Required*	<code>int</code>
targets Required*	<code>list[str]</code>
executor_ids Required*	<code>list[str]</code>
status Required*	<code>str</code>
pending_request_info_events Required*	<code>list[RequestInfoEvent]</code>

Returns

 Expand table

Type	Description
<code>None</code>	

`__new__`

Python

```
__new__(**kwargs)
```

Attributes

checkpoint_id

Python

```
checkpoint_id: str
```

executor_ids

Python

```
executor_ids: list[str]
```

iteration_count

Python

```
iteration_count: int
```

pending_request_info_events

Python

```
pending_request_info_events: list[RequestInfoEvent]
```

status

Python

```
status: str
```

targets

Python

```
targets: list[str]
```

timestamp

Python

```
timestamp: str
```

WorkflowContext Class

Execution context that enables executors to interact with workflows and other executors.

Overview

WorkflowContext provides a controlled interface for executors to send messages, yield outputs, manage state, and interact with the broader workflow ecosystem. It enforces type safety through generic parameters while preventing direct access to internal runtime components.

Type Parameters

The context is parameterized to enforce type safety for different operations:

WorkflowContext (no parameters)

For executors that only perform side effects without sending messages or yielding outputs:

Python

```
async def log_handler(message: str, ctx: WorkflowContext) -> None:
    print(f"Received: {message}") # Only side effects
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out to other executors:

Python

```
async def processor(message: str, ctx: WorkflowContext[int]) -> None:
    result = len(message)
    await ctx.send_message(result) # Send int to downstream executors
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```
async def dual_output(message: str, ctx: WorkflowContext[int, str]) -> None:
    await ctx.send_message(42) # Send int message
    await ctx.yield_output("complete") # Yield str workflow output
```

Union Types

Multiple types can be specified using union notation:

Python

```
async def flexible(message: str, ctx: WorkflowContext[int | str, bool | dict]) ->
None:
    await ctx.send_message("text") # or send 42
    await ctx.yield_output(True) # or yield {"status": "done"}
```

Initialize the executor context with the given workflow context.

Constructor

Python

```
WorkflowContext()
```

Parameters

[Expand table](#)

Name	Description
executor Required*	The executor instance that this context belongs to.
source_executor_ids Required*	The IDs of the source executors that sent messages to this executor. This is a list to support fan_in scenarios where multiple sources send aggregated messages to the same executor.
shared_state Required*	The shared state for the workflow.
runner_context Required*	The runner context that provides methods to send messages and events.
trace_contexts	Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None

Name	Description
source_span_ids	Optional source span IDs from multiple sources for linking (not for nesting). Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the executor context with the given workflow context.
<code>__new__</code>	
<code>add_event</code>	Add an event to the workflow context.
<code>get_executor_state</code>	Retrieve previously persisted state for this executor, if any.
<code>get_sent_messages</code>	Get all messages sent via <code>send_message()</code> during this handler execution.
<code>get_shared_state</code>	Get a value from the shared state.
<code>get_source_executor_id</code>	Get the ID of the source executor that sent the message to this executor.
<code>get_yielded_outputs</code>	Get all outputs yielded via <code>yield_output()</code> during this handler execution.
<code>is_streaming</code>	Check if the workflow is running in streaming mode.
<code>request_info</code>	Request information from outside of the workflow. Calling this method will cause the workflow to emit a <code>RequestInfoEvent</code>, carrying the provided <code>request_data</code> and <code>request_type</code>. External systems listening for such events can then process the request and respond accordingly. Executors must have the corresponding response handlers defined using the <code>@response_handler</code> decorator to handle the incoming responses.
<code>send_message</code>	Send a message to the workflow context.
<code>set_executor_state</code>	Store executor state in shared state under a reserved key. Executors call this with a JSON-serializable dict capturing the minimal state needed to resume. It replaces any previously stored state.
<code>set_shared_state</code>	Set a value in the shared state.
<code>yield_output</code>	Set the output of the workflow.

`__init__`

Initialize the executor context with the given workflow context.

Python

```
__init__(executor: Executor, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None)
```

Parameters

 Expand table

Name	Description
executor Required*	Executor The executor instance that this context belongs to.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor. This is a list to support fan_in scenarios where multiple sources send aggregated messages to the same executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] None Optional source span IDs from multiple sources for linking (not for nesting). Default value: None

__new__

Python

```
__new__(**kwargs)
```

add_event

Add an event to the workflow context.

Python

```
async add_event(event: WorkflowEvent) -> None
```

Parameters

[Expand table](#)

Name	Description
event Required*	WorkflowEvent

Returns

[Expand table](#)

Type	Description
None	

get_executor_state

Retrieve previously persisted state for this executor, if any.

Python

```
get_executor_state() -> dict[str, Any] | None
```

Returns

[Expand table](#)

Type	Description
dict[str, Any] None	

get_sent_messages

Get all messages sent via `send_message()` during this handler execution.

Python

```
get_sent_messages() -> list[Any]
```


Returns

[Expand table](#)

Type	Description
<code>list[Any]</code>	A list of messages that were sent to downstream executors.

get_shared_state

Get a value from the shared state.

Python
<code>async get_shared_state(key: str) -> Any</code>

Parameters

[Expand table](#)

Name	Description
<code>key</code> Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code>Any</code>	

get_source_executor_id

Get the ID of the source executor that sent the message to this executor.

Python
<code>get_source_executor_id() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

Exceptions

[Expand table](#)

Type	Description
<code>RuntimeError</code>	If there are multiple source executors, this method raises an error.

get_yielded_outputs

Get all outputs yielded via `yield_output()` during this handler execution.

Python
<code>get_yielded_outputs() -> list[Any]</code>

Returns

[Expand table](#)

Type	Description
<code>list[Any]</code>	A list of outputs that were yielded as workflow outputs.

is_streaming

Check if the workflow is running in streaming mode.

Python
<code>is_streaming() -> bool</code>

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if the workflow was started with <code>run_stream()</code> , False if started with <code>run()</code> .

request_info

Request information from outside of the workflow.

Calling this method will cause the workflow to emit a `RequestInfoEvent`, carrying the provided `request_data` and `request_type`. External systems listening for such events can then process the request and respond accordingly.

Executors must have the corresponding response handlers defined using the `@response_handler` decorator to handle the incoming responses.

Python

```
async request_info(request_data: object, response_type: type) -> None
```

Parameters

 Expand table

Name	Description
request_data Required*	<code>object</code> The data associated with the information request.
response_type Required*	<code>type</code> The expected type of the response, used for validation.

Returns

 Expand table

Type	Description
<code>None</code>	

send_message

Send a message to the workflow context.

--

Python

```
async send_message(message: T_Out, target_id: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
message Required*	<xref:agent_framework._workflows._workflow_context.T_Out> The message to send. This must conform to the output type(s) declared on this context.
target_id	<code>str None</code> The ID of the target executor to send the message to. If None, the message will be sent to all target executors. Default value: None

Returns

[Expand table](#)

Type	Description
<code>None</code>	

set_executor_state

Store executor state in shared state under a reserved key.

Executors call this with a JSON-serializable dict capturing the minimal state needed to resume. It replaces any previously stored state.

Python

```
set_executor_state(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
state Required*	<code>dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code>None</code>	

set_shared_state

Set a value in the shared state.

Python
<code>async set_shared_state(key: str, value: Any) -> None</code>

Parameters

 Expand table

Name	Description
key Required*	<code>str</code>
value Required*	<code>Any</code>

Returns

 Expand table

Type	Description
<code>None</code>	

yield_output

Set the output of the workflow.

Python
<code>async yield_output(output: T_W_Out) -> None</code>

Parameters

 Expand table

Name	Description
output Required*	<xref:agent_framework._workflows._workflow_context.T_W_Out> The output to yield. This must conform to the workflow output type(s) declared on this context.

Returns

 Expand table

Type	Description
None	

Attributes

shared_state

Get the shared state.

source_executor_ids

Get the IDs of the source executors that sent messages to this executor.

WorkflowErrorDetails Class

Structured error information to surface in error events/results.

Constructor

Python
<code>WorkflowErrorDetails()</code>

Parameters

[Expand table](#)

Name	Description
<code>error_type</code> Required*	
<code>message</code> Required*	
<code>traceback</code>	Default value: None
<code>executor_id</code>	Default value: None
<code>extra</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>
<code>__new__</code>
<code>from_exception</code>

`__init__`

Python
<code>__init__(error_type: str, message: str, traceback: str None = None, executor_id:</code>

```
str | None = None, extra: dict[str, Any] | None = None) -> None
```

Parameters

 Expand table

Name	Description
error_type Required*	str
message Required*	str
traceback	str None Default value: None
executor_id	str None Default value: None
extra	dict[str, Any] None Default value: None

Returns

 Expand table

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

`from_exception`

Python

```
from_exception(exc: BaseException, *, executor_id: str | None = None, extra: dict[str, Any] | None = None) -> WorkflowErrorDetails
```

Parameters

[Expand table](#)

Name	Description
exc Required*	BaseException
executor_id Required*	str None
extra Required*	dict[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
executor_id	Default value: None
extra	Default value: None

Returns

[Expand table](#)

Type	Description
WorkflowErrorDetails	

Attributes

error_type

Python
<code>error_type: str</code>

executor_id

Python
<code>executor_id: str None = None</code>

extra

Python

```
extra: dict[str, Any] | None = None
```

message

Python

```
message: str
```

traceback

Python

```
traceback: str | None = None
```

WorkflowEvent Class

Base class for workflow events.

Initialize the workflow event with optional data.

Constructor

Python
<code>WorkflowEvent()</code>

Parameters

[Expand table](#)

Name	Description
<code>data</code>	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the workflow event with optional data.
<code>__new__</code>	

`__init__`

Initialize the workflow event with optional data.

Python
<code>__init__(data: Any None = None)</code>

Parameters

[Expand table](#)

Name	Description
data	Any None Default value: None

`__new__`

Python
<code>__new__(**kwargs)</code>

WorkflowEventSource Enum

Identifies whether a workflow event came from the framework or an executor.

Use *FRAMEWORK* for events emitted by built-in orchestration paths—even when the code that raises them lives in runner-related modules—and *EXECUTOR* for events surfaced by developer-provided executor implementations.

Fields

 Expand table

EXECUTOR
FRAMEWORK

WorkflowExecutor Class

An executor that wraps a workflow to enable hierarchical workflow composition.

Overview

WorkflowExecutor makes a workflow behave as a single executor within a parent workflow, enabling nested workflow architectures. It handles the complete lifecycle of sub-workflow execution including event processing, output forwarding, and request/response coordination between parent and child workflows.

Execution Model

When invoked, WorkflowExecutor:

1. Starts the wrapped workflow with the input message
2. Runs the sub-workflow to completion or until it needs external input
3. Processes the sub-workflow's complete event stream after execution
4. Forwards outputs to the parent workflow as messages
5. Handles external requests by routing them to the parent workflow
6. Accumulates responses and resumes sub-workflow execution

Event Stream Processing

WorkflowExecutor processes events after sub-workflow completion:

Output Forwarding

All outputs from the sub-workflow are automatically forwarded to the parent:

When *allow_direct_output* is False (default):

Python

```
# An executor in the sub-workflow yields outputs
await ctx.yield_output("sub-workflow result")

# WorkflowExecutor forwards to parent via ctx.send_message()
# Parent receives the output as a regular message
```

When *allow_direct_output* is True:

Request/Response Coordination

When sub-workflows need external information:

Python

```
# An executor in the sub-workflow makes request
request = MyDataRequest(query="user info")

# WorkflowExecutor captures RequestInfoEvent and wraps it in a
SubWorkflowRequestMessage
# then send it to the receiving executor in parent workflow. The executor in parent
workflow
# can handle the request locally or forward it to an external source.
# The WorkflowExecutor tracks the pending request, and implements a response
handler.
# When the response is received, it executes the response handler to accumulate
responses
# and resume the sub-workflow when all expected responses are received.
# The response handler expects a SubWorkflowResponseMessage wrapping the response
data.
```

State Management

WorkflowExecutor maintains execution state across request/response cycles:

- Tracks pending requests by request_id
- Accumulates responses until all expected responses are received
- Resumes sub-workflow execution with complete response batch
- Handles concurrent executions and multiple pending requests

Type System Integration

WorkflowExecutor inherits its type signature from the wrapped workflow:

Input Types

Matches the wrapped workflow's start executor input types:

Python

```
# If sub-workflow accepts str, WorkflowExecutor accepts str
```

```
workflow_executor = WorkflowExecutor(my_workflow, id="wrapper")
assert workflow_executor.input_types == my_workflow.input_types
```

Output Types

Combines sub-workflow outputs with request coordination types:

Python

```
# Includes all sub-workflow output types
# Plus SubWorkflowRequestMessage if sub-workflow can make requests
output_types = workflow.output_types + [SubWorkflowRequestMessage] # if applicable
```

Error Handling

WorkflowExecutor propagates sub-workflow failures:

- Captures WorkflowFailedEvent from sub-workflow
- Converts to WorkflowErrorEvent in parent context
- Provides detailed error information including sub-workflow ID

Concurrent Execution Support

WorkflowExecutor fully supports multiple concurrent sub-workflow executions:

Per-Execution State Isolation

Each sub-workflow invocation creates an isolated ExecutionContext:

Python

```
# Multiple concurrent invocations are supported
workflow_executor = WorkflowExecutor(my_workflow, id="concurrent_executor")

# Each invocation gets its own execution context
# Execution 1: processes input_1 independently
# Execution 2: processes input_2 independently
# No state interference between executions
```

Request/Response Coordination

Responses are correctly routed to the originating execution:

- Each execution tracks its own pending requests and expected responses
- Request-to-execution mapping ensures responses reach the correct sub-workflow
- Response accumulation is isolated per execution
- Automatic cleanup when execution completes

Memory Management

- Unlimited concurrent executions supported
- Each execution has unique UUID-based identification
- Cleanup of completed execution contexts
- Thread-safe state management for concurrent access

Important Considerations

Shared Workflow Instance: All concurrent executions use the same underlying workflow instance. For proper isolation, ensure that the wrapped workflow and its executors are stateless.

Python

```
# Avoid: Stateful executor with instance variables
class StatefulExecutor(Executor):
    def __init__(self):
        super().__init__(id="stateful")
        self.data = [] # This will be shared across concurrent executions!
```

Integration with Parent Workflows

Parent workflows can intercept sub-workflow requests:

Implementation Notes

- Sub-workflows run to completion before processing their results
- Event processing is atomic - all outputs are forwarded before requests
- Response accumulation ensures sub-workflows receive complete response batches
- Execution state is maintained for proper resumption after external requests
- Concurrent executions are fully isolated and do not interfere with each other

Initialize the WorkflowExecutor.

Constructor

Python

WorkflowExecutor()

Parameters

[Expand table](#)

Name	Description
workflow Required*	The workflow to execute as a sub-workflow.
id Required*	Unique identifier for this executor.
allow_direct_output	Whether to allow direct output from the sub-workflow. By default, outputs from the sub-workflow are sent to other executors in the parent workflow as messages. When this is set to true, the outputs are yielded directly from the WorkflowExecutor to the parent workflow's event stream. Default value: False

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional keyword arguments passed to the parent constructor.

Methods

[Expand table](#)

__init__	Initialize the WorkflowExecutor.
__new__	
can_handle	Override can_handle to only accept messages that the wrapped workflow can handle. This prevents the WorkflowExecutor from accepting messages that should go to other executors because the handler <i>process_workflow</i> has no type restrictions.
clone	
execute	Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using `@handler` decorator.

`from_dict`

`from_json`

`handle_response`

Handle response from parent for a forwarded request.

This handler accumulates responses and only resumes the sub-workflow when all expected responses have been received for that execution.

`is_request_supported`

Check if the executor supports request of the given type and handling a response of the given type.

`on_checkpoint_restore`

Restore the WorkflowExecutor state from a checkpoint snapshot.

`on_checkpoint_save`

Get the current state of the WorkflowExecutor for checkpointing purposes.

`process_workflow`

Execute the sub-workflow with raw input data.

This handler starts a new sub-workflow execution. When the sub-workflow needs external information, it pauses and sends a request to the parent.

`to_dict`

`to_json`

`__init__`

Initialize the WorkflowExecutor.

Python

```
__init__(workflow: Workflow, id: str, allow_direct_output: bool = False, **kwargs: Any)
```

Parameters

[Expand table](#)

Name	Description
workflow Required*	Workflow The workflow to execute as a sub-workflow.
id	str

Name	Description
Required*	Unique identifier for this executor.
allow_direct_output	bool Whether to allow direct output from the sub-workflow. By default, outputs from the sub-workflow are sent to other executors in the parent workflow as messages. When this is set to true, the outputs are yielded directly from the WorkflowExecutor to the parent workflow's event stream. Default value: False
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional keyword arguments passed to the parent constructor.

__new__

Python
<code>__new__(**kwargs)</code>

can_handle

Override can_handle to only accept messages that the wrapped workflow can handle.

This prevents the WorkflowExecutor from accepting messages that should go to other executors because the handler *process_workflow* has no type restrictions.

Python
<code>can_handle(message: Message) -> bool</code>

Parameters

[Expand table](#)

Name	Description
message Required*	Message

Returns

[Expand table](#)

Type	Description
bool	

clone

Python
<pre>clone(*, deep: bool = True) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
deep Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
deep	Default value: True

Returns

[Expand table](#)

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

 Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] None Optional source span IDs from multiple sources for linking. Default value: None

Returns

 Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python
<code>from_dict(data: dict[str, Any]) -> TModel</code>

Parameters

[Expand table](#)

Name	Description
data Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

from_json

Python
<code>from_json(raw: str) -> TModel</code>

Parameters

[Expand table](#)

Name	Description
raw Required*	<code>str</code>

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._workflows._model_utils.TModel></code>	

handle_response

Handle response from parent for a forwarded request.

This handler accumulates responses and only resumes the sub-workflow when all expected responses have been received for that execution.

Python

```
async handle_response(response: SubWorkflowResponseMessage, ctx:
WorkflowContext[Any, Never]) -> None
```

Parameters

[Expand table](#)

Name	Description
response Required*	SubWorkflowResponseMessage The response to a previous request.
ctx Required*	WorkflowContext[Any, Never] The workflow context.

Returns

[Expand table](#)

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python


```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

[Expand table](#)

Name	Description
request_type Required*	<code>type[Any]</code> The type of the request message
response_type Required*	<code>type[Any]</code> The type of the expected response message

Returns

[Expand table](#)

Type	Description
<code>bool</code>	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Restore the WorkflowExecutor state from a checkpoint snapshot.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

[Expand table](#)

Name	Description
state Required*	<code>dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
None	

on_checkpoint_save

Get the current state of the WorkflowExecutor for checkpointing purposes.

Python
<code>async on_checkpoint_save() -> dict[str, Any]</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	

process_workflow

Execute the sub-workflow with raw input data.

This handler starts a new sub-workflow execution. When the sub-workflow needs external information, it pauses and sends a request to the parent.

Python
<code>async process_workflow(input_data: object, ctx: WorkflowContext[Any, Never]) -> None</code>

Parameters

[Expand table](#)

Name	Description
input_data Required*	<code>object</code> The input data to send to the sub-workflow.
ctx Required*	<code>WorkflowContext[Any, Never]</code> The workflow context from the parent.

Returns

 Expand table

Type	Description
<code>None</code>	

to_dict

Python
<code>to_dict() -> dict[str, Any]</code>

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	

to_json

Python
<code>to_json() -> str</code>

Returns

 Expand table

Type	Description
<code>str</code>	

Attributes

input_types

Get the input types based on the underlying workflow's input types plus WorkflowExecutor-specific types.

Returns

[Expand table](#)

Type	Description
	A list of input types that the WorkflowExecutor can accept.

output_types

Get the output types based on the underlying workflow's output types.

Returns

[Expand table](#)

Type	Description
	A list of output types that the underlying workflow can produce. Includes the SubWorkflowRequestMessage type if any executor in the sub-workflow is request-response capable.

workflow_output_types

Get the list of workflow output types that this executor can produce via yield_output().

Returns

[Expand table](#)

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

WorkflowFailedEvent Class

Built-in lifecycle event emitted when a workflow run terminates with an error.

Constructor

Python
<code>WorkflowFailedEvent()</code>

Parameters

 Expand table

Name	Description
details Required*	
data	Default value: None

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code><code>__init__</code>(details: WorkflowErrorDetails, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
details Required*	WorkflowErrorDetails
data	Any None Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

WorkflowOutputEvent Class

Event triggered when a workflow executor yields output.

Initialize the workflow output event.

Constructor

Python
<code>WorkflowOutputEvent()</code>

Parameters

 Expand table

Name	Description
<code>data</code> Required*	The output yielded by the executor.
<code>source_executor_id</code> Required*	ID of the executor that yielded the output.

Methods

 Expand table

<code>__init__</code>	Initialize the workflow output event.
<code>__new__</code>	

`__init__`

Initialize the workflow output event.

Python
<code>__init__(data: Any, source_executor_id: str)</code>

Parameters

 Expand table

Name	Description
data Required*	Any The output yielded by the executor.
source_executor_id Required*	str ID of the executor that yielded the output.

__new__

Python
<code>__new__(**kwargs)</code>

WorkflowRunResult Class

Container for events generated during non-streaming workflow execution.

Overview

Represents the complete execution results of a workflow run, containing all events generated from start to idle state. Workflows produce outputs incrementally through `ctx.yield_output()` calls during execution.

Event Structure

Maintains separation between data-plane and control-plane events:

- Data-plane events: Executor invocations, completions, outputs, and requests (in main list)
- Control-plane events: Status timeline accessible via `status_timeline()` method

Key Methods

- `get_outputs()`: Extract all workflow outputs from the execution
- `get_request_info_events()`: Retrieve external input requests made during execution
- `get_final_state()`: Get the final workflow state (IDLE, IDLE_WITH_PENDING_REQUESTS, etc.)
- `status_timeline()`: Access the complete status event history

Constructor

Python
<code>WorkflowRunResult()</code>

Parameters

[🔗](#) Expand table

Name	Description
<code>events</code> Required*	

Name	Description
status_events	Default value: None

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>append</code>	Append object to the end of the list.
<code>clear</code>	Remove all items from list.
<code>copy</code>	Return a shallow copy of the list.
<code>count</code>	Return number of occurrences of value.
<code>extend</code>	Extend list by appending elements from the iterable.
<code>get_final_state</code>	Return the final run state based on explicit status events. Returns the last WorkflowStatusEvent.state observed. Raises if none were emitted.
<code>get_outputs</code>	Get all outputs from the workflow run result.
<code>get_request_info_events</code>	Get all request info events from the workflow run result.
<code>index</code>	Return first index of value. Raises ValueError if the value is not present.
<code>insert</code>	Insert object before index.
<code>pop</code>	Remove and return item at index (default last). Raises IndexError if list is empty or index is out of range.
<code>remove</code>	Remove first occurrence of value. Raises ValueError if the value is not present.
<code>reverse</code>	Reverse <i>IN PLACE</i> .
<code>sort</code>	Sort the list in ascending order and return None. The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values.

The reverse flag can be set to sort in descending order.

[status_timeline](#)

Return the list of status events emitted during the run (control-plane).

`__init__`

Python

```
__init__(events: list[WorkflowEvent], status_events: list[WorkflowStatusEvent] |  
None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
events Required*	list[WorkflowEvent]
status_events	list[WorkflowStatusEvent] None Default value: None

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python

```
__new__(**kwargs)
```

append

Append object to the end of the list.

Python

```
append(object, /)
```

Positional-Only Parameters

 Expand table

Name	Description
object Required*	

clear

Remove all items from list.

Python

```
clear()
```

copy

Return a shallow copy of the list.

Python

```
copy()
```

count

Return number of occurrences of value.

Python

```
count(value, /)
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

extend

Extend list by appending elements from the iterable.

Python
<code>extend(iterable, /)</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
iterable Required*	

get_final_state

Return the final run state based on explicit status events.

Returns the last WorkflowStatusEvent.state observed. Raises if none were emitted.

Python
<code>get_final_state() -> WorkflowRunState</code>

Returns

[Expand table](#)

Type	Description
WorkflowRunState	

get_outputs

Get all outputs from the workflow run result.

Python

```
get_outputs() -> list[Any]
```

Returns

[Expand table](#)

Type	Description
list[Any]	A list of outputs produced by the workflow during its execution.

get_request_info_events

Get all request info events from the workflow run result.

Python

```
get_request_info_events() -> list[RequestInfoEvent]
```

Returns

[Expand table](#)

Type	Description
list[RequestInfoEvent]	A list of RequestInfoEvent instances found in the workflow run result.

index

Return first index of value.

Raises ValueError if the value is not present.

Python

```
index(value, start=0, stop=9223372036854775807, /)
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	
start	Default value: 0
stop	Default value: 9223372036854775807

insert

Insert object before index.

Python
<code>insert(index, object, /)</code>

Positional-Only Parameters

 Expand table

Name	Description
index Required*	
object Required*	

pop

Remove and return item at index (default last).

Raises IndexError if list is empty or index is out of range.

Python
<code>pop(index=-1, /)</code>

Positional-Only Parameters

 Expand table

Name	Description
<code>index</code>	Default value: -1

remove

Remove first occurrence of value.

Raises `ValueError` if the value is not present.

Python
<code>remove(value, /)</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>value</code> Required*	

reverse

Reverse *IN PLACE*.

Python
<code>reverse()</code>

sort

Sort the list in ascending order and return `None`.

The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values.

The reverse flag can be set to sort in descending order.

Python

```
sort(*, key=None, reverse=False)
```

Keyword-Only Parameters

 Expand table

Name	Description
key	Default value: None
reverse	Default value: False

status_timeline

Return the list of status events emitted during the run (control-plane).

Python

```
status_timeline() -> list[WorkflowStatusEvent]
```

Returns

 Expand table

Type	Description
list[WorkflowStatusEvent]	

WorkflowRunState Enum

Run-level state of a workflow execution.

Semantics:

- **STARTED**: Run has been initiated and the workflow context has been created. This is an initial state before any meaningful work is performed. In this codebase we emit a dedicated *WorkflowStartedEvent* for telemetry, and typically advance the status directly to *IN_PROGRESS*. Consumers may still rely on *STARTED* for state machines that need an explicit pre-work phase.
- **IN_PROGRESS**: The workflow is actively executing (e.g., the initial message has been delivered to the start executor or a superstep is running). This status is emitted at the beginning of a run and can be followed by other statuses as the run progresses.
- **IN_PROGRESS_PENDING_REQUESTS**: Active execution while one or more request-for-information operations are outstanding. New work may still be scheduled while requests are in flight.
- **IDLE**: The workflow is quiescent with no outstanding requests and no more work to do. This is the normal terminal state for workflows that have finished executing, potentially having produced outputs along the way.
- **IDLE_WITH_PENDING_REQUESTS**: The workflow is paused awaiting external input (e.g., emitted a *RequestInfoEvent*). This is a non-terminal state; the workflow can resume when responses are supplied.
- **FAILED**: Terminal state indicating an error surfaced. Accompanied by a *WorkflowFailedEvent* with structured error details.
- **CANCELLED**: Terminal state indicating the run was cancelled by a caller or orchestrator. Not currently emitted by default runner paths but included for integrators/orchestrators that support cancellation.

Fields

 Expand table

CANCELLED
FAILED
IDLE

IDLE_WITH_PENDING_REQUESTS
IN_PROGRESS
IN_PROGRESS_PENDING_REQUESTS
STARTED

WorkflowStartedEvent Class

Built-in lifecycle event emitted when a workflow run begins.

Initialize the workflow event with optional data.

Constructor

Python
<code>WorkflowStartedEvent()</code>

Parameters

[Expand table](#)

Name	Description
data	Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the workflow event with optional data.
<code>__new__</code>	

`__init__`

Initialize the workflow event with optional data.

Python
<code>__init__(data: Any None = None)</code>

Parameters

[Expand table](#)

Name	Description
data	Any None Default value: None

`__new__`

Python
<code>__new__(**kwargs)</code>

WorkflowStatusEvent Class

Built-in lifecycle event emitted for workflow run state transitions.

Initialize the workflow status event with a new state and optional data.

Constructor

Python
<code>WorkflowStatusEvent()</code>

Parameters

[Expand table](#)

Name	Description
<code>state</code> Required*	The new state of the workflow run.
<code>data</code>	Optional additional data associated with the state change. Default value: None

Methods

[Expand table](#)

<code>__init__</code>	Initialize the workflow status event with a new state and optional data.
<code>__new__</code>	

`__init__`

Initialize the workflow status event with a new state and optional data.

Python
<code>__init__(state: WorkflowRunState, data: Any None = None)</code>

Parameters

 Expand table

Name	Description
state Required*	WorkflowRunState The new state of the workflow run.
data	Any None Optional additional data associated with the state change. Default value: None

__new__

Python
<code>__new__(**kwargs)</code>

WorkflowValidationError Class

Base exception for workflow validation errors.

Constructor

Python
<code>WorkflowValidationError()</code>

Parameters

 Expand table

Name	Description
<code>message</code> Required*	
<code>validation_type</code> Required*	

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>add_note</code>	Exception.add_note(note) – add a note to the exception
<code>with_traceback</code>	Exception.with_traceback(tb) – set self.traceback to tb and return self.

`__init__`

Python
<code>__init__(message: str, validation_type: ValidationTypeEnum)</code>

Parameters

[Expand table](#)

Name	Description
message Required*	str
validation_type Required*	ValidationTypeEnum

__new__

Python
<code>__new__(**kwargs)</code>

add_note

`Exception.add_note(note)` – add a note to the exception

Python
<code>add_note()</code>

with_traceback

`Exception.with_traceback(tb)` – set `self.traceback` to `tb` and return `self`.

Python
<code>with_traceback()</code>

Attributes

args

WorkflowViz Class

A class for visualizing workflows using graphviz and Mermaid.

Initialize the WorkflowViz with a workflow.

Constructor

Python
<code>WorkflowViz()</code>

Parameters

[Expand table](#)

Name	Description
<code>workflow</code> Required*	The workflow to visualize.

Methods

[Expand table](#)

<code>__init__</code>	Initialize the WorkflowViz with a workflow.
<code>__new__</code>	
<code>export</code>	Export the workflow visualization to a file or return the file path.
<code>save_pdf</code>	Convenience method to save as PDF.
<code>save_png</code>	Convenience method to save as PNG.
<code>save_svg</code>	Convenience method to save as SVG.
<code>to_digraph</code>	Export the workflow as a DOT format digraph string.
<code>to_mermaid</code>	Export the workflow as a Mermaid flowchart string.

`__init__`

Initialize the WorkflowViz with a workflow.

Python

```
__init__(workflow: Workflow)
```

Parameters

 Expand table

Name	Description
workflow Required*	Workflow The workflow to visualize.

`__new__`

Python

```
__new__(**kwargs)
```

export

Export the workflow visualization to a file or return the file path.

Python

```
export(format: Literal['svg', 'png', 'pdf', 'dot'] = 'svg', filename: str | None = None, include_internal_executors: bool = False) -> str
```

Parameters

 Expand table

Name	Description
format	Literal ['svg', 'png', 'pdf', 'dot'] The output format. Supported formats: 'svg', 'png', 'pdf', 'dot'. Default value: "svg"
filename	str None Optional filename to save the output. If None, creates a temporary file. Default value: None

Name	Description
<code>include_internal_executors</code>	<code>bool</code> Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	The path to the saved file.

Exceptions

[Expand table](#)

Type	Description
<code>ImportError</code>	If graphviz is not installed.
<code>ValueError</code>	If an unsupported format is specified.

save_pdf

Convenience method to save as PDF.

Python
<pre>save_pdf(filename: str, include_internal_executors: bool = False) -> str</pre>

Parameters

[Expand table](#)

Name	Description
<code>filename</code> Required*	<code>str</code> The filename to save the PDF file.
<code>include_internal_executors</code>	<code>bool</code> Whether to include internal executors in the visualization. Default is False.

Name	Description
	Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	The path to the saved PDF file.

save_png

Convenience method to save as PNG.

Python
<code>save_png(filename: str, include_internal_executors: bool = False) -> str</code>

Parameters

[Expand table](#)

Name	Description
filename Required*	<code>str</code> The filename to save the PNG file.
include_internal_executors	<code>bool</code> Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	The path to the saved PNG file.

save_svg

Convenience method to save as SVG.

Python

```
save_svg(filename: str, include_internal_executors: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
filename Required*	str The filename to save the SVG file.
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

[Expand table](#)

Type	Description
str	The path to the saved SVG file.

to_digraph

Export the workflow as a DOT format digraph string.

Python

```
to_digraph(include_internal_executors: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A string representation of the workflow in DOT format.

to_mermaid

Export the workflow as a Mermaid flowchart string.

Python

```
to_mermaid(include_internal_executors: bool = False) -> str
```

Parameters

 Expand table

Name	Description
<code>include_internal_executors</code>	<code>bool</code> Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A string representation of the workflow in Mermaid flowchart syntax.